

L^AT_EX for All

L^AT_EX for All

*From Your First Document to Publication-Ready Scholarly
Writing*

Krishnendu Sinha, Ph.D.

Associate Professor of Zoology
Siksha Bhavana, Visva-Bharati
Santiniketan, West Bengal, India

L^AT_EX for All

From Your First Document to Publication-Ready Scholarly Writing

Author: Krishnendu Sinha, Ph.D.

Copyright © 2026 Krishnendu Sinha, Ph.D.

ISBN: 9798190207026

First edition, independently published by the author through Amazon Kindle Direct Publishing (KDP), 2026.

Except where otherwise noted, the original text, L^AT_EX source, diagrams, and teaching data in this work are licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0).

Code examples are additionally available under the MIT License. Cover artwork and third-party material are excluded from these licences unless explicitly stated otherwise.

The software names and trademarks mentioned in this book belong to their respective owners.

To my family.

Preface

Many people first meet $\text{\LaTeX}$ at an awkward moment. A thesis has grown too large for comfortable manual formatting. An equation has become difficult to edit. A journal has supplied a template full of unfamiliar commands. The learner is expected to produce a polished document while also trying to understand a new way of writing.

This book begins earlier and moves more gently. It assumes that you may never have seen a source file, a command, or a compiler. In the first lessons we use the smallest document that $\text{\LaTeX}$ can compile. We then add one useful idea at a time, inspect the result, and return to improve the same scholarly project. The aim is not to memorise hundreds of commands. It is to understand enough structure to make sound decisions, find trustworthy documentation, and repair problems calmly.

Our continuing project is an article called “A Reproducible Comparison of Two Measurement Methods”. Its values are simulated demonstration data, not real findings. That neutral example lets us practise the architecture of a scientific article without pretending to conduct a study. Shorter examples from biology, chemistry, physics, mathematics, earth science, computing, engineering, health, and interdisciplinary work show how the same principles travel across fields.

You will see errors in these pages. Some are deliberate. A missing brace or a misspelt command is not evidence that you are unsuited to $\text{\LaTeX}$; it is a clue about what the compiler could not understand. We shall learn to read the first useful clue, examine the nearby source, and test a small correction.

Three professional outcomes guide the journey: a scholarly manuscript, a thesis or dissertation, and a technical or scientific report. Each requires more than attractive pages. It must also have maintainable source, reliable references, accessible figures and tables, clear metadata, and a reproducible build. By the end, you should be able to create those documents, explain their structure, and prepare clean files for another person to compile.

Software changes, and publisher or institutional instructions differ. This book therefore distinguishes lasting $\text{\LaTeX}$ ideas from interface details and local rules. When a current instruction controls a submission, follow that instruction. When you are free to choose, prefer a simple, semantic, and portable solution.

Who this book is for

This book is for a complete beginner. You do not need experience with programming, markup, command-line tools, or publishing. If words such as *preamble*, *package*, and *compiler* are new, you are in the right place: each will be introduced before it is required.

The examples are intended for students, teachers, researchers, laboratory and field workers, data analysts, engineers, and technical professionals. They draw from the biological, physical, chemical, mathematical, earth and environmental, computer and data, engineering, technological, health, and interdisciplinary sciences. You may skip examples outside your field without losing the main argument.

Readers who already use $\text{\LaTeX}$ can use the later chapters as a structured course in maintainable scholarly documents. Optional sections marked *Advanced Topic* can be left until a project makes them useful.

What you will be able to do

By the end of the book, you should be able to:

- explain how source files, document classes, packages, compilers, and PDF output work together;
- create and organise a portable multi-file $\text{\LaTeX}$ project;
- write clear prose, mathematics, quantities, tables, figures, cross-references, citations, and bibliographies;
- diagnose common errors by reading the log and reducing a problem to a minimal working example;
- build a publication-ready generic scholarly manuscript;
- adapt a complete thesis or dissertation without inventing or ignoring institutional requirements;
- prepare a professional technical or scientific report; and
- proof, archive, reproduce, and package source for submission.

These goals describe independent practice. You should not merely recognise a command when you see it; you should be able to decide whether it belongs in your document and test that it behaves as intended.

How to use this book

Work at a computer and compile often. Reading a code sample is useful, but changing it and examining the new PDF is where understanding becomes practical. Keep a separate practice project so that experiments cannot damage important work.

For a faster, practical route, compile the first complete code panel in each chapter before reading every explanation. Make the changes named in the Code Lab, compare the PDFs, and return to the surrounding text when the result surprises you. This keeps the source file open while you learn.

Choose a working path

The beginner-friendly path uses Overleaf in a web browser. It avoids an installation during the first lesson and places source, compiler messages, and PDF preview in one project. The local-computer path uses TeX Live on macOS or Linux, or MiKTeX/TeX Live on Windows. TeXstudio and Visual Studio Code with L^AT_EX Workshop are optional editors; `latexmk` is the preferred automated build tool. The L^AT_EX source is the same on both paths.

Start with whichever path lowers the immediate barrier. Later, try the other path so that you understand how to keep and compile your own complete source.

Use the spiral project

At the end of each chapter, improve the continuing measurement-methods article. Do not jump to the final version at once. The value lies in seeing why each layer is added and which problem it solves.

Make errors on purpose

Keep a working copy, make the requested error, compile it, and read the first meaningful message. Then repair the source. A controlled error is a safe way to learn what a message means before it appears in a deadline-driven project.

Follow external instructions when they govern

A journal class, thesis manual, funder, or client may impose rules that differ from the generic examples. Verify the current instructions and change only what they require. The book explains the reasoning needed to make those changes without dismantling the document's structure.

What this book covers—and what it does not

This book teaches the complete working path from a first source file to a carefully checked article, thesis, or technical report. Its primary scope is practical document preparation: structured prose, mathematics and units, tables, figures, automatic references, bibliographies, multi-file projects, error diagnosis, reusable document organisation, and final PDF checks. Every chapter now ends with a complete practical coding section that can be compiled before the longer explanation is revisited.

The book does not attempt advanced macro or class programming, TikZ drawing, Beamer presentation design, multilingual publishing systems, or the specialist production workflow of every publisher. Those are substantial subjects in their own right. The generic patterns here prepare readers to understand and adapt a verified local, institutional, journal, or publisher specification without claiming to replace it.

The companion repository is named `sinhakrishnendu/LaTeX_for_all`. It is optional: every essential example and instruction needed to follow the book is printed in these pages.

Opening-page design

Part-title pages always open on a right-hand (recto) page. Individual chapters begin on the next available page. This is a deliberate economical-paperback choice: it preserves strong part divisions without adding a blank leaf before every chapter.

Conventions used in code and examples

Code is printed in a monospaced typeface. A command appears as `\section`; a package as `graphicx`; and a file as `main.tex`. Required material is written in braces, as in `{article}`. Optional material is written in brackets, as in `[11pt]`. A descriptive item that you must replace appears as `<your title>`.

Complete code panels can be copied into a plain-text source file. Their quotation marks and commands use straight ASCII characters. A code panel that is intentionally invalid is labelled *Broken example: do not copy as a working document*. The corresponding repair follows nearby.

The recurring boxes have consistent jobs:

Classroom Note

clarifies an idea without interrupting the main path.

Try It Now

asks for an immediate, small action.

Pause and Predict

asks you to decide what will happen before compiling.

Common Error

shows a realistic failure, cause, and repair.

Good Practice

distinguishes maintainable professional habits.

Science in Practice

connects a command to scholarly work.

Advanced Topic

marks material that a beginner may safely postpone.

Submission Check

turns a requirement into an auditable check.

Mini Project

develops a cumulative document.

Values in the continuing article are labelled as simulated demonstration data. The examples do not claim genuine scientific results. Web interfaces may change; descriptive instructions identify the task rather than depending on an exact button position.

Acknowledgements

I gratefully acknowledge Visva-Bharati, my workplace, for the academic environment in which much of my teaching and scholarly work has developed. Its traditions of learning, inquiry, and interdisciplinary exchange have helped shape the educational spirit of this book. I also value the many conversations with colleagues and students that have continually sharpened my understanding of how technical ideas can be taught clearly.

I gratefully acknowledge the open-source movement and the international L^AT_EX community. Their freely shared software, documentation, examples, bug reports, and patient assistance have made reliable scholarly typesetting available to learners and researchers everywhere. This book is possible because generations of contributors have treated knowledge and tools as things to improve together and share responsibly.

I gratefully acknowledge ChatGPT, developed by OpenAI, for substantial assistance in preparing this edition. It contributed to developmental editing, the practical restructuring of the chapters, technical rewriting, L^AT_EX engineering, the preparation and review of code examples, compilation and consistency checks, and the inspection of figures and page layout. This support helped make the book clearer, more practical, and more reliable. I directed the project, supplied the subject expertise and publishing decisions, evaluated and revised the suggestions, and extensively cross-checked the final text and code. Final authorship, judgement, and responsibility for the published work remain mine.

Contents

Preface	i
Who this book is for	ii
What you will be able to do	iii
How to use this book	iv
What this book covers—and what it does not	v
Conventions used in code and examples	vi
Acknowledgements	vii
I Meeting L^AT_EX	1
0 Hello, L^AT_EX!	2
0.1 A document preparation system	2
0.2 Two files with different jobs	3
0.3 A first look at source	4
0.4 Class, packages, and content	4
0.5 How this differs from a word processor	5
0.6 Where L ^A T _E X is especially useful	5
0.7 Realistic limitations	6
0.8 What happens when compilation goes wrong	6
0.9 The journey ahead	7
0.10 Code lab: source in, PDF out	7
0.11 Chapter project	8
0.12 Summary	8
0.13 Checklist	8
0.14 Exercises	8
0.15 Complete Practical Coding	9
1 Your First L^AT_EX Document	10
1.1 Choose a workspace	10
1.1.1 The online path: Overleaf	10
1.1.2 The local-computer path	11
1.2 The smallest useful document	12
1.3 What compilation does	13
1.3.1 A first local build with latexmk	13

1.4	Compile, inspect, revise	14
1.5	Your first controlled error	14
1.6	Saving and keeping both forms	15
1.7	The first complete working pattern	15
1.8	Chapter project	16
1.9	Summary	16
1.10	Checklist	16
1.11	Exercises	17
1.12	Complete Practical Coding	17
2	Understanding the Source	19
2.1	Commands: instructions beginning with a backslash	19
2.2	Required arguments use braces	20
2.3	Optional arguments use brackets	20
2.4	Environments have a beginning and an end	21
2.5	Comments explain source to people	22
2.6	Whitespace: spaces, lines, and paragraphs	22
2.7	Characters with special jobs	23
2.8	The preamble and the body	24
2.9	A minimal working example	25
2.10	The first-aid debugging routine	26
2.11	Reading source from left to right	27
2.12	Chapter project	27
2.13	Summary	28
2.14	Checklist	28
2.15	Exercises	29
2.16	Complete Practical Coding	29
II	Writing and Organising Text	31
3	Writing Clear Scientific Prose	32
3.1	Paragraphs carry ideas	32
3.2	Emphasis and type styles	33
3.3	Quotation marks, hyphens, and dashes	33
3.4	Spaces that should stay together	34
3.5	Footnotes and web addresses	34
3.6	Unicode without mystery	35
3.7	Chapter project	36
3.8	Summary	36
3.9	Checklist	36
3.10	Exercises	36
3.11	Complete Practical Coding	37
4	Building Scholarly Structure	38

4.1	Metadata before appearance	38
4.2	The abstract	39
4.3	Sections describe the argument	39
4.4	A table of contents	40
4.5	Appendices	41
4.6	A complete structural skeleton	41
4.7	Chapter project	42
4.8	Summary	42
4.9	Checklist	43
4.10	Exercises	43
4.11	Complete Practical Coding	43
5	Lists, Quotations, and Notes	45
5.1	Unordered lists with itemize	45
5.2	Ordered lists with enumerate	45
5.3	Description lists	46
5.4	Nested lists	46
5.5	Short and long quotations	47
5.6	Footnotes and optional margin notes	48
5.7	Careful customisation with enumitem	48
5.8	Chapter project	49
5.9	Summary	49
5.10	Checklist	49
5.11	Exercises	49
5.12	Complete Practical Coding	50
III	Mathematics and Scientific Notation	51
6	Writing Mathematics from the Beginning	52
6.1	Inline mathematics belongs in a sentence	52
6.2	Displayed mathematics	53
6.3	Scripts need groups	53
6.4	Fractions, roots, and Greek letters	53
6.5	Functions and operators	54
6.6	Brackets that fit	55
6.7	Numbering and referencing an equation	55
6.8	Aligned equations	56
6.9	Chapter project	56
6.10	Summary	56
6.11	Checklist	57
6.12	Exercises	57
6.13	Complete Practical Coding	57
7	Advanced Mathematical Writing	59

7.1	Break mathematics at relationships	59
7.2	Cases and matrices	60
7.3	Calculus and large operators	60
7.4	Definitions, theorems, and proofs	61
7.5	Numbering by section	62
7.6	Code lab: one mathematical page	62
7.7	Chapter project	63
7.8	Summary	63
7.9	Checklist	63
7.10	Exercises	63
7.11	Complete Practical Coding	64
8	Quantities, Units, and Scientific Conventions	66
8.1	Load siunitx and separate roles	66
8.2	Ranges and lists	67
8.3	Uncertainty and significant figures	67
8.4	Percentages, angles, and degrees	68
8.5	Decimal alignment in tables	68
8.6	Optional discipline-specific conventions	68
	8.6.1 Chemical formulae and reactions	68
	8.6.2 Species names and sequences	69
	8.6.3 Physical constants, vectors, and matrices	69
8.7	Chapter project	69
8.8	Summary	70
8.9	Checklist	70
8.10	Exercises	70
8.11	Complete Practical Coding	70
IV	Tables, Figures, and Cross-References	73
9	Creating Professional Tables	74
9.1	The smallest tabular	74
9.2	Rules that organise rather than cage	75
9.3	The table float, caption, and label	75
9.4	Flexible text columns	75
9.5	A statistical table	76
9.6	Tables across pages	76
9.7	Optional: data from CSV	77
9.8	Code lab: turn observations into a table	78
9.9	Chapter project	78
9.10	Summary	78
9.11	Checklist	79
9.12	Exercises	79
9.13	Complete Practical Coding	79

10 Adding and Managing Figures	81
10.1 Prepare the graphic before placing it	81
10.2 The smallest included graphic	81
10.3 Caption, label, and float	82
10.4 Why a figure moves	83
10.5 Multi-panel figures	83
10.6 Landscape and rotated material	84
10.7 Quality and accessibility checks	84
10.8 Code lab: a figure that still compiles during drafting	85
10.9 Chapter project	85
10.10Summary	85
10.11Checklist	86
10.12Exercises	86
10.13Complete Practical Coding	86
11 Labels, References, and Automatic Numbering	88
11.1 A label is a permanent source name	88
11.2 Basic references	89
11.3 Readable references with cleveref	89
11.4 Links and metadata	89
11.5 Why references need repeated compilation	90
11.6 A cross-reference audit	90
11.7 Code lab: rename without renumbering	91
11.8 Chapter project	92
11.9 Summary	92
11.10Checklist	92
11.11Exercises	92
11.12Complete Practical Coding	93
V Citations and Scholarly Sources	95
12 Citations and Bibliographies	96
12.1 Concepts before commands	96
12.2 One verified record	96
12.3 Connect the database with biblatex	97
12.4 Useful citation forms	98
12.5 Records for scholarly source types	98
12.6 Common bibliography failures	99
12.7 Publisher workflows: BibTeX and natbib	99
12.8 Scholarly responsibility	99
12.9 Code lab: build a bibliography from two files	100
12.10Chapter project	101
12.11Summary	101
12.12Checklist	101

12.13Exercises	101
12.14Complete Practical Coding	102
VI Controlling a Professional Document	105
13 Document Classes, Packages, and Page Design	106
13.1 Choose the broad architecture	106
13.2 Class options	107
13.3 Packages solve named problems	107
13.4 Read documentation at the point of need	108
13.5 Page size and margins	108
13.6 Line spacing and paragraphs	108
13.7 Headers, footers, and page numbering	109
13.8 Title pages and special orientations	109
13.9 Microtypography	110
13.10Chapter project	110
13.11Summary	110
13.12Checklist	110
13.13Exercises	111
13.14Complete Practical Coding	111
14 Long Documents and Multi-File Projects	113
14.1 One main document	113
14.2 input and include	114
14.3 A durable folder structure	114
14.4 Front, main, and back matter	115
14.5 Appendices and supplementary material	115
14.6 Lists of figures and tables	115
14.7 Nomenclature, acronyms, and glossary	115
14.8 An index	116
14.9 Shared commands	116
14.10Cross-file references	116
14.11Portability audit	117
14.12Chapter project	117
14.13Summary	117
14.14Checklist	118
14.15Exercises	118
14.16Complete Practical Coding	118
VII Major Scientific Document Forms	121
15 Building a Thesis or Dissertation	122
15.1 Requirements come before styling	122

15.2	Tour the supplied project	123
15.3	Centralise identity and title metadata	123
15.4	Design the main driver	124
15.5	Build front matter deliberately	125
15.6	Lists, abbreviations, and symbols	125
15.7	Shape a chapter as an argument	125
15.8	Published and joint work	126
15.9	Figures, tables, and generated evidence	126
15.10	Appendices and supplements	126
15.11	Selective drafting and full builds	127
15.12	Institutional compliance matrix	127
15.13	Code lab: assemble a thesis from small files	127
15.14	Chapter project	128
15.15	Summary	129
15.16	Checklist	129
15.17	Exercises	129
15.18	Complete Practical Coding	129
16	Preparing a Journal Manuscript	132
16.1	Start with the target journal	132
16.2	Tour the independent article	132
16.3	Title and short title	133
16.4	Authors, affiliations, and correspondence	133
16.5	A safe anonymity toggle	134
16.6	Write a structured abstract	134
16.7	IMRaD as a claim structure	134
16.8	Results that agree across forms	135
16.9	Declarations are part of the scholarship	135
16.10	Publisher classes and package restraint	136
16.11	Line numbers and review spacing	136
16.12	Supplementary files	136
16.13	A clean submission package	137
16.14	Code lab: switch between review and identified copies	137
16.15	Chapter project	138
16.16	Summary	138
16.17	Checklist	138
16.18	Exercises	139
16.19	Complete Practical Coding	139
17	Writing a Technical or Project Report	141
17.1	Begin with the decision	141
17.2	Tour the report project	141
17.3	The cover is controlled metadata	142
17.4	Document control and distribution	142
17.5	Write the executive summary last	143

17.6 Purpose, scope, and assumptions	143
17.7 Methods and assurance	143
17.8 Findings are not recommendations	144
17.9 Use tables and figures for decisions	144
17.10 Risk and uncertainty	145
17.11 Recommendations must be executable	145
17.12 Limitations and non-conformances	145
17.13 Appendices and evidence registers	145
17.14 Release and archival workflow	146
17.15 Code lab: build a decision-facing report	146
17.16 Chapter project	147
17.17 Summary	147
17.18 Checklist	147
17.19 Exercises	148
17.20 Complete Practical Coding	148

VIII Working Efficiently and Safely 151

18 Collaboration and Reproducibility 152

18.1 Reproducible means reconstructable	152
18.2 Names are navigation	152
18.3 Comments explain reasons	153
18.4 Custom commands define a shared language	154
18.5 Four ways to manage revision	154
18.5.1 Numbered snapshots	154
18.5.2 Cloud L ^A T _E X collaboration	154
18.5.3 Visible change marking	154
18.5.4 Git and hosted repositories	154
18.6 Resolve collaboration at the level of ownership	155
18.7 README as the first reproducibility test	155
18.8 Pin what affects the build	156
18.9 Avoid absolute paths	156
18.10 Backups need independent copies	157
18.11 Licence and permission awareness	157
18.12 The clean-room rehearsal	157
18.13 Chapter project	158
18.14 Summary	158
18.15 Checklist	158
18.16 Exercises	158
18.17 Complete Practical Coding	159

19 Troubleshooting L^AT_EX Systematically 161

19.1 The eight-step debugging loop	161
19.2 Preserve the evidence	162

19.3	Read an error record	162
19.4	Minimal working examples	162
19.5	Binary isolation	163
19.6	L ^A T _E X First-Aid Clinic	163
19.6.1	Undefined control sequence	163
19.6.2	Missing closing brace	164
19.6.3	Mismatched environment	164
19.6.4	Math delimiter not closed	164
19.6.5	File not found	165
19.6.6	Missing package or class	165
19.6.7	Unicode character error	165
19.6.8	Undefined reference	165
19.6.9	Undefined citation or empty bibliography	165
19.6.10	Package option clash	166
19.7	Errors, warnings, and visual problems	166
19.7.1	Overfull boxes	166
19.7.2	Underfull boxes	166
19.7.3	Float placement	166
19.8	When auxiliary files are stale	167
19.9	Compile in the correct sequence	167
19.10	Package documentation and bug reports	167
19.11	Chapter project	168
19.12	Summary	168
19.13	Checklist	168
19.14	Exercises	168
19.15	Complete Practical Coding	169
20	From Source Files to Submission	171
20.1	Define the release candidate	171
20.2	Build a compliance matrix	172
20.3	Proofread at three levels	172
20.3.1	Scientific and logical review	172
20.3.2	Source review	172
20.3.3	Rendered-PDF review	172
20.4	Cross-references and citations	173
20.5	Equations and scientific notation	173
20.6	Tables	173
20.7	Figures and image preflight	174
20.8	Metadata	174
20.9	Fonts, page properties, and PDF features	174
20.10	Accessibility	175
20.11	Remove draft apparatus	175
20.12	Supplementary files	175
20.13	Create a clean submission folder	175

20.14	Inspect the final PDF	176
20.15	Upload rehearsal	176
20.16	Archive both release and working project	177
20.17	Code lab: make a clean release candidate	177
20.18	Chapter project	178
20.19	Summary	178
20.20	Checklist	178
20.21	Exercises	179
20.22	Complete Practical Coding	179

IX Capstone Projects 181

21 Three Complete Projects 182

21.1	How to use the capstones	182
21.2	Project 1: A publication-ready scholarly article	183
21.2.1	The task	183
21.2.2	Planning notes	183
21.2.3	Expected folder structure	183
21.2.4	Complete source and important commands	184
21.2.5	Compilation instructions	184
21.2.6	Final article checklist	184
21.2.7	Extension exercises	185
21.3	Project 2: A complete thesis or dissertation	185
21.3.1	The task	185
21.3.2	Planning notes	185
21.3.3	Expected folder structure	185
21.3.4	Complete source and important commands	186
21.3.5	Compilation instructions	186
21.3.6	Final thesis checklist	186
21.3.7	Extension exercises	187
21.4	Project 3: A professional technical report	187
21.4.1	The task	187
21.4.2	Planning notes	187
21.4.3	Expected folder structure	187
21.4.4	Complete source and important commands	188
21.4.5	Compilation instructions	188
21.4.6	Final report checklist	188
21.4.7	Extension exercises	188
21.5	Compare the same evidence across forms	189
21.6	Compile every example from one control point	189
21.7	Code lab: compile all three capstones	190
21.8	Chapter project	190
21.9	Summary	190
21.10	Checklist	191

21.11Exercises	191
21.12Complete Practical Coding	191
Command quick-reference guide	194
Common symbols	200
Common error-message guide	204
Package guide	209
Glossary	215
Concise exercise solutions	223
Further reading and verified documentation	234
About the author	237

Part I

Meeting L^AT_EX

Hello, L^AT_EX!

Imagine that you are preparing a report with three equations, a table, and a figure. You insert a new paragraph near the beginning. The figure moves, a heading remains at the bottom of a page, and every later figure number must be checked. The scientific argument has not changed, but the page has demanded your attention.

L^AT_EX offers a different division of labour. You describe the document's content and structure in a plain-text source file. A program interprets those instructions and builds the typeset document. You still make design decisions, but you make them as rules and relationships rather than repeated adjustments to individual pages.

Learning outcomes

- Explain what L^AT_EX is and what it is not.
- Distinguish a source file from a compiled PDF.
- Describe the job of a compiler, document class, and package.
- Compare a structured L^AT_EX workflow with a word-processor workflow.
- Identify situations in which L^AT_EX is useful and situations in which another tool may be more practical.
- Describe the learning route from a minimal document to a scholarly manuscript, thesis, and report.

0.1 A document preparation system

L^AT_EX is a *document preparation system*: a collection of rules and programs for turning structured instructions into typeset output. It is widely used for scientific and technical communication, particularly when a document contains mathematics, references, numbered elements, or many interdependent parts. The L^AT_EX Project describes it as a system for preparing documents and provides the core documentation maintained with the software (The L^AT_EX Project 2026).

L^AT_EX is not simply a font menu or a single visual editor. The familiar part of the work is writing sentences. The unfamiliar part is that some plain-text instructions tell L^AT_EX what those sentences mean. For example, a heading is identified as a heading rather than made larger by hand. A figure is given a caption and a permanent label rather than assigned a typed number.

Classroom Note

The name is pronounced in several ways. You may hear something close to “lay-tech” or “lah-tech”. Pronunciation does not affect the work. The capitalisation is written $\text{\LaTeX}$ in ordinary prose.

0.2 Two files with different jobs

A beginning $\text{\LaTeX}$ project normally has at least two forms of the document:

The source file

A plain-text file such as `main.tex`. It contains your words and $\text{\LaTeX}$ instructions. You edit this file.

The output file

A typeset file such as `main.pdf`. It contains the pages you read, print, share, or submit. You inspect this file, but you normally do not use a PDF editor to change its content.

The source is a little like a recipe: it states the ingredients and the operations. The PDF is the prepared result. The analogy has limits—a PDF can be built repeatedly from the same source without consuming anything—but it helps us keep the two files’ roles separate.

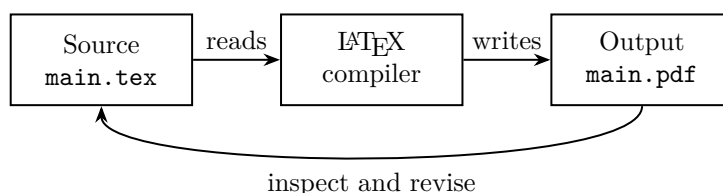


Figure 1. The basic $\text{\LaTeX}$ cycle. We edit the source, compile it, inspect the PDF, and revise the source.

The program that reads the source and creates output is a *compiler*. The act of running it is *compilation*. As Figure 1 shows, compilation is not a final ceremony. It is part of a short cycle: write a small amount, compile, inspect, and improve.

Pause and Predict

Suppose you find a misspelt word in the PDF. Which file should you edit: the PDF or `main.tex`? Predict what must happen after the edit before reading on.

You edit `main.tex`, compile again, and inspect the newly generated PDF.

0.3 A first look at source

The next chapter will build and explain the smallest useful document. For now, look at its shape:

Preview: a minimal L ^A T _E X document
<pre>\documentclass{article} \begin{document} Hello, \LaTeX{}! \end{document}</pre>

The lines beginning with a backslash are *commands*: instructions for L^AT_EX. The word in braces after `\documentclass` selects a broad type of document. Here, `article` supplies general rules suitable for a short scholarly document. The paired `\begin{document}` and `\end{document}` lines mark the region whose contents should appear on the page. The ordinary sentence between them becomes ordinary typeset text.

What you should see

After successful compilation, the PDF contains a single page with the words “Hello, L^AT_EX!” near the upper-left area of the text block. It does not show the commands. They guided the construction of the page.

Try It Now
Without compiling, point to the line that selects the document type, the line that starts the visible content, the visible sentence, and the line that ends the visible content. This simple source-reading habit will make later error messages easier to investigate.

0.4 Class, packages, and content

Three layers will recur throughout the book.

1. A *document class* supplies the broad architecture. The standard classes `article`, `report`, and `book` make different structural assumptions.
2. A *package* is a specialised toolbox added for a purpose. A package might manage units, improve tables, or include graphics.
3. Your content includes the prose, data, equations, figures, citations, and document structure that communicate your work.

We will not load a large toolbox before we need it. The first working document uses only a class and a body. Later, each package will arrive with a problem it solves, a smallest example, and an explanation.

Science in Practice

A physicist may use the same `article` class for equations, a zoologist for a species description, and a computer scientist for an algorithm. Their content differs, but the structural ideas—title, sections, figures, references, and bibliography—remain recognisable. Discipline-specific tools can be added without rebuilding the document from nothing.

0.5 How this differs from a word processor

The difference is not that one tool has buttons and the other has none. Many $\text{\LaTeX}$ editors provide buttons, menus, and PDF previews. The deeper difference is the working model.

Table 1. Two common approaches to preparing a document. The descriptions are tendencies, not absolute rules.

Question	Visual word-processor approach	Structured $\text{\LaTeX}$ approach
What do you edit?	A visual representation of the pages	Plain-text source that describes content and structure
How is a heading made?	Often by choosing visual properties or a style	By identifying text as a section or subsection
How are numbers updated?	Automatically when features are used consistently; manually when typed	From labels and references during compilation
Where is broad design stored?	In document styles and direct formatting	Mainly in the class, packages, and preamble
What is shared for editing?	Usually one document file	Source files, bibliography, figures, and build instructions

A well-structured word-processor document can also use styles, automatic captions, and references. A poorly structured $\text{\LaTeX}$ file can also contain fragile manual adjustments. The benefit comes from the method, not from the file extension alone.

0.6 Where $\text{\LaTeX}$ is especially useful

$\text{\LaTeX}$ deserves serious consideration when a document has several of the following features:

- mathematical notation or many numbered equations;
- a large bibliography with a defined citation style;
- figures, tables, sections, and appendices that refer to one another;
- repeated structures across many chapters;

- a journal, conference, or institution that supplies a L^AT_EX class;
- source that must remain inspectable, portable, and suitable for version control;
- output that must be rebuilt reliably after revisions.

Those strengths help with articles, theses, technical reports, books, lecture notes, and documentation. They do not mean that L^AT_EX is automatically the best tool for every page.

0.7 Realistic limitations

There is an initial learning cost. A missing character can stop compilation, and an error message may look forbidding before you know which line matters. Highly visual free-form layouts can take more work than they do in a design application. Collaborators who do not use L^AT_EX may need a different review workflow. Accessibility remediation and publisher conversion still require care; attractive PDF pages alone do not guarantee an accessible document.

Some tasks are simply quicker elsewhere. A one-page event poster may suit a graphics editor. A form that colleagues must edit visually may suit a word processor. A data graphic is usually created in statistical or illustration software and then included in L^AT_EX. Good scholarly practice means choosing a tool for the work, not defending one tool for every purpose.

Good Practice

Before adopting a journal or thesis template, verify its current official instructions. Do not replace its class, bibliography system, margins, or font choices merely because a personal setup looks more familiar.

0.8 What happens when compilation goes wrong

The compiler reads literal instructions. If an instruction is incomplete, it cannot reliably guess what you intended. It writes a *log*: a technical record of the build, including errors and warnings. An editor may display a shorter explanation beside the source.

Common Error

Mistake: treating the first red message as a judgement on the whole document.

What it means: the compiler has reported a specific point where its reading failed or became uncertain. Later messages may be consequences of the first problem.

First response: save the source, read the first meaningful error, note the reported line, and inspect that line and the few lines before it. We will practise this routine throughout the book.

Warnings do not always stop a build. A warning may report a line that is too wide or a reference that needs another pass. Errors and warnings therefore need interpretation, not panic or automatic deletion.

0.9 The journey ahead

Our sequence is cumulative. First we make one page. Then we understand the source, organise prose, write mathematics and quantities, build tables and figures, connect references, and manage a bibliography. Only after those pieces are familiar do we control large projects and complete the three major document types.

The end point is independence: you should be able to decide what the document needs, build the smallest useful version, inspect it, locate reliable package documentation, and prepare a clean submission without mysterious copied preamble fragments.

0.10 Code lab: source in, PDF out

Code Lab

Create a new project containing only the file below. Compile it once, change the title, add a second paragraph, and compile again. Your useful evidence is the changed PDF—not whether you memorised the commands.

A complete first experiment

```
\documentclass{article}

\title{My first \LaTeX{} experiment}
\author{Your Name}

\begin{document}
\maketitle

This sentence came from a source file.

This is a second paragraph.
\end{document}
```

Now delete the closing brace after `\title{`, compile, and read the first error. Restore the brace immediately. You have completed the full working cycle: edit, compile, inspect, diagnose, repair.

0.11 Chapter project

Mini Project

Create a folder named `measurement-methods-article`. Do not add a complex template. In a plain-text note, write the following three statements:

1. The project will compare Method A and Method B using simulated demonstration data.
2. The values do not represent a real experiment or clinical study.
3. The eventual document will contain an abstract, sections, equations, a table, figures, references, an appendix, and submission checks.

This note defines the scholarly task before we choose formatting. In the next chapter, it becomes your first compilable source file.

0.12 Summary

L^AT_EX turns structured plain-text source into typeset output. The source and PDF have different jobs, and a compiler connects them. A document class provides broad architecture; packages add specialised tools. L^AT_EX is strong when a scholarly document has many structured, numbered, or reusable parts, but it has a learning cost and is not the best tool for every visual task. Our working habit will be to edit, compile, inspect, and revise in small steps.

0.13 Checklist

- ☐ I can identify the source file and the output PDF.
- ☐ I can explain compilation without using unexplained jargon.
- ☐ I can state the different roles of a class and a package.
- ☐ I can give one reason to choose L^AT_EX and one reason to choose another tool.
- ☐ I have created the planning folder and labelled the future data as simulated.

0.14 Exercises

- 0.1 In one sentence each, define *source file*, *compiler*, and *output file*.
- 0.2 A colleague changes a number directly in a PDF and expects the L^AT_EX source to update. Explain the broken link in this workflow.
- 0.3 Read the preview source in this chapter. Predict which line would be visible as ordinary text in the PDF.
- 0.4 Name two document features that make a structured workflow valuable.

- 0.5 Classify each item as class, package, content, or output: a results paragraph; book; a bibliography toolbox; `report.pdf`.
- 0.6 A collaborator needs a freely positioned one-page infographic and will never edit source. Give a reason not to choose $\text{\LaTeX}$ automatically.
- 0.7 Challenge: describe a document from your field that you would like to build by the end of this book. List its recurring structures.

0.15 Complete Practical Coding

Copy this file, compile it, and compare the source with the PDF.

Complete practical file: source becomes a PDF

```
\documentclass{article}

\title{My First \LaTeX{} Project}
\author{Your Name}
\date{\today}

\begin{document}
\maketitle

\section{Question}
What happens when this source is compiled?

\section{Answer}
The document class supplies the design, while the body supplies
the content and structure.

\end{document}
```

Code Lab

Change the title, add one section, and deliberately misspell `\section`. Compile after each change, then repair the error.

Your First L^AT_EX Document

The best first document is not a template with forty lines of unexplained settings. It is a page that you understand completely. We shall create that page now. Choose the online path if you want to begin without installing software, or the local path if a TeX system is already available. Both paths lead to the same `main.tex` source and the same essential PDF.

Learning outcomes

- Create a new L^AT_EX project online or on a local computer.
- Identify the project's main source file.
- Write the smallest useful document with the `article` class.
- Compile the source and inspect the PDF preview.
- Explain each line of the minimal document.
- Locate the log when compilation fails.
- Save or download both source and PDF output.

1.1 Choose a workspace

A *L^AT_EX project* is the group of files needed to build a document. Our first project needs only one source file. Later projects will also contain figures, bibliography data, and separate section files.

1.1.1 The online path: Overleaf

Overleaf runs the L^AT_EX tools on a remote service and displays an editor and PDF preview in a web browser. It is convenient for the first lesson because there is no local installation to diagnose. The names and positions of interface controls may change, so treat the steps below as tasks rather than a permanent map of the screen. The current official documentation explains project creation, recompilation, main documents, and downloads (Overleaf 2026d; Overleaf 2026b; Overleaf 2026c; Overleaf 2026a).

1. Sign in to Overleaf and create a new blank project.
2. Give it a useful name, such as `first-latex-document`.
3. In the file list, select `main.tex`. Delete any sample content so that the file is empty.

4. Type the source shown in Section 1.2 exactly.
5. Run the project's Recompile action.
6. Inspect the PDF preview and, if necessary, open the log or error panel.

Overleaf treats one `.tex` file as the *main document*: the file sent to the compiler first. In a one-file project, `main.tex` is the natural choice. In a multi-file project, a chapter file is not normally the main document because it may depend on settings stored elsewhere.

Classroom Note

Overleaf normally saves edits automatically, but a remotely saved project is not a substitute for an independent backup. Download the source as a ZIP file at sensible milestones and download the PDF separately. Current Overleaf documentation notes that the ordinary source download and PDF download are separate actions (Overleaf 2026a).

1.1.2 The local-computer path

Local work requires a *TeX distribution*: a maintained collection of the compiler, standard classes, packages, fonts, and utilities. Common choices are TeX Live on macOS and Linux, and either MiKTeX or TeX Live on Windows. macOS users often install MacTeX, which is a TeX Live distribution packaged for the platform. Installation details change, so use the current official installer for your operating system rather than an old third-party tutorial.

You also need a plain-text editor. TeXstudio is a focused L^AT_EX editor. Visual Studio Code can become a L^AT_EX editor through the L^AT_EX Workshop extension. Neither editor changes the language in `main.tex`; it changes how you edit, compile, and view it.

After installing a distribution and editor:

1. Create a folder named `first-latex-document`.
2. In that folder, create a plain-text file named `main.tex`.
3. Type the minimal source in Section 1.2.
4. Save the file before compiling.
5. Use the editor's build action, or run `latexmk` as described in Section 1.3.1.

Do not write the source in an application that silently saves a word-processing format. The file must be plain text and its name must end in `.tex`, not `.tex.txt`.

Good Practice

Use a short folder name made from letters, numbers, hyphens, or underscores. For example, `first-latex-document` travels more reliably than a name containing punctuation and several spaces. Later we will develop a complete file-naming policy for collaborative projects.

1.2 The smallest useful document

Type these four lines. Do not add a title, packages, page settings, or copied preamble material yet.

Complete example: main.tex

```
\documentclass{article}
\begin{document}
Hello, \LaTeX{}!
\end{document}
```

Save the source and compile it. A successful build produces a PDF. If your editor shows a green tick or a message such as “process exited normally”, that status is helpful, but the PDF itself still deserves inspection.

What you should see

You should see one page containing “Hello, L^AT_EX!” as an ordinary line of text. The type, margins, page size, and page number come from class defaults. Your result may differ slightly across current TeX installations, but the sentence should be present and readable.

Source anatomy

Line 1: `\documentclass{article}`. The backslash begins a command. The command name is `documentclass`. The braces contain a required piece of information: the class name `article`. A class acts like an architectural plan. It defines the broad structures and default appearance available to the document.

Line 2: `\begin{document}`. This line opens the `document` *environment*. An environment is a region with a defined beginning and end. Here it marks the beginning of material that can appear in the output.

Line 3: `ordinary text`. This line contains no command. L^AT_EX treats it as text to typeset. We used a comma and exclamation mark in the ordinary way.

Line 4: `\end{document}`. This line closes the environment opened on line 2. L^AT_EX ignores source after this line during an ordinary document build, so the document should genuinely end here.

The portion before `\begin{document}` is called the *preamble*. The portion between the opening and closing lines is the *document body*. Our preamble currently contains only the class declaration.

Try It Now

Replace the sentence with your own plain sentence, save the file, and compile again. Confirm that the sentence changes but the page's broad design does not. Then add a second sentence on the next source line and compile once more. What does L^AT_EX do with the line break?

The second source line normally continues the same paragraph. A source line is not automatically a printed line. This behaviour lets us wrap source for readability without commanding a visual break after every line. Blank lines and paragraphs receive a full explanation in Chapter 2.

1.3 What compilation does

When you compile, the engine reads the class declaration first. It then reads the body in order, interprets commands, and constructs pages. It writes a PDF and several temporary *auxiliary files*. An auxiliary file stores information needed by the build, such as a table-of-contents entry or a label number.

This explains why a longer document sometimes needs more than one compiler pass. On the first pass, L^AT_EX records information. On a later pass, it reads that information and places it in the PDF. You do not need to manage those passes by hand when a tool such as `latexmk` can detect them.

1.3.1 A first local build with `latexmk`

A *terminal* is a text-based window in which you give the computer a command. Open a terminal in the folder containing `main.tex`, then enter:

Terminal command

```
latexmk -pdf main.tex
```

Here `latexmk` is the build tool, `-pdf` requests PDF output, and `main.tex` is the main source. `latexmk` runs the required programs again when it detects unfinished references or bibliography work. Its official manual describes this automated build role (Collins 2026).

To remove generated auxiliary files after an experiment, use:

Clean generated files

```
latexmk -C main.tex
```

The capital `-C` removes generated products recognised by `latexmk`. Check that you are in the intended project folder before using any `clean` command. Your `main.tex` source should remain.

Advanced Topic

An editor's build button usually runs one or more command-line programs behind the interface. Learning the `latexmk` command is useful because it gives a project a reproducible instruction that does not depend on one editor. You may continue using the build button while you are learning.

1.4 Compile, inspect, revise

Compilation succeeding is necessary, but it is not the same as the document being correct. Develop a four-part routine:

1. **Compile** after a small, coherent edit.
2. **Inspect** the area that should have changed.
3. **Read** any error and the most relevant warnings.
4. **Revise** the source, not the generated PDF.

Frequent builds keep cause and effect close together. If you write 200 lines before compiling, an error could be anywhere in those lines. If you write five lines, the search area is much smaller.

Pause and Predict

Change `article` to `book`, but do not compile yet. Will the words “Hello, L^AT_EX!” remain? Might some broad page choices change? Make a prediction, compile, and compare.

Then restore `article`. We will compare classes properly after learning the structures they provide.

1.5 Your first controlled error

Keep a working copy of the successful source. In the practice copy, remove the backslash from the second line:

Broken example: do not use as a working document

```
\documentclass{article}
begin{document}
Hello, \LaTeX{}!
\end{document}
```

Compile. The wording varies with the engine and editor, but the log should

report a problem involving a missing `\begin{document}` or text in the wrong place. L^AT_EX saw the characters `begin{document}` as text, not as the command required to open the body.

Repair the line:

Corrected line
<code>\begin{document}</code>

Compile again and confirm that the successful PDF returns.

Common Error
Symptom: the editor reports “Missing <code>\begin{document}</code>”. Likely causes: the opening command is missing, misspelt, or lacks its backslash; or visible text has accidentally been placed in the preamble. Repair: compare the top of the file with the four-line working example. Correct the first structural difference, save, and compile again. Do not respond by adding several unexplained commands at once.

1.6 Saving and keeping both forms

The editable scholarly record is not just the PDF. Keep `main.tex` and, when the project grows, all figures, data used to generate figures, bibliography files, local class/style files permitted for sharing, and build instructions. The PDF records what a particular build produced; the source makes revision and reproduction possible.

On Overleaf, use the current project menu to download source and the PDF-view control to download the typeset PDF. The source download is normally a ZIP archive. Extract it and verify that `main.tex` is present. On a local computer, make a backup outside the project folder or in a properly managed versioned location.

Science in Practice
Suppose an environmental monitoring report is revised after one unit label is corrected. Keeping only the old PDF preserves the error. Keeping source plus the data, figure, and build instruction makes the correction traceable and the revised report reproducible.

1.7 The first complete working pattern

We can now state our first eight-step pattern explicitly:

1. **Identify the problem:** create one typeset page.
2. **Describe the output:** one sentence in a readable PDF.
3. **Select the structure:** the standard `article` class and document environment.

4. **Write the smallest source:** four lines.
5. **Compile and inspect:** confirm the sentence appears.
6. **Correct errors:** compare the first structural difference.
7. **Improve carefully:** change one sentence, then rebuild.
8. **Check fitness:** retain both source and PDF. This is a practice page, not yet a submission-ready article.

That last sentence matters. A file can compile successfully without meeting a scholarly purpose. Technical validity and publication readiness are related, but they are not identical.

1.8 Chapter project

Mini Project

Inside `measurement-methods-article`, create `main.tex` with this source:

Running project: first compilable version

```
\documentclass{article}
\begin{document}
This project compares Method A and Method B.

All values are simulated demonstration data.
\end{document}
```

Compile it. The blank source line begins a new paragraph. Inspect the PDF and keep both source and output. Add a plain-text `README.md` stating that the project is educational and the data are simulated. Do not add titles, sections, equations, or packages yet; we shall earn each layer.

1.9 Summary

A first project can be created online or locally. Its main source file is the file the compiler reads first. The minimal document selects the `article` class, opens the document body, contains ordinary text, and closes the body. Compilation produces a PDF and auxiliary files. Frequent small builds make errors easier to locate, and `latexmk` automates repeated passes. A responsible archive keeps editable source as well as output.

1.10 Checklist

- ☐ I have compiled the four-line document successfully.
- ☐ I can point to the preamble and document body.

- ☐ I can explain every line without calling it “just required code”.
- ☐ I know where my editor or online project shows the build log.
- ☐ I deliberately created and repaired the missing-backslash error.
- ☐ I saved or downloaded both source and PDF.
- ☐ My running project clearly labels its content as simulated.

1.11 Exercises

- 1.1 Write the four-line minimal document from memory, then compare it with the printed source before compiling.
- 1.2 What is the difference between the `article` class and the `document` environment in this example?
- 1.3 Predict what happens when two ordinary source lines have no blank line between them. Test your prediction.
- 1.4 Change “Hello, L^AT_EX!” to a sentence from your field. Compile and identify which source line changed the output.
- 1.5 Repair this broken opening without adding new commands: `begin{document}`.
- 1.6 Explain why a successful compiler exit does not prove that a report is scientifically or typographically correct.
- 1.7 Challenge: compile the same source online and locally, if both paths are available. Record any visible difference and avoid assuming that a difference is an error.

1.12 Complete Practical Coding

This complete report is a reusable first project.

Complete practical file: first short report

```
\documentclass[11pt]{article}

\title{A Short Observation Report}
\author{Your Name}
\date{\today}

\begin{document}
\maketitle

\begin{abstract}
This report records one objective, one method, and one result.
```

```
\end{abstract}

\section{Objective}
Compare two sets of demonstration measurements.

\section{Method}
Record each value under the same conditions.

\section{Result}
Group B had the larger demonstration mean.

\section{Conclusion}
The source compiled and the report has a clear structure.

\end{document}
```

Code Lab

Compile the file with `latexmk -pdf main.tex`. Replace the result, add a second paragraph, and confirm that the PDF updates.

Understanding the Source

You can now compile a document. The next step is to read its source as a set of small, recognisable structures. This skill is more valuable than memorising a large preamble. When you can identify a command, its information, and the region in which it acts, you can usually ask a precise question and test a small repair.

We shall examine punctuation that L^AT_EX treats specially. Nothing here requires programming experience. The symbols are simply part of the written grammar used to communicate with the compiler.

Learning outcomes

- Distinguish commands, required arguments, and optional arguments.
- Match the beginning and end of an environment.
- Use comments to annotate source safely.
- Predict how spaces, line endings, and blank lines affect prose.
- Type common reserved characters in ordinary text.
- Separate the preamble from the document body.
- Construct a minimal working example and repair two common syntax errors.

2.1 Commands: instructions beginning with a backslash

In our first source, `\documentclass`, `\begin`, and `\end` are *commands*. A command tells L^AT_EX to perform an operation or insert something that ordinary text alone cannot express. Most command names made from letters are case-sensitive. `\begin` and `\Begin` are not the same instruction.

A command name made from letters ends when L^AT_EX reaches a non-letter. In this example, the opening brace ends the name `documentclass`:

A command and its information

```
\documentclass{article}
```

The backslash is not decoration. Without it, L^AT_EX sees ordinary characters. This is why removing the backslash in Chapter 1 changed the meaning of the line.

Classroom Note

Not every backslash is followed by a long word. Some commands use a single non-letter symbol. We will meet them when a task requires them. For now, read “backslash plus a name” as the usual command pattern.

2.2 Required arguments use braces

An *argument* supplies information to a command. A *required argument* is normally enclosed in curly braces:

Required argument pattern

```
\commandname{required information}
```

The pattern line is explanatory rather than a standalone document. Our actual class declaration has the same parts: `\documentclass` is the command and `article` is its required argument. Braces tell L^AT_EX exactly which characters belong to that argument.

Commands can have more than one argument, and an argument can contain other commands. We will introduce those cases one at a time. The immediate habit is to find each opening brace and its matching closing brace.

Pause and Predict

Compare these two lines. Which one has a complete required argument?

One complete and one incomplete line

```
\documentclass{article}  
\documentclass{article
```

The first has a matching pair of braces. The second leaves the argument open, so L^AT_EX may continue reading later lines while searching for the missing closing brace.

2.3 Optional arguments use brackets

Some commands accept additional choices in square brackets. These are *optional arguments*. The class command can accept options before its required class name:

Complete example: an optional class argument

```
\documentclass[11pt]{article}  
\begin{document}
```

```
This document requests 11-point base text.
\end{document}
```

Here `[11pt]` is optional and `{article}` is required. If we omit `[11pt]`, the class uses its default base size. The square brackets must appear in the position defined by the command. They are not a general substitute for braces.

What you should see

The PDF contains one sentence. Compared with the default minimal article, the base type should be slightly larger. The class still controls the margins, paragraph treatment, and page number.

Try It Now

Compile the example with `[10pt]`, `[11pt]`, and `[12pt]`, one at a time. Keep all other source unchanged. Compare the same sentence and the space it occupies. Which change is easiest to see?

2.4 Environments have a beginning and an end

An *environment* applies a structure or behaviour to a region. Its general shape is:

Environment pattern

```
\begin{environment-name}
Content inside the environment
\end{environment-name}
```

The name after `\begin` must match the name after `\end`. Our document body is the region inside the `document` environment. Later, other environments will create lists, quotations, equations, tables, figures, and proofs.

Environments can be nested, like boxes placed inside larger boxes. The inner environment must end before the outer environment ends. Indentation is not a $\text{\LaTeX}$ requirement, but it helps human readers see that nesting.

Common Error

Broken example: mismatched names

Broken example: do not compile as working source

```
\documentclass{article}
\begin{document}
The names do not match.
\end{article}
```

Cause: the source opens `document` but tries to close `article`. A class name is not an environment name.

Correction: replace the final line with `\end{document}`. When the names match, compile again.

2.5 Comments explain source to people

The percent sign begins a *comment*. L^AT_EX ignores the percent sign and the remainder of that source line:

Complete example: comments

```
\documentclass{article} % Select the standard article class.
\begin{document}
This sentence appears in the PDF.
% This reminder does not appear in the PDF.
\end{document}
```

Comments can explain a decision, mark a task, or temporarily remove a line from a test. They should help a future reader understand *why* the source is organised as it is. A comment that merely repeats the command adds little.

Because the percent sign starts a comment, a literal percentage in prose needs a backslash: `25\%` produces 25%. We will return to scientific percentages and quantities in the chapter on units.

Good Practice

Write comments that survive beyond the current minute. “Use the wide figure because labels become unreadable at column width” records a reason. “Make figure wide” records only an instruction whose purpose may be forgotten. Remove stale comments before submission.

2.6 Whitespace: spaces, lines, and paragraphs

In ordinary prose, L^AT_EX usually treats one or several spaces as a single interword space. A single source line ending also behaves like a space. A blank line ends the current paragraph and begins another.

Complete example: source spacing

```
\documentclass{article}
\begin{document}
Method A and      Method B
appear in one paragraph even though
the source changes line.

This sentence begins a new paragraph.
\end{document}
```

What you should see

The first three source lines form one typeset paragraph with ordinary word spacing. The blank line creates a paragraph break before the final sentence. $\text{\LaTeX}$ does not print the row of extra source spaces.

This behaviour lets us wrap source at readable widths. Pressing Return is not a request for a printed line break. In normal prose, use a blank line when you mean a new paragraph. Commands for deliberate line breaks exist, but they should solve a real structural need rather than imitate a typewriter.

Pause and Predict

Remove the blank line before the final sentence and compile. Will there be one paragraph or two? Now restore the blank line and add three more empty source lines. Will the printed gap become three times larger? Test both predictions.

Ordinarily, several blank source lines still mean one paragraph break. If a document needs more vertical space, its structure or design should specify the reason. Repeated empty lines are not a reliable page-layout method.

2.7 Characters with special jobs

Several ASCII characters are *reserved characters*: $\text{\LaTeX}$ uses them as part of its instruction language. To print many of them literally, place a backslash before the character.

The special role can depend on context. For example, an ampersand will later separate columns in a table or alignment. Outside such a structure, an unescaped ampersand usually produces an error.

Complete example: literal reserved characters

```
\documentclass{article}
\begin{document}
Recovery was 75\%, and A\&B shared sample\_01.
```

Table 2.1. Common reserved characters and a source form that prints them in ordinary text. The last three use named commands.

Character	Common role in source	Source used to print it
#	numbered parameter in definitions	\#
\$	mathematics delimiter	\\$
%	comment marker	\%
&	table alignment marker	\&
_	subscript in mathematics	_
{ and }	argument grouping	\{ and \}
\	command marker	\textbackslash
^	superscript marker	\textasciicircum
~	unbreakable space / active symbol	\textasciitilde

```
The budget was \$500 for item \#3.
\end{document}
```

What you should see

The PDF prints the percentage sign, ampersand, underscore, dollar sign, and hash symbol. The backslashes used to escape them are not printed.

Science in Practice

Scientific source often contains reserved characters: 95% for a percentage, sample_A1 for a file-like identifier, and R&D in an institutional phrase. Learning to recognise the character’s source role prevents apparently ordinary data labels from becoming syntax errors.

2.8 The preamble and the body

The *preamble* begins at the first source line and ends immediately before `\begin{document}`. It declares the class and later will load packages, define reusable commands, and set document-wide behaviour. The *document body* contains the material that forms the document’s content.

Preamble and body

```
\documentclass[11pt]{article} % Preamble

\begin{document}                % Body begins
This sentence is document content.
\end{document}                  % Body ends
```

The blank line in the tiny preamble is for human readability; it is not a paragraph because typeset body content has not begun. Ordinary visible prose does not belong in the preamble.

Common Error

Symptom: “Missing `\begin{document}`” appears even though an opening line exists farther down.

Possible cause: ordinary text was accidentally placed before the opening line. $\text{\LaTeX}$ encountered content where it expected setup instructions.

Repair: move the prose into the body, then compile. If the message remains, compare the spelling and braces of the opening line with the minimal working document.

2.9 A minimal working example

A *minimal working example*, often shortened to *MWE*, is the smallest complete source that still demonstrates a behaviour or problem. “Working” means complete enough to compile until the target issue occurs; it does not promise error-free output when the issue itself is an error.

An MWE helps in three ways:

- it separates the problem from unrelated chapters and packages;
- it lets another person reproduce exactly what you observed; and
- it often reveals the cause while you are removing irrelevant source.

Suppose an ampersand causes an error in a 150-page report. A useful MWE might be only this:

Broken MWE: unescaped ampersand

```
\documentclass{article}
\begin{document}
Method A & Method B
\end{document}
```

The example is complete, and the unescaped ampersand preserves the problem. The repair is precise:

Corrected MWE

```
\documentclass{article}
\begin{document}
Method A \& Method B
\end{document}
```

Do not send an entire confidential thesis when five non-confidential lines can demonstrate a technical fault. Replace sensitive content with a neutral example and confirm that the fault remains.

2.10 The first-aid debugging routine

Let us apply a calm routine to a missing brace.

Broken example: missing closing brace

```
\documentclass{article}
\begin{document}
This source has an unfinished group.
\end{document
```

The compiler reaches the end while still looking for the closing brace of the argument to `\end`. It may report a runaway argument, an emergency stop, or an unexpected end of file. The reported line can be at or after the true cause, because L^AT_EX noticed the problem only when it could no longer continue.

Use this sequence:

1. Read the first meaningful error, not only the last line of the log.
2. Note the reported line and inspect it plus a few preceding lines.
3. Match braces and environment names.
4. Make one small correction: add the closing brace.
5. Save and compile again.
6. Inspect the PDF; do not stop at a successful status message.

Corrected closing line

```
\end{document}
```

If the small correction works, it provides evidence for the cause. If it does not, return to the first new meaningful error. Avoid changing several unrelated lines, because then you cannot tell which change mattered.

Try It Now

Make a safe copy of your first document. Delete the closing brace from `\documentclass{article}`, compile, and record the first useful message. Restore the brace and compile again. Compare this log with the missing-brace example above: the wording may differ because the unfinished argument begins in a different place.

2.11 Reading source from left to right

When a line looks unfamiliar, ask four questions:

1. Where does the command name begin and end?
2. Which braces contain required information?
3. Are there square brackets containing an optional choice?
4. Does the line open or close an environment that must be matched?

Apply the questions to:

Source-reading practice
<pre>\documentclass[12pt]{article}</pre>

The command is `\documentclass`. The optional argument is `[12pt]`. The required argument is `{article}`. The line does not open an environment. This mechanical reading is simple, but it scales to longer commands introduced later.

Good Practice
Keep the source readable for a colleague: indent nested regions consistently, place logical units on separate lines, and avoid dense sequences of commands when a clearer arrangement is available. $\text{\LaTeX}$ may ignore some whitespace; human readers do not.

2.12 Chapter project

Mini Project
Replace the running project's <code>main.tex</code> with the annotated version below. The document still uses only structures already introduced.

Running project: readable source anatomy

```
\documentclass[11pt]{article}

% Educational example: the values are simulated.
\begin{document}
This project compares Method A and Method B.
The eventual aim is a reproducible scholarly article.

All values are simulated demonstration data.
They do not describe a real experiment or clinical study.

Planned completeness is 100\%, and the project folder is
measurement\_methods.
\end{document}
```

Before compiling, predict the number of paragraphs and identify every escaped reserved character. Compile, inspect, and then write one helpful comment of your own. Update the README only if your new comment records information that belongs at project level rather than beside one source decision.

2.13 Summary

Commands usually begin with a backslash. Braces group required arguments; square brackets commonly contain optional arguments. Environments have matching beginning and ending names. A percent sign starts a source comment, while reserved characters need special treatment when printed literally. Ordinary spaces and line endings collapse in prose, but a blank line starts a new paragraph. The preamble configures the document; the body contains its content. A minimal working example and the first-meaningful-error routine turn a large mystery into a small test.

2.14 Checklist

- ☐ I can identify a command name and its arguments.
- ☐ I match every opening environment with the same closing name.
- ☐ I can add a comment and explain why it does not print.
- ☐ I can predict whether source lines form one paragraph or two.
- ☐ I can print common reserved characters safely.
- ☐ I can point to the preamble and body of a complete source file.
- ☐ I can reduce a simple error to a minimal working example.

2.15 Exercises

- 2.1 Identify the command, optional argument, and required argument in `\documentclass[12pt]{book}`.
- 2.2 Explain why braces and brackets are not interchangeable in the class declaration.
- 2.3 Predict the PDF paragraph count when five source lines contain no blank line. Test your answer.
- 2.4 Write source that prints: `Accuracy was 95% for sample_7`.
- 2.5 Find and repair the mismatch: `\begin{document}` followed by `\end{article}`.
- 2.6 Create a five-line MWE that demonstrates an unescaped ampersand error, then create the corrected version.
- 2.7 Read this error strategy: “Change five nearby lines and compile once.” Explain why it produces weak evidence about the cause.
- 2.8 Challenge: take a harmless problem from your practice source and reduce it until removing any remaining line would stop the example from demonstrating the same problem.

2.16 Complete Practical Coding

The next file demonstrates commands, arguments, environments, comments, and reserved characters in one place.

Complete practical file: source anatomy

```
\documentclass[11pt]{article} % class and optional argument
\usepackage[T1]{fontenc}      % package command

\newcommand{\sample}[1]{\texttt{#1}}

\begin{document}

\section{Commands and arguments}
\textbf{Bold text} uses a required argument.
The reusable command prints \sample{sample-01}.

\section{Environment}
\begin{quote}
An environment has matching begin and end commands.
\end{quote}

\section{Reserved characters}
```

```
The recorded values were 95\%, A \@ B, sample\_01, and \$25.
```

```
% This comment is visible in the source but not in the PDF.
```

```
\end{document}
```

Code Lab

Remove one closing brace, compile, and read the first error. Restore the brace, then change the definition of `\sample` in one place.

Part II

Writing and Organising Text

Writing Clear Scientific Prose

A document can compile perfectly and still be hard to read. Scientific prose needs visible paragraphs, purposeful emphasis, accurate symbols, and source that another author can revise without decoding improvised formatting. In this chapter we give ordinary text the care that mathematical notation will receive later.

Learning outcomes

- Create paragraphs and distinguish them from line breaks.
- Apply emphasis, bold, and italic text for stated purposes.
- Type quotation marks, dashes, ellipses, and reserved characters.
- Insert non-breaking spaces, footnotes, and web addresses.
- Format a biological species name conventionally.
- Explain a safe beginner approach to Unicode source.

3.1 Paragraphs carry ideas

A paragraph is a unit of thought, not merely a block that fits a page. In source, leave one blank line when the argument moves to a new paragraph. $\text{\LaTeX}$ decides the printed line endings.

Complete example: two paragraphs

```
\documentclass{article}
\begin{document}
Method A records one value for each specimen. Method B records
the same property independently.

We compare paired values because both methods examine the same
specimens.
\end{document}
```

The PDF contains two paragraphs. The single source line ending inside each paragraph becomes an ordinary space. This separation lets source lines remain short enough to review.

The command `\par` requests a line break without starting a new paragraph. It is useful in structures such as an address or a short title block, but it is usually wrong between prose

paragraphs. A blank line expresses the meaning more accurately.

Common Error

Symptom: prose has a ragged sequence of short printed lines.

Cause: every source line ends with

.

Correction: remove the forced breaks and let $\text{\LaTeX}$ compose the paragraph. Keep a blank source line only where a new paragraph begins.

3.2 Emphasis and type styles

Use `\emph{text}` when the meaning is emphasis. In ordinary text, $\text{\LaTeX}$ generally renders it in italics; inside existing emphasis it can switch back, which preserves the distinction.

Complete example: emphasis and styles

```
\documentclass{article}
\begin{document}
The values are \emph{simulated}, not observations.

\textbf{Warning:} do not interpret them as clinical evidence.

The species name \textit{Drosophila melanogaster} is italic.
\end{document}
```

`\textbf` requests bold text; use it for a genuine visual role such as a short warning label, not for every important sentence. `\textit` requests italic shape without claiming emphasis, which suits terms conventionally set in italics. Semantic structure remains preferable: a section title should be made with `\section`, not with a large bold line.

Science in Practice

Biological genus and species names are conventionally italic, while the authority is not: *Panthera leo* (Linnaeus, 1758). After the first full mention, a field may permit an unambiguous abbreviation such as *P. leo*. Follow the relevant disciplinary style and avoid automatic abbreviation when two genera could share the initial.

3.3 Quotation marks, hyphens, and dashes

In pdf $\text{\LaTeX}$ source, type an opening double quotation as two grave accents and a closing one as two apostrophes. Single quotations use one of each:

Complete example: quotations and dashes

```
\documentclass{article}
\begin{document}
The manual calls this a ``minimal working example''.
The term `minimal' does not mean incomplete.

A high-resolution image uses a hyphen. Pages 12--18 use an
en dash. A sudden interruption---used sparingly---uses an em dash.
\end{document}
```

One hyphen joins a compound; two hyphens in source produce an en dash for a range or relationship; three produce an em dash for a break in thought. British editorial styles differ over spaces around an em dash. This book uses none and applies the choice consistently.

Use `\ldots` for a deliberate ellipsis in text: Three full stops may have uneven spacing. An ellipsis signals omitted or trailing material; it should not become a decorative ending for uncertain prose.

Pause and Predict

Predict the output of 10-12, 10-12, and 10--12 before compiling them. Which source represents a numerical range?

3.4 Spaces that should stay together

A tilde in prose creates a *non-breaking space*. It prevents a line break between related items while printing an ordinary-looking space:

Useful non-breaking spaces

```
Figure~3
Dr~Rao
Section~2.1
```

These fragments are not a complete document, but each shows a relationship that should not split across lines. Later, automatic cross-references will replace typed numbers such as **Figure 3**. Do not insert tildes between every pair of words; excessive restrictions can produce poor spacing.

3.5 Footnotes and web addresses

A footnote command contains the note where its marker belongs:

Complete example: a footnote and URL

```

\documentclass{article}
\usepackage{hyperref}
\begin{document}
The data are simulated.\footnote{The file is supplied only for
typesetting practice.}

Documentation is available at
\url{https://www.latex-project.org/help/documentation/}.
\end{document}

```

`\footnote` prints a marker and places the note at the bottom of the page. Use footnotes for short supplementary remarks, not for essential methods or a hidden chain of citations. The `hyperref` package provides `\url`, which allows a web address to break more safely and makes it a link in the PDF.

Long or unstable URLs are rarely good citation substitutes. A bibliography record can carry the author, title, date, access date, and persistent identifier. Chapter 12 develops that workflow.

3.6 Unicode without mystery

Modern $\text{\LaTeX}$ source is normally saved as UTF-8, an encoding capable of representing many writing systems. A current engine can read common accented Latin characters directly, but font and engine support still matter. A file copied from a website may also contain invisible spaces or typographic quotes that look similar to ASCII source syntax.

For beginner examples we keep command syntax and quotation marks in ASCII. An accent command such as `Jos\é` is portable in a small $\text{pdf}\text{\LaTeX}$ example. Later, a project requiring extensive multilingual text may choose $\text{Lua}\text{\LaTeX}$ or $\text{Xe}\text{\LaTeX}$ with suitable fonts after checking the publisher’s workflow.

Good Practice

Do not repair a mysterious character by repeatedly changing the compiler. Identify the character, confirm that the file is UTF-8, and test a minimal example. A journal template may require a specific engine.

3.7 Chapter project

Mini Project

Improve the prose in the running article. Add two background paragraphs, keeping the existing simulated-data statement. Use `\emph` once to stress that the values are simulated, type `Method A\&B` only if an ampersand is genuinely part of a name, and add a short footnote explaining that Method A and Method B are neutral teaching labels. Compile and check that no forced line breaks interrupt the prose.

3.8 Summary

Blank source lines make paragraphs; ordinary source line endings do not force printed breaks. `\emph` expresses emphasis, while `\textbf` and `\textit` request particular shapes. ASCII quotation conventions, source hyphens, `\ldots`, non-breaking spaces, escaped characters, footnotes, and `\url` give prose precise typography. UTF-8 is the normal source encoding, but unusual characters still need compatible fonts and engines.

3.9 Checklist

- ☐ I use blank lines, not repeated
, for prose paragraphs.
- ☐ I can explain why a word is emphasised, bold, or italic.
- ☐ I can type opening and closing quotation marks in ASCII source.
- ☐ I choose the right source form for a hyphen, range, or em dash.
- ☐ I can add a footnote and a safe URL.
- ☐ I have checked the running article’s paragraph structure.

3.10 Exercises

- 3.1 Explain the difference between a source line ending and a blank line.
- 3.2 Correct a paragraph that ends every source line with
.
- 3.3 Write source for the quotation: The result was “unexpected”.
- 3.4 Choose a hyphen or dash for each: pages 4 to 9; evidence based; an interruption in a sentence.
- 3.5 Give one suitable and one unsuitable use of a footnote.

3.6 Format *Escherichia coli* and its later abbreviation.

3.7 Debug source in which 50% accidentally comments out the rest of the line.

3.8 Challenge: edit a paragraph from your field using only semantic emphasis and explain every typographic choice.

3.11 Complete Practical Coding

Use semantic commands for meaning and keep manual styling restrained.

Complete practical file: readable scientific prose

```
\documentclass{article}
\usepackage[T1]{fontenc}
\usepackage{lmodern}
\usepackage[british]{babel}

\newcommand{\species}[1]{\textit{#1}}

\begin{document}

\section{Observation}
We observed \species{Example species} in 12 plots. The first
measurement was made in the morning; the second was made two
hours later. The difference was small---not zero---and should be
reported without unnecessary emphasis.

The sample identifier was \texttt{plot\_07}. A short footnote can
hold a useful qualification without interrupting the sentence.\footnote
{The
demonstration values are not research data.}

\section{Interpretation}
Use \emph{emphasis} for meaning. Use \textbf{bold text} mainly for
short labels, not whole paragraphs.

\end{document}
```

Code Lab

Replace both paragraphs with your own prose, add one species name, and confirm that every identifier and reserved character prints correctly.

Building Scholarly Structure

Readers do not experience a scientific document as one uninterrupted stream. They use a title to identify it, an abstract to judge relevance, headings to navigate the argument, and appendices to find supporting detail. L^AT_EX works best when the source names these roles instead of imitating their appearance.

Learning outcomes

- Create title metadata and print it with `\maketitle`.
- Insert an abstract and keywords in a generic article.
- Organise content with sections and subsections.
- Generate a table of contents and explain repeat compilation.
- Add an appendix without typing its letter by hand.
- Distinguish logical structure from manual visual formatting.

4.1 Metadata before appearance

A title, author, and date describe the document. In the standard article class we store them in the preamble and ask the class to typeset them:

Complete example: article title

```
\documentclass{article}

\title{A Reproducible Comparison of Two Measurement Methods}
\author{Student Author}
\date{2026}

\begin{document}
\maketitle
This is a teaching document with simulated data.
\end{document}
```

Source anatomy

`\title`, `\author`, and `\date` store required arguments. They do not print the title by themselves. `\maketitle` in the body asks the class to construct the title block from the stored information. An empty `\date{}` suppresses the date; `\today` uses the compilation date. For a versioned scholarly file, an explicit date is often easier

to audit.

What you should see

The PDF begins with a centred title block using the article class’s design, followed by the teaching statement. The source does not specify font sizes or vertical distances for each line.

Try It Now

Change only the class from `article` to `report`. Compile and compare the title treatment. Restore `article`. This demonstrates that metadata can remain stable while the class changes broad design.

4.2 The abstract

An abstract is a compact account of purpose, approach, principal result, and meaning. The standard `article` class provides an environment:

Abstract and keywords

```
\begin{abstract}
We demonstrate a reproducible workflow for comparing two
measurement methods using simulated paired values. The example
is pedagogical and makes no empirical claim.
\end{abstract}

\noindent\textbf{Keywords:} agreement; measurement; reproducibility;
simulated data
```

This fragment belongs after `\maketitle` and before the first section. The keyword line is an explicitly formatted convention for a generic article; the standard class has no universal `\keywords` command. Publisher classes often provide their own command or environment. Use theirs when instructed.

Good Practice

Do not put citations, undefined abbreviations, or conclusions unsupported by the body into an abstract unless the target venue explicitly permits them. Typesetting cannot repair an abstract that does not represent the study.

4.3 Sections describe the argument

Use the heading level that represents the relationship between ideas:

Complete example: section hierarchy

```
\documentclass{article}
\begin{document}
\section{Introduction}
The study asks whether two methods agree.

\section{Methods}
\subsection{Study design}
Paired simulated values are used.

\subsection{Analysis}
Differences are calculated for each pair.

\section{Results}
Results will be added after the method is defined.
\end{document}
```

`\section` creates a major division; `\subsection` creates a division inside the current section; `\subsubsection` is available when a third level is genuinely needed. Do not skip from a section to a subsubsection simply to obtain a smaller heading.

The starred form, such as `\section*`, usually suppresses numbering and a table-of-contents entry. It suits occasional unnumbered matter, not every heading in a manuscript that requires navigation.

Common Error

Mistake: making a heading with `\textbf` and blank lines.

Consequence: it looks like a heading but has no structural level, number, contents entry, or reliable target for a reference.

Correction: replace it with the appropriate section command and let the class format it.

4.4 A table of contents

The command `\tableofcontents` prints headings recorded during compilation. Place it where the contents should appear, commonly after front matter in a long report or thesis.

On the first pass, L^AT_EX writes heading information to a `.toc` auxiliary file. On the next pass it can read that file. This is why a new section may not appear in the contents until compilation repeats. `latexmk` notices the change and runs the needed pass.

Short journal articles rarely include a table of contents. The command is useful for reports, theses, books, and long internal documents. Structure and reader need should decide, not the mere availability of a command.

4.5 Appendices

The command `\appendix` changes the numbering of later top-level divisions. In an article, later sections normally become Appendix A, Appendix B, and so on.

Appendix pattern

```
\appendix
\section{Additional checks}
This appendix records sensitivity checks for the simulated example.
```

Do not type “Appendix A” inside the section title. L^AT_EX supplies the letter and updates it if another appendix is inserted earlier.

Science in Practice

An earth-science paper might place a detailed geological sample description in an appendix while keeping the sampling rationale in Methods. The appendix is still part of the scholarly record and should be cited from the main text, labelled clearly, and reviewed with the same care.

4.6 A complete structural skeleton

The source below contains a document’s architecture but very little prose. It is useful as a planning skeleton because every heading represents a purpose.

Complete example: generic scholarly skeleton

```
\documentclass[11pt]{article}

\title{A Reproducible Comparison of Two Measurement Methods}
\author{Student Author}
\date{2026}

\begin{document}
\maketitle

\begin{abstract}
This educational example uses simulated paired values.
\end{abstract}

\noindent\textbf{Keywords:} agreement; reproducibility;
simulated data

\section{Introduction}
State the problem and objective.

\section{Methods}
```

```
Describe data generation and analysis.

\section{Results}
Report the findings without interpretation.

\section{Discussion}
Interpret the demonstration and its limitations.

\section{Conclusion}
Answer the objective briefly.

\appendix
\section{Simulated data record}
Document how the teaching values were created.
\end{document}
```

The imperative sentences are planning notes, not final manuscript prose. A useful skeleton exposes missing logical components early; it should not tempt us to fill headings with generic text that says nothing.

Pause and Predict

Insert a new section before Results. Predict what happens to all later section numbers after compilation. Now imagine those numbers had been typed into the headings by hand.

4.7 Chapter project

Mini Project

Turn the running article into the structural skeleton above. Replace **Student Author** with `<Author name>`; keep it visibly a placeholder rather than inventing an identity. Retain the explicit simulated- data statement. Compile twice or use `latexmk`. Confirm that the abstract, keywords, five main sections, and appendix appear in the expected order.

4.8 Summary

Title commands store metadata and `\maketitle` prints it according to the class. The abstract environment marks a scholarly role; generic keywords need a documented local convention. Section commands express hierarchy and enable automatic numbering and contents entries. `\tableofcontents` may require a repeat pass, and `\appendix` changes later numbering without hard-coded letters. Logical structure should lead visual formatting.

4.9 Checklist

- ☐ My title information is stored in the preamble.
- ☐ My abstract states what the demonstration does and does not claim.
- ☐ Every heading command represents a real level of the argument.
- ☐ I have not typed section or appendix numbers by hand.
- ☐ I understand why the contents may need another compiler pass.
- ☐ The running article now has a complete scholarly skeleton.

4.10 Exercises

- 4.1 Explain the different jobs of `\title` and `\maketitle`.
- 4.2 Add an explicit date to a minimal titled article.
- 4.3 Convert a bold manual heading into semantic source.
- 4.4 Predict the numbering after inserting a section before Methods.
- 4.5 Explain why the generic article’s keyword line is not universal.
- 4.6 Add two automatically lettered appendices.
- 4.7 Debug an abstract placed before `\begin{document}`.
- 4.8 Challenge: plan the heading hierarchy for a report in your field and justify every level.

4.11 Complete Practical Coding

This pattern supplies the complete logical skeleton of a scholarly article.

Complete practical file: scholarly structure

```
\documentclass{article}

\title{A Structured Demonstration Study}
\author{Your Name}
\date{}

\begin{document}
\maketitle

\begin{abstract}
State the question, method, principal result, and conclusion.
\end{abstract}
```

```
\section{Introduction}
Define the problem and objective.

\section{Methods}
Describe materials, design, measurements, and analysis.

\section{Results}
Report the findings without interpretation.

\section{Discussion}
Interpret the findings and limitations.

\section{Conclusion}
Answer the objective directly.

\appendix
\section{Supplementary calculation}
Place supporting detail here.

\end{document}
```

Code Lab

Add a real title and objective, insert a subsection under Methods, and move the appendix before the conclusion to observe how numbering responds.

Lists, Quotations, and Notes

Methods often contain criteria; reports contain recommendations; teaching documents contain steps. A list can make those relationships visible, but a list used for every pair of sentences fragments the argument. L^AT_EX provides structures for unordered, ordered, and labelled items, plus distinct forms for short and multi-paragraph quotations.

Learning outcomes

- Create unordered, ordered, and description lists.
- Nest lists without losing their logical hierarchy.
- Choose between the `quote` and `quotation` environments.
- Insert footnotes at suitable locations.
- Explain when margin notes are unsafe or unnecessary.
- Customise list spacing conservatively with `enumitem`.

5.1 Unordered lists with `itemize`

Use `itemize` when sequence does not change meaning:

Complete example: an unordered list

```
\documentclass{article}
\begin{document}
The comparison requires:
\begin{itemize}
  \item paired measurements;
  \item a recorded unit; and
  \item an explicit missing-data rule.
\end{itemize}
\end{document}
```

Each `\item` begins an item. The punctuation is part of the sentence surrounding the list: these fragments use semicolons and the last ends with a full stop. Complete-sentence items can each end with a full stop instead.

5.2 Ordered lists with `enumerate`

Use `enumerate` when order, rank, or later reference matters:

Complete example: a procedure

```
\begin{enumerate}
  \item Record the Method A value.
  \item Record the Method B value.
  \item Calculate the paired difference.
  \item Inspect the result before summarising it.
\end{enumerate}
```

This fragment belongs inside a document body. L^AT_EX supplies the numbers and renumbers later items when one is inserted. If the prose must refer repeatedly to a particular item, attach a label rather than typing its number; labels are introduced in Chapter 11.

Pause and Predict

Add a new second item. Predict the number of the original second item and every item after it. What maintenance problem would arise if the numbers were typed as ordinary text?

5.3 Description lists

A description list pairs a term with its explanation:

Complete example: labelled definitions

```
\begin{description}
  \item[Method A] The reference teaching method.
  \item[Method B] The comparison teaching method.
  \item[Difference] Method B minus Method A for one pair.
\end{description}
```

The text in square brackets is the item label, not an automatically numbered optional setting. Keep labels short enough to remain readable. For a large alphabetical vocabulary, a glossary is more suitable than a very long list in the middle of a chapter.

5.4 Nested lists

A list can contain another list when the hierarchy is genuine:

Complete example: one nested level

```
\begin{enumerate}
  \item Validate the input.
    \begin{itemize}
      \item Check that each pair has an identifier.
    \end{itemize}
\end{enumerate}
```



```

    \item Check that both values use the same unit.
  \end{itemize}
  \item Calculate the differences.
\end{enumerate}

```

Indentation helps readers of the source see which environment is inside which. Deeply nested lists are difficult to read in source and PDF. If the hierarchy needs three or four levels, consider headings, a table, or separate procedural stages.

Common Error

Symptom: “Something’s wrong—perhaps a missing `\item`”.

Likely cause: text has been placed directly inside a list before its first `\item`, or an `item` command is missing.

Repair: inspect the first lines after `\begin{itemize}` or `\begin{enumerate}`. Make every list entry begin with `\item`.

5.5 Short and long quotations

Use the `quote` environment for a short displayed quotation. Use `quotation` when the quoted material contains multiple paragraphs; it marks their paragraph structure.

Displayed quotation pattern

```

The protocol defines repeatability as follows:
\begin{quote}
  [Insert a short, verified quotation and cite its source.]
\end{quote}
The analysis applies that definition to simulated values.

```

The bracketed sentence is a placeholder, not a quotation. Do not invent words and place them in quotation marks. When exact wording is needed, check the source, quote only what is necessary, preserve its meaning, and cite it. Most scholarly explanation should paraphrase responsibly in the author’s own voice.

Science in Practice

A standards report may need one exact definition because later requirements depend on its wording. A literature review rarely needs long blocks of copied text. In both cases, quotation formatting is separate from permission, accuracy, and citation responsibility.

5.6 Footnotes and optional margin notes

Footnotes are useful for a concise qualification that would interrupt the main sentence. Put the command after the word or punctuation associated with the note, following the chosen style consistently. Do not put a full data table or indispensable method in a footnote.

Margin notes require a package or class command and can collide with floats, two-column layouts, or binding margins. They are therefore optional advanced material, not a default scholarly annotation system.

Advanced Topic

Some classes provide `\marginpar{text}`. Before using it, check whether the document is one- or two-sided and whether the target venue permits margin material. Draft reminders are often safer as source comments because they cannot accidentally become publication content.

5.7 Careful customisation with enumitem

The `enumitem` package can adjust labels and spacing. Load it in the preamble, then state a local purpose:

Complete example: a compact local list

```
\documentclass{article}
\usepackage{enumitem}
\begin{document}
\begin{enumerate}[label=Step \arabic*:,itemsep=0.25em]
  \item Import the paired values.
  \item Confirm the unit.
  \item Calculate differences.
\end{enumerate}
\end{document}
```

The option changes this list only. Avoid setting every list to zero spacing; the apparent saving can make a dense technical page harder to scan. A class may already define list spacing appropriate to its layout.

Good Practice

Choose a list because the relationship is unordered, sequential, or term-and-definition. Keep parallel grammatical form: do not mix “Record the value”, “Calculation of the mean”, and “The plot is inspected” in one procedural list.

5.8 Chapter project

Mini Project

In the Methods section of the running article, add a three-item ordered list describing data validation, difference calculation, and inspection. Add an unordered list of two inclusion criteria. Keep the items grammatically parallel. Add a source comment for a future verified definition; do not insert an invented quotation.

5.9 Summary

`itemize` expresses an unordered set, `enumerate` a sequence, and `description` a labelled relationship. Nested lists should reflect a real, shallow hierarchy. `quote` suits a short displayed quotation; `quotation` supports multiple quoted paragraphs. Footnotes hold concise supplements, while margin notes are optional and layout-sensitive. `enumitem` can customise a list when the reason is explicit.

5.10 Checklist

- ☐ I chose the list type from the relationship between items.
- ☐ Every entry begins with `\item`.
- ☐ Nested environments close in the reverse order they opened.
- ☐ My list items use parallel grammar and consistent punctuation.
- ☐ Any exact quotation has a verified source.
- ☐ Important methods and evidence remain in the main text.

5.11 Exercises

- 5.1 Choose a list type for materials, a protocol, and a glossary of three terms.
- 5.2 Convert three manually numbered lines into `enumerate`.
- 5.3 Add one nested checklist under the first procedural step.
- 5.4 Repair a list whose first entry lacks `\item`.
- 5.5 Explain the difference between `quote` and `quotation`.
- 5.6 Rewrite a list so that all items begin with active verbs.
- 5.7 Give a reason to keep a draft note as a source comment rather than a margin note.

5.8 Challenge: redesign a dense list from a report as either prose, a table, or two smaller lists, and justify the choice.

5.12 Complete Practical Coding

Lists and quotations should reveal structure rather than decorate prose.

Complete practical file: lists, quotation, and note

```
\documentclass{article}

\begin{document}

\section{Selection criteria}
\begin{itemize}
  \item complete records;
  \item a valid sample identifier;
  \item measurements from both conditions.
\end{itemize}

\section{Procedure}
\begin{enumerate}
  \item Check the identifier.
  \item Record the initial value.
  \item Apply the treatment.
  \item Record the final value.
\end{enumerate}

\section{Terms}
\begin{description}
  \item[Observation] A recorded value or description.
  \item[Replicate] An independently repeated unit.
\end{description}

\begin{quote}
  Use a displayed quotation only when the wording itself matters.
\end{quote}

A footnote can hold a brief qualification.\footnote{Keep essential
  reasoning
  in the main text.}

\end{document}
```

Code Lab

Add one nested list only where the hierarchy is genuine. Rewrite the displayed quotation as ordinary prose and compare the two pages.

Part III

**Mathematics and Scientific
Notation**

Writing Mathematics from the Beginning

Mathematics in a scientific document is part of a sentence. It must be accurate, readable, and connected to the variables around it. L^AT_EX gives mathematics a separate mode because symbols such as a minus sign, a variable, and a fraction need different spacing from ordinary prose.

Learning outcomes

- Distinguish inline and displayed mathematics.
- Create scripts, fractions, roots, Greek letters, and functions.
- Size brackets around structured expressions.
- Align related equations with `amsmath`.
- Number, label, and refer to an equation.
- Avoid obsolete display syntax and manual spacing repairs.

6.1 Inline mathematics belongs in a sentence

Place short mathematics between `$` delimiters:

Complete example: inline mathematics

```
\documentclass{article}
\begin{document}
Let  $d_i = B_i - A_i$  be the difference for pair  $i$ .
\end{document}
```

The dollar signs switch mathematics on and off; they do not print. Letters in math mode are treated as variables and normally appear italic. The minus sign has mathematical spacing. Keep sentence punctuation outside or after the formula as grammar requires.

Common Error

Symptom: the rest of a paragraph appears in mathematical italics or the compiler reports a missing dollar sign.

Cause: an opening `$` has no matching closing delimiter.

Repair: inspect the equation and the source just before the reported line. Match the delimiters, then compile again.

6.2 Displayed mathematics

Use `\[` and `\]` for an unnumbered display:

Unnumbered display

```
The arithmetic mean is
\[
  \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.
\]
```

The display is still part of the sentence, so it ends with a full stop. Indentation is for source readability.

Do not use `$$ ... $$`. That plain-TeX form bypasses L^AT_EX's display management and can give inconsistent spacing. Numbered displays use an appropriate environment, not a manually typed number.

6.3 Scripts need groups

The caret creates a superscript and the underscore a subscript in mathematics:

Scripts and grouping

```
$x^2$, $x_i$, $x_{ij}$, $10^{-3}$, and $a^{n+1}$
```

Without braces, only the next token belongs to the script. Thus `xij` makes only `i` a subscript and leaves `j` at the main level. Group multi-character scripts: `x{ij}`.

Try It Now

Compile `$x_{ij}$` and `$x_{ij}$` side by side. Describe the baseline of the letter `j` in each output.

6.4 Fractions, roots, and Greek letters

`\frac` takes a numerator and denominator. `\sqrt` takes the expression under a root and accepts an optional root index.

Complete example: common constructions

```

\documentclass{article}
\usepackage{amsmath}
\begin{document}
\[
  z = \frac{x - \mu}{\sigma}, \quad
  r = \sqrt{x^2 + y^2}, \quad
  q = \sqrt[3]{V}.
\]
\end{document}

```

Commands such as `\mu`, `\sigma`, `\alpha`, and `\Delta` insert Greek letters. Upper- and lower-case command names can represent different symbols. Do not paste a look-alike Unicode character into an otherwise ASCII formula without checking the engine and font support.

The `\qquad` spacing in this compact demonstration separates three independent expressions. It should not be used to repair the internal structure of one badly expressed formula.

6.5 Functions and operators

Standard functions have upright names and mathematical spacing:

Functions and operators

```

 $\sin \theta$ ,  $\log x$ ,  $\exp(-kt)$ , and  $\max_i x_i$ 

```

Use the supplied commands rather than typing italic letters such as `sin`. For a recurring named operator not supplied by L^AT_EX, `amsmath` provides `\DeclareMathOperator`; define it once in the preamble after understanding its mathematical role.

Ordinary words inside mathematics use `\text` from `amsmath`:

Words inside a formula

```

\[
  y = \begin{cases}
    1, & \text{if the pair is complete}, \\
    0, & \text{otherwise}.
  \end{cases}
\]

```

The ampersand marks the alignment point and ends a row inside this structured environment. They are not paragraph-formatting tricks.

6.6 Brackets that fit

Ordinary parentheses do not grow automatically around a tall fraction. `\left` and `\right` request paired delimiters sized to their contents:

Sized delimiters

```
\[
  \left(\frac{a+b}{c+d}\right)^2
\]
```

Every `\left` needs a matching `\right`. When only one visible delimiter is required, a full stop can act as an invisible partner, but that technique should be introduced only with a real notation need. The `mathtools` package also supports reusable paired-delimiter commands for larger projects.

6.7 Numbering and referencing an equation

The `equation` environment numbers one display. Place the label after the equation begins and give it a descriptive prefix:

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n (B_i - A_i). \quad (6.1)$$

The source is:

Numbered and labelled equation

```
\begin{equation}
  \bar{d}=\frac{1}{n}\sum_{i=1}^n(B_i-A_i).
  \label{eq:mean-difference-example}
\end{equation}

Equation~\eqref{eq:mean-difference-example} defines the mean difference
.
```

`\eqref` prints the equation number with parentheses. Compilation may need a repeat pass after a new label. A label is a stable name; the visible number can change when an earlier equation is inserted.

Science in Practice

In the running comparison, A_i and B_i are paired simulated values and $\bar{d}$ estimates their mean signed difference. The equation does not by itself establish agreement; the surrounding methods must define sampling, units, missingness, and interpretation.

6.8 Aligned equations

Use `align` for related lines. The ampersand marks the alignment point, usually before a relation such as equals:

Complete example: aligned equations

```
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\begin{align}
d_i &= B_i - A_i, \\
\bar{d} &= \frac{1}{n} \sum_{i=1}^n d_i.
\end{align}
\end{document}
```

Use `align*` when none of the lines should be numbered. Use `\notag` selectively when a single line in a numbered alignment should not receive a number.

Do not use the older `eqnarray` environment. Its spacing around relation symbols is inferior and its control is limited. Do not align equations by inserting guessed spaces; state the alignment point structurally.

Good Practice

Define every symbol near its first use, keep notation consistent, and inspect the meaning as well as the typesetting. A perfectly aligned equation with an undefined variable is not publication-ready.

6.9 Chapter project

Mini Project

Load `amsmath` and `mathtools` in the running article. In Methods, define $d_i = B_i - A_i$. Add a labelled equation for the mean difference and refer to it from the prose with `\eqref`. State clearly that the values are simulated and that the sign depends on the chosen subtraction order.

6.10 Summary

Dollar delimiters create inline mathematics; `\[` and `\]` create an unnumbered display. Braces group scripts and arguments. Mathematical commands produce fractions, roots, symbols, functions, text, and sized delimiters. `equation` numbers one display; `align` organises related lines. Labels and `\eqref` replace typed numbers. Avoid `$$`, `eqnarray`, and manual spacing repairs.

6.11 Checklist

- ☐ Every math delimiter and brace has a partner.
- ☐ Multi-character scripts are grouped.
- ☐ Function names are upright commands.
- ☐ Display punctuation follows the surrounding sentence.
- ☐ Equation references come from labels.
- ☐ Every symbol in the running article is defined.

6.12 Exercises

- 6.1 Write inline source for $E = mc^2$.
- 6.2 Correct `x_ij` so both letters form the subscript.
- 6.3 Typeset the fraction $(x - \mu)/\sigma$ with a built-up fraction.
- 6.4 Replace italic `sin` with the mathematical function command.
- 6.5 Create two equations aligned at their equals signs.
- 6.6 Label an equation and refer to it without typing its number.
- 6.7 Explain why `$$` and `eqnarray` are discouraged.
- 6.8 Challenge: typeset and define one equation from your discipline, then ask a colleague whether the notation is unambiguous.

6.13 Complete Practical Coding

The complete file below combines inline, displayed, numbered, and aligned mathematics.

Complete practical file: essential mathematics

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}

For observations  $x_1, \ldots, x_n$ , the arithmetic mean is
\begin{equation}
\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.
\label{eq:mean}
\end{equation}
```

For paired measurements, define

```
\begin{align}
d_i &= x_i - y_i, \\
\overline{d} &= \frac{1}{n} \sum_{i=1}^n d_i, \\
\operatorname{MAE} &= \frac{1}{n} \sum_{i=1}^n |d_i|.
\end{align}
```

Equation~\ref{eq:mean} is numbered automatically. A square root can be written inline as $s = \sqrt{s^2}$.

```
\end{document}
```

Code Lab

Add a weighted mean, refer to it by label, then insert another equation before it and confirm that the printed reference remains correct.

Advanced Mathematical Writing

This optional chapter is for documents that need derivations, piecewise definitions, matrices, calculus, or theorem-like statements. Readers whose work uses only short equations may continue to quantities and units and return here when a project requires these structures.

Learning outcomes
• Break a long derivation into aligned, readable lines. • Typeset cases, matrices, and vectors. • Write derivatives, integrals, sums, and limits. • Define theorem-like environments with <code>amsthm</code>. • Produce a definition, theorem, and proof with consistent numbering.

7.1 Break mathematics at relationships

A long equation should not be squeezed by reducing its type size. Break it at a meaningful relation and use `aligned` inside one numbered equation when the lines form one logical statement:

One numbered multi-line derivation
<pre>\begin{equation} \begin{aligned} &\sum_{i=1}^n (d_i - \bar{d})^2 \\ &\quad \&= \sum_{i=1}^n (d_i^2 - 2d_i \bar{d} + \bar{d}^2) \quad \backslash\backslash \\ &\quad \&= \sum_{i=1}^n d_i^2 - n \bar{d}^2. \end{aligned} \end{equation}</pre>

The outer environment supplies one number; `aligned` supplies alignment without separate numbers. Use `align` instead when individual lines need separate labels or stand as distinct equations.

Pause and Predict
If the outer <code>equation</code> is replaced by <code>\[... \]</code>, will the derivation keep a visible number? Predict, then test.

7.2 Cases and matrices

The `cases` environment creates a left brace and two aligned columns:

$$I_i = \begin{cases} 1, & \text{if both simulated values are present,} \\ 0, & \text{otherwise.} \end{cases} \quad (7.1)$$

Matrix environments differ mainly in delimiters:

Matrices and a vector

```
\[
\mathbf{X} =
\begin{bmatrix}
1 & A_1 \\
1 & A_2 \\
\vdots & \vdots \\
1 & A_n
\end{bmatrix},
\quad
\boldsymbol{\beta} =
\begin{pmatrix}
\beta_0 \\
\beta_1
\end{pmatrix}.
\]
```

`matrix` has no delimiters; `pmatrix` uses parentheses; `bmatrix` uses square brackets. Ampersands separate columns and `\\` separates rows. Every row should have a consistent number of columns.

`\mathbf` makes Latin letters upright bold. `\boldsymbol`, supplied by `amsmath`, also handles many Greek symbols. Vector typography varies by discipline; define and apply one convention consistently.

Common Error

Symptom: “Extra alignment tab has been changed to `\cr`.”

Cause: one matrix row has too many ampersands, so it declares more columns than the structure expects.

Repair: count entries and ampersands in each row. A two-column row has one ampersand.

7.3 Calculus and large operators

L^AT_EX source mirrors the hierarchy of standard notation:

$$\begin{aligned}\frac{dy}{dx} &= 2x, \\ \frac{\partial f}{\partial x} &= 2x + y, \\ \int_0^T v(t) dt &= s(T) - s(0), \\ \sum_{i=1}^n x_i &\longrightarrow n\mu, \\ \lim_{n \rightarrow \infty} \bar{x}_n &= \mu.\end{aligned}$$

This book uses an upright differential d , with a thin space $\,$, before it in an integral. That is a documented style choice, not a universal L^AT_EX requirement. The commands `\partial`, `\int`, `\sum`, `\lim`, `\infty`, and `\to` express the symbols and operators directly.

7.4 Definitions, theorems, and proofs

Load `amsthm` and define theorem structures in the preamble, not in the middle of the argument:

Preamble theorem definitions

```
\usepackage{amsthm}

\theoremstyle{definition}
\newtheorem{definition}{Definition}[section]
\theoremstyle{plain}
\newtheorem{theorem}[definition]{Theorem}
```

The optional `[section]` resets definition numbers in each section. Sharing `[definition]` gives definitions and theorems one common sequence. In the body:

Definition 7.1 (Paired difference). For paired values A_i and B_i , the signed paired difference is $d_i = B_i - A_i$.

Theorem 7.2. *If $A_i = B_i$ for every complete pair, then the mean signed difference is zero.*

Proof. For every complete pair, $d_i = B_i - A_i = 0$. Therefore the sum of all differences is zero, and division by the positive number of complete pairs gives $\bar{d} = 0$. $\square$

The proof environment supplies its heading and end mark. A mathematical result can be typeset elegantly and still be trivial, false, or irrelevant; proof and peer review remain intellectual tasks.

Science in Practice

In computer science, a lemma may establish one property used by an algorithm’s correctness theorem. In statistics, assumptions may be stated as definitions or conditions rather than theorems. Use theorem structures only when they match the discipline’s reasoning, not to make ordinary observations look more authoritative.

7.5 Numbering by section

Numbering that resets by section helps readers locate results in long mathematical documents. In a short article, a single sequence may be simpler. The optional argument to `\newtheorem` defines the relationship; do not manually type “Theorem 2.3”.

Labels work with theorem-like environments. Place a label immediately after the environment begins and refer to it through the same reference system used for equations and figures.

Good Practice

Before choosing an elaborate notation, write down what each object means, its dimensions, its allowed values, and the convention for bold, italic, and upright symbols. Consistent simple notation is easier to review than ornate notation introduced without need.

7.6 Code lab: one mathematical page

Code Lab

Compile this page before changing it. Then replace the matrix entries, add a third case, and change the theorem’s section number by inserting a new section above it. Notice which numbers update without manual editing.

Equations, a matrix, and a theorem in one file

```
\documentclass{article}
\usepackage{amsmath,amsthm}
\newtheorem{theorem}{Theorem}[section]

\begin{document}
\section{A compact model}
\begin{equation}
A=\begin{pmatrix}1&2\\3&4\end{pmatrix},
\quad
f(x)=\begin{cases}
```



```

      x^2, & x\geq 0,\\
      -x, & x<0.
\end{cases}
\end{equation}

\begin{theorem}
If  $n$  is even, then  $n^2$  is even.
\end{theorem}
\begin{proof}
Write  $n=2k$ . Then  $n^2=2(2k^2)$ .
\end{proof}
\end{document}

```

7.7 Chapter project

Mini Project

Optional extension: add Equation (7.1) in your own source as an indicator for complete pairs, then state in prose how incomplete pairs are handled. If repeated measurements form a matrix, give its dimensions. Do not add theorem structures to the running article unless it actually proves a general statement.

7.8 Summary

`aligned` can break one numbered derivation; `align` can number separate related lines. `cases` expresses piecewise definitions. Matrix environments use ampersands for columns and row breaks for rows. Structured commands `typeset` calculus and large operators. `amsthm` provides semantic definition, theorem, lemma, and proof structures whose numbering is declared once in the preamble.

7.9 Checklist

- ☐ Long derivations break at meaningful mathematical relationships.
- ☐ Matrix rows have consistent column counts.
- ☐ Vector and differential typography follows a stated convention.
- ☐ Every theorem-like structure has a scholarly purpose.
- ☐ Numbering comes from environment definitions, not typed labels.

7.10 Exercises

7.1 Put a two-line derivation under one equation number.

7.2 Create a piecewise function with two cases and explanatory text.

- 7.3 Typeset a two-by-two matrix in parentheses and in square brackets.
- 7.4 Repair a matrix row containing too many ampersands.
- 7.5 Write an integral with limits and an upright differential.
- 7.6 Define lemmas and theorems in one numbering sequence.
- 7.7 Explain when numbering by section helps and when it adds clutter.
- 7.8 Challenge: typeset a short proof or derivation from your field and ask whether every step is both mathematically and typographically clear.

7.11 Complete Practical Coding

This file adds cases, a matrix, and a theorem without building a custom macro system.

Complete practical file: advanced mathematics

```
\documentclass{article}
\usepackage{amsmath}
\usepackage{amsthm}

\newtheorem{proposition}{Proposition}

\begin{document}

Define the absolute-value function by
\[
|x|=\begin{cases}
x, & x\geq 0, \\
-x, & x<0.
\end{cases}
\]

The transformation
\[
\begin{bmatrix}u\\v\end{bmatrix}
=
\begin{bmatrix}a&b\\c&d\end{bmatrix}
\begin{bmatrix}x\\y\end{bmatrix}
\]
uses ordinary matrix notation.

\begin{proposition}
For every real number  $x$ ,  $|x|\geq 0$ .
\end{proposition}
```

```
\begin{proof}
The result follows directly from the two cases in the definition.
\end{proof}

\end{document}
```

Code Lab

Add a three-case function and a second proposition. Keep each display inside the sentence that introduces or explains it.

Quantities, Units, and Scientific Conventions

A value without a clear unit can make a result unusable. A value with an inconsistent decimal mark or ambiguous uncertainty can make a table difficult to audit. The `siunitx` package separates a number from its unit, applies a consistent convention, and aligns numerical columns.

Learning outcomes

- Format numbers and scientific notation with `siunitx`.
- Typeset quantities, ranges, uncertainties, angles, and percentages.
- Align numerical values at the decimal marker.
- Distinguish measurement precision from displayed decimal places.
- Apply concise conventions for chemistry, taxonomy, constants, vectors, and matrices.

8.1 Load `siunitx` and separate roles

Load the package in the preamble. Use `\num` for a number and `\qty` for a number with a unit:

Complete example: numbers and quantities

```
\documentclass{article}
\usepackage{siunitx}
\begin{document}
The count was \num{12500}. The distance was
\qty{12.4}{\milli\metre}, and the rate was
\qty{3.2e-4}{\metre\per\second}.
\end{document}
```

`\num{12500}` groups digits according to the package settings. `\qty` receives a numerical argument and a unit argument. The unit is built from semantic commands: `\milli\metre`; division is expressed by `\per`. Input `3.2e-4` is formatted as scientific notation.

Do not type an italic `m` in math mode to imitate metre. A unit is upright and its spacing from the number follows a convention.

8.2 Ranges and lists

Use dedicated commands rather than inserting a hyphen between two complete quantities:

Ranges and lists

The interval was `\qtyrange{4.2}{7.8}{\milli\metre}`.

The counts were `\numlist{12;15;19}`.

Temperatures ranged from
`\qty{-5}{\degreeCelsius}` to `\qty{18}{\degreeCelsius}`.

The semicolon separates items in `\numlist` input. The output punctuation can adapt to the document language and package configuration.

Pause and Predict

Compare typed text 5-10 mg with `\qtyrange{5}{10}{\milli\gram}`. Which version gives L^AT_EX explicit knowledge of both the range and the unit?

8.3 Uncertainty and significant figures

Parenthetical uncertainty input places the uncertainty in the final digits:

Quantities with uncertainty

The estimated length was `\qty{12.4(3)}{\milli\metre}`.

The mean difference was
`\qty[separate-uncertainty=true]{0.42(15)}{\milli\metre}`.

The first value represents 12.4 ± 0.3 millimetres. The option in the second command requests an explicit plus-minus form. Confirm the uncertainty notation used by your field and journal; do not infer measurement quality from the number of digits alone.

Rounding is an analytical and reporting decision. `siunitx` can round consistently, but it cannot decide how many significant figures the evidence supports. Preserve unrounded values in the analysis and apply a documented display rule.

8.4 Percentages, angles, and degrees

Percent and angle examples

Coverage was `\qty{95}{\percent}`.

The angle was `\ang{23.5}`.

The temperature was `\qty{20.0}{\degreeCelsius}`.

`\ang` accepts an angular value; `\degreeCelsius` is a complete unit. Do not attach a degree sign manually to a Celsius value when a semantic unit is available.

8.5 Decimal alignment in tables

An `S` column parses numbers and aligns them at the decimal marker:

Table 8.1. Simulated paired differences, shown only to demonstrate decimal alignment.

Pair	Difference (mm)
P01	0.42
P02	−1.30
P03	12.05

Braces around the column heading protect it from numerical parsing. The `table-format=2.2` reservation allows up to two integer and two decimal digits; a minus sign is accommodated when present. Inspect the real range before choosing a format.

Common Error

Symptom: a text entry in an `S` column produces a parsing error or poor alignment.

Cause: the column expects numerical input.

Correction: protect a short heading with braces, or give textual rows a suitable non-numeric column. Do not disguise missing data as an unexplained number.

8.6 Optional discipline-specific conventions

8.6.1 Chemical formulae and reactions

The `mhchem` package provides `\ce` for chemical notation:

Optional chemistry source

```
\usepackage[version=4]{mhchem}
```

Water is `\ce{H2O}`, and a reaction may be written as
`\ce{2H2 + O2 -> 2H2O}`.

This fragment shows preamble and body lines, not a complete document. Check the current `mhchem` manual for supported reaction arrows, charges, isotopes, and the required version option.

8.6.2 Species names and sequences

Use italics for a genus-species binomial as the disciplinary convention requires: *Arabidopsis thaliana*. A DNA sequence such as `ACGTTGCA` is data-like text; a monospaced font can distinguish its fixed symbols, but long sequences need line breaking, numbering, and often a specialised package or supplementary file.

8.6.3 Physical constants, vectors, and matrices

State whether a value is exact by definition or measured, give its unit, and cite the authoritative source when appropriate. Use one declared vector convention, such as **v**, and a matrix convention such as **A**. Do not alternate bold, arrowed, and plain vectors without meaning.

Science in Practice

An engineering tolerance 25.0(5)mm, an environmental concentration 8.2 µg L⁻¹, and a physical speed 3.0 × 10⁸ m s⁻¹ share one source grammar while retaining their disciplinary meanings.

Good Practice

Define units in column headings when every value shares the unit. Do not repeat the unit in every table cell. Explain missing values, uncertainty, rounding, and transformations in the caption or a table note.

8.7 Chapter project**Mini Project**

Load `siunitx` in the running article. Declare that both methods report millimetres. Format the example range, mean difference, and uncertainty with `\qty`. Add a three-row simulated decimal-aligned table as a temporary methods check; Chapter 9 will turn it into a complete publication table.

8.8 Summary

siunitx formats numbers separately from units and supplies commands for quantities, ranges, lists, uncertainties, percentages, angles, and scientific notation. Its `S` column aligns parsed numbers. Formatting consistency is not a substitute for justified significant figures. Optional tools such as `mhchem` express discipline-specific notation; taxonomy, sequences, constants, vectors, and matrices also require declared conventions.

8.9 Checklist

- ☐ Every reported measurement has an unambiguous unit.
- ☐ Ranges and uncertainties use semantic commands.
- ☐ Display rounding follows a stated analytical rule.
- ☐ Numerical table columns align at the decimal marker.
- ☐ Discipline-specific notation follows a verified convention.
- ☐ Simulated quantities remain labelled as demonstrations.

8.10 Exercises

- 8.1 Typeset 1,250,000 and 2.5×10^{-6} m.
- 8.2 Create a range from 4 to 9 milligrams.
- 8.3 Express 7.2 ± 0.4 centimetres using uncertainty input.
- 8.4 Make an `S` column for values from -12.30 to 8.05 .
- 8.5 Explain why decimal places do not by themselves establish precision.
- 8.6 Typeset a Celsius temperature and a percentage.
- 8.7 Identify the role of `\ce` and the package that supplies it.
- 8.8 Challenge: audit a table from your field for units, rounding, uncertainty, and missing-value definitions.

8.11 Complete Practical Coding

Use `siunitx` so numbers and units remain separate in the source.

Complete practical file: quantities and units
<pre>\documentclass{article} \usepackage{booktabs}</pre>


```

\usepackage{siunitx}

\sisetup{separate-uncertainty=true}

\begin{document}

The sample mass was \qty{12.5}{\milli\gram}, the temperature was
\qty{22}{\degreeCelsius}, and the interval was
\qtyrange{18}{24}{\hour}.

\begin{table}[htbp]
  \centering
  \caption{Demonstration measurements.}
  \begin{tabular}{lS[table-format=2.1]S[table-format=1.1]}
    \toprule
    Group & {Mean (mg)} & {SD (mg)} \\
    \midrule
    A & 12.4 & 1.8 \\
    B & 16.7 & 2.1 \\
    \bottomrule
  \end{tabular}
\end{table}

An uncertainty can be written as \qty{12.4(18)}{\milli\gram}.

\end{document}

```

Code Lab

Replace the quantities with values from your field, check significant figures, and align a column containing at least one two-digit value.

Part IV

Tables, Figures, and Cross-References

Creating Professional Tables

A table is not a storage container for every recorded value. It is an argument in rows and columns: the reader should understand what is being compared, which unit applies, and what each summary means. We begin with the smallest grid and add only the structures required for a scholarly table.

Learning outcomes

- Build a `tabular` with rows, columns, and alignment.
- Place a table, caption, and label in a float.
- Use `booktabs` rules and avoid unnecessary vertical lines.
- Create flexible and multi-line columns with `tabularx`.
- Align numerical columns with `siunitx`.
- Prepare table notes and a multi-page `longtable`.
- Explain a safe optional route from CSV data to a checked table.

9.1 The smallest tabular

The `tabular` environment aligns cells. Its required argument describes the columns: `l`, `c`, and `r` mean left, centred, and right.

Complete example: a two-column tabular

```
\documentclass{article}
\begin{document}
\begin{tabular}{lr}
Method & Complete pairs \\
A & 12 \\
B & 12
\end{tabular}
\end{document}
```

Ampersands separate cells; ends a row. Every row needs the same column logic. The environment aligns content but supplies neither a caption nor a number.

Common Error

Symptom: “Extra alignment tab has been changed to `\cr`.”

Cause: a row contains too many ampersands for the declared columns.

Repair: for two columns, confirm that each row has one ampersand.

9.2 Rules that organise rather than cage

Load `booktabs` and use `\toprule`, `\midrule`, and `\bottomrule`:

Table 9.1. Descriptive summary of simulated paired measurements.

Method	Mean (mm)	SD (mm)
A	5.68	2.71
B	5.74	2.70

The `@{}` at each edge removes extra outer padding. The three rules show the table boundary and distinguish the header; they do not draw a box around every cell. Vertical rules and a rule after every row usually interrupt the horizontal reading path.

9.3 The table float, caption, and label

The `table` environment is a *float*: an object $\LaTeX$ may move to a good page position. The optional `[htbp]` offers here, top, bottom, or a float page; it is a preference list, not a command to violate page design.

Place `\caption` before `\label` so the label records the table number. Use a descriptive name such as `tab:descriptive-summary`. Refer to the table from the prose so the reader knows why it matters.

Good Practice

A caption should identify the population or material, variables, units, and important qualifications. Do not use “Results” as a caption. Keep detailed interpretation in the prose rather than hiding it beneath the table.

9.4 Flexible text columns

The `tabularx` environment takes a width and lets X columns share the remaining space:

Complete example: a descriptive table

```
\documentclass{article}
\usepackage{booktabs,tabularx}
```

```
\begin{document}
\begin{tabularx}{\textwidth}{@{}lX@{}}
\toprule
Item & Description \\
\midrule
Identifier & Stable code assigned to one simulated pair. \\
Difference & Method B minus Method A, reported in millimetres. \\
\bottomrule
\end{tabularx}
\end{document}
```

For a fixed-width paragraph column, use `p{width}`. It wraps text and top-aligns the cell. Choose the width from the page and content, then inspect the result; a table with many paragraph columns may be better redesigned.

9.5 A statistical table

Table 9.2. Illustrative analysis of simulated differences. Confidence limits are included only to demonstrate structure.

Quantity	Estimate	Lower limit	Upper limit
Mean difference (mm)	0.06	−0.18	0.30
SD of differences (mm)	0.31		

Note. Values are simulated. The interval is not evidence about a real instrument or population.

Short table notes can sit in a minipage aligned with the table. A journal class may supply a dedicated notes environment; prefer it when available. An empty braced cell protects an S column from non-numeric content.

9.6 Tables across pages

Use `longtable` for a table that must break across pages. It is not placed inside a table float.

Longtable pattern with repeated headings

```
\begin{longtable}{@{}lSS@{}}
\caption{Complete simulated measurements.}\label{tab:all-pairs}\\
\toprule
{Pair} & {Method A (mm)} & {Method B (mm)} \\
\midrule
\endfirsthead
\multicolumn{3}{\tablecaption\thetable continued}\\
\toprule
{Pair} & {Method A (mm)} & {Method B (mm)} \\
```

```

\midrule
\endhead
\midrule
\multicolumn{3}{r}{Continued on next page}\\
\endfoot
\bottomrule
\endlastfoot
P01 & 1.20 & 1.40 \\
P02 & 2.10 & 1.90 \\
% Continue with verified or explicitly simulated rows.
\end{longtable}

```

The first head appears once; the later head repeats. Foot definitions control continuation text and the final rule. Test with enough rows to force a page break, and confirm that the caption, number, header, and notes remain clear.

9.7 Optional: data from CSV

Copying a large dataset into source invites transcription errors. Packages can read CSV files, or a reproducible analysis script can export a finished `.tex` table. This is an advanced workflow because imported values still need validation, rounding, escaping, captions, and version control.

Advanced Topic

Keep raw data separate from presentation. Record the script and software version that created the table. Inspect the generated source and PDF. Never assume that successful import proves that columns have the intended units or that missing values were handled correctly.

Science in Practice

A geological sample table may need sample code, location, lithology, depth, and analytical result. If descriptions dominate, use flexible text columns; if measurements dominate, use decimal-aligned columns. If the full record is very long, show a concise analytical table in the chapter and place the complete record in an appendix or repository.

9.8 Code lab: turn observations into a table

Code Lab

Compile the complete file, replace one simulated value, and add a third method. Then remove the caption’s word “Simulated” and explain why that would be scientifically misleading even though the source still compiles.

A compact, publication-style numerical table

```
\documentclass{article}
\usepackage{booktabs}
\usepackage{siunitx}

\begin{document}
\begin{table}[htbp]
  \centering
  \caption{Simulated recovery measurements.}
  \label{tab:recovery}
  \begin{tabular}{l S[table-format=2.1] S[table-format=1.2]}
    \toprule
    Method & {Mean (\%)} & {SD} \\
    \midrule
    A & 91.4 & 2.10 \\
    B & 94.8 & 1.73 \\
    \bottomrule
  \end{tabular}
\end{table}
\end{document}
```

9.9 Chapter project

Mini Project

Replace the temporary numerical check with a numbered table containing the simulated Method A mean, Method B mean, and mean difference. Use `booktabs`, `S` columns, one shared unit in each heading, and a note stating that all values are simulated. Label and mention the table in Results.

9.10 Summary

`tabular` aligns cells; a `table` float adds placement, caption, and numbering. `booktabs` uses a small hierarchy of horizontal rules. `tabularx` and `paragraph` columns manage text; `siunitx` aligns numbers. Notes define qualifications without replacing results prose. `longtable` supports repeated headings across pages. Imported tables remain subject to validation and visual inspection.

9.11 Checklist

- ☐ Each row has the declared number of columns.
- ☐ The caption says what the table contains and which data are simulated.
- ☐ The label follows the caption and has a `tab:` prefix.
- ☐ Units, rounding, missing values, and notes are unambiguous.
- ☐ Numerical columns align at the decimal marker.
- ☐ I did not use vertical rules merely by habit.

9.12 Exercises

- 9.1 Build a two-column table with three data rows.
- 9.2 Diagnose a row with one ampersand too many.
- 9.3 Add `booktabs` rules and remove vertical rules.
- 9.4 Convert a long description column to `tabularx`.
- 9.5 Align positive and negative values with an `S` column.
- 9.6 Write a caption and note for explicitly simulated environmental data.
- 9.7 Explain why `longtable` is not placed inside `table`.
- 9.8 Challenge: redesign a real table from your field for one clear reader task and document every omitted column.

9.13 Complete Practical Coding

This complete example covers a compact numeric table and a wrapping text table. Both use captions, labels, and restrained horizontal rules.

Complete practical file: publication-style tables

```
\documentclass{article}
\usepackage{booktabs}
\usepackage{tabularx}
\usepackage{siunitx}

\begin{document}

\begin{table}[htbp]
  \centering
  \caption{Summary of two demonstration groups.}
  \label{tab:summary}
```

```
\begin{tabular}{lS[table-format=2.1]S[table-format=1.1]}
\toprule
Group & {Mean} & {SD} \\
\midrule
A & 12.4 & 1.8 \\
B & 16.7 & 2.1 \\
\bottomrule
\end{tabular}
\end{table}

\begin{table}[htbp]
\centering
\caption{Observations requiring wrapped text.}
\label{tab:notes}
\begin{tabularx}{\linewidth}{@{}lXr@{}}
\toprule
Code & Observation & Count \\
\midrule
A & Response remained stable during observation. & 12 \\
B & Response increased after treatment. & 15 \\
\bottomrule
\end{tabularx}
\end{table}

Tables~\ref{tab:summary} and~\ref{tab:notes} serve different jobs.

\end{document}
```

Code Lab

Add a third row, lengthen one observation, and change one numeric value to three digits. Adjust the declared **S** format instead of shrinking text.

Adding and Managing Figures

A figure should let the reader see evidence, a process, or a relationship that would be harder to grasp in prose. $\text{\LaTeX}$ does not create most scientific plots; it places a prepared graphic, supplies a caption and number, and integrates the figure with the document.

Learning outcomes

- Distinguish raster and vector graphics and choose suitable formats.
- Include an image from a relative path and scale it safely.
- Build a numbered figure with caption and label.
- Explain float placement without treating movement as an error.
- Create a multi-panel figure with `subcaption`.
- Check resolution, colour, grayscale, and caption accessibility.

10.1 Prepare the graphic before placing it

A *raster image* stores a grid of pixels. Photographs, microscopy images, and scans are naturally raster; PNG and JPEG are common formats. A *vector graphic* stores geometric paths and text. Diagrams, line drawings, and many statistical plots remain sharp when scaled; PDF is the most convenient vector format for pdf $\text{\LaTeX}$.

Use JPEG for continuous-tone photographs when appropriate, PNG for lossless raster graphics, and PDF for vector artwork. Do not convert a low-resolution JPEG to PDF and call it vector; the pixels remain.

Good Practice

Export plots at their intended publication dimensions with readable labels. Keep editable plot code or source beside the export. Do not enlarge a small raster image in $\text{\LaTeX}$ and expect new detail to appear.

10.2 The smallest included graphic

Load `graphicx`, place the file inside the project, and use a relative path:

Complete project example: include one PDF graphic

```
\documentclass{article}
\usepackage{graphicx}
\begin{document}
\includegraphics[width=0.75\linewidth]{method_comparison.pdf}
\end{document}
```

This complete source assumes the original file `method_comparison.pdf` is beside `main.tex`. The optional argument sets the displayed width to 75% of the current line width; the height changes proportionally. Do not give both width and height unless distortion is intended.

10.3 Caption, label, and float

Wrap the graphic in a `figure` environment:

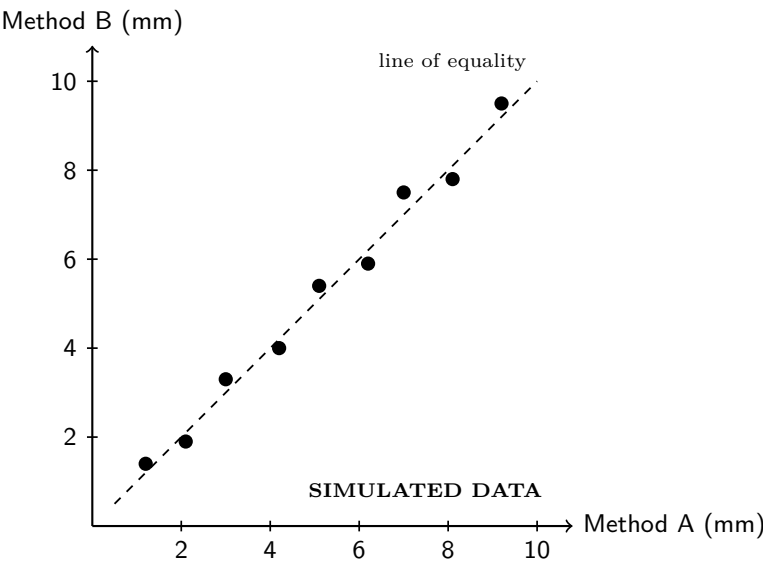


Figure 10.1. Relationship between two methods for simulated paired values. The dashed line marks equality; points above it have larger Method B values.

The source uses `\centering` inside the float, includes the graphic, writes an informative caption, and labels the number produced by the caption. The figure is original and the graphic itself states that the data are simulated.

Science in Practice

The dashed line in Figure 10.1 provides a reference, but a cloud near that line is not a complete agreement analysis. A figure must not imply a stronger

scientific conclusion than the method supports.

10.4 Why a figure moves

A float is allowed to move so that L^AT_EX can avoid large gaps, stranded captions, and figures cut by page boundaries. The placement letters **h**, **t**, **b**, and **p** mean here, top, bottom, and float page. L^AT_EX considers them with size and page constraints.

Place a figure near its first discussion in source and refer to it by label. If it moves one page, the reference remains correct. Repeatedly adding stronger placement commands can make the surrounding pages worse.

Common Error

Symptom: a figure appears later than its source position.
Cause: insufficient legal space remained on the page, or earlier floats were waiting.
Response: confirm that the graphic is sensibly sized and placed near its first mention. Inspect nearby floats. Treat movement as layout behaviour, not automatically as failure.

10.5 Multi-panel figures

Load subcaption. Each **subfigure** gets its own caption and label, while the outer figure gets the overall caption and number.

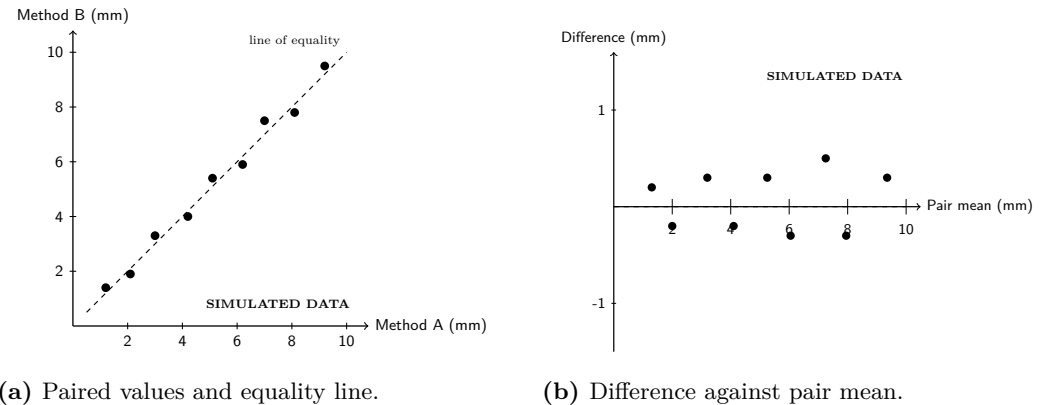


Figure 10.2. Two views of the same simulated teaching data. Panel (a) shows the original paired scale; panel (b) makes signed differences visible.

Panel labels should be referenced automatically. The overall caption explains why the panels belong together and defines visual encodings that are not obvious. Make axis type, units, and sample size visible in the figure or caption as the target style requires.

10.6 Landscape and rotated material

Large maps or wide tables may need a landscape page. The `pdflscape` package provides a `landscape` environment and rotates the PDF view for a reader:

Optional landscape pattern
<pre>\usepackage{pdflscape} \begin{landscape} % Place one genuinely wide figure or table here. \end{landscape}</pre>

This is a preamble fragment followed by a body fragment. Do not rotate a figure merely because its labels were exported too small. First redesign the graphic. Publisher systems may request landscape figures as separate files.

10.7 Quality and accessibility checks

Before submission, inspect the graphic at 100% PDF view and on a printed test if print matters. Check:

- labels, symbols, and lines at final size;
- units and definitions of abbreviations;
- raster resolution against current venue instructions;
- embedded or outlined fonts where required;
- colour combinations in grayscale and for colour-vision differences;
- distinction by shape, line style, or label rather than colour alone;
- a caption that communicates the figure’s purpose and evidence.

An informative caption assists all readers, but it is not a complete substitute for tagged-PDF alternative text in an accessible publishing workflow. Ask the publisher which accessibility metadata and source they accept.

Submission Check
Open every final exported figure rather than trusting its filename. Confirm that no crop removed labels, no panel order changed, and the manuscript cites the correct file. Keep permission and licence records for every non-original asset.

10.8 Code lab: a figure that still compiles during drafting

Code Lab

Place a file named `measurement-methods.pdf` beside this source and compile. Rename the image temporarily and compile again. The framed fallback keeps the draft buildable while making the missing asset visible.

A complete figure document with a visible fallback

```
\documentclass{article}
\usepackage{graphicx}

\begin{document}
\begin{figure}[htbp]
  \centering
  \IfFileExists{measurement-methods.pdf}{%
    \includegraphics[width=.82\linewidth]
      {measurement-methods.pdf}%
  }{%
    \fbox{\rule{0pt}{45mm}\rule{.78\linewidth}{0pt}}%
  }
  \caption{Agreement between two simulated measurement methods.}
  \label{fig:agreement}
\end{figure}

Figure~\ref{fig:agreement} is referenced from the prose.
\end{document}
```

10.9 Chapter project

Mini Project

Add the supplied original `method_comparison.pdf` to the running article with a caption that says the values are simulated. Add the two-panel figure as a later diagnostic. Label the outer figure and both panels. Mention each figure from Results without typing a number. Record the editable TikZ sources and their original status in the asset log.

10.10 Summary

Raster graphics store pixels; vector graphics store scalable geometry. `graphicx` includes files from relative paths and scales them proportionally. A `figure` float integrates placement, caption, number, and label. Movement is normal page design. `subcaption` organises panels; `pdfscape` can support genuinely wide material. Publication checks cover resolution, labels, units, grayscale, accessibility, provenance, and

captions.

10.11 Checklist

- ☐ The chosen format suits photographic or line-based content.
- ☐ The file path is relative and portable.
- ☐ Caption and label follow the included graphic.
- ☐ Text and lines remain legible at final size.
- ☐ Meaning does not depend on colour alone.
- ☐ Original or licensed provenance is recorded.

10.12 Exercises

- 10.1 Classify a photograph, logo, line plot, and scanned map as primarily raster or vector.
- 10.2 Include a PDF graphic at half the line width.
- 10.3 Add a caption and correctly positioned label.
- 10.4 Explain why the [h] request may not place a float exactly here.
- 10.5 Build two subfigures with an overall caption.
- 10.6 Identify two ways to preserve meaning in grayscale.
- 10.7 Diagnose a “file not found” error caused by a machine-specific path.
- 10.8 Challenge: audit one figure for evidence, resolution, accessibility, caption completeness, and provenance.

10.13 Complete Practical Coding

The panels below are generated with ordinary rules, so the file is complete without an external image. Replace each framed panel with `\includegraphics` when using real artwork.

Complete practical file: aligned figure panels
<pre>\documentclass{article} \usepackage{graphicx} \usepackage{subcaption} \newcommand{\demopanel}[1]{% \fbox{\parbox[c][35mm][c]{.88\linewidth}{\centering #1}}}</pre>


```

\begin{document}

\begin{figure}[htbp]
  \centering
  \begin{subfigure}[t]{.47\linewidth}
    \centering
    \demopanel{Condition A}
    \caption{Baseline condition.}
    \label{fig:condition-a}
  \end{subfigure}\hfill
  \begin{subfigure}[t]{.47\linewidth}
    \centering
    \demopanel{Condition B}
    \caption{Treatment condition.}
    \label{fig:condition-b}
  \end{subfigure}
  \caption{Two aligned demonstration panels with one shared caption.}
  \label{fig:conditions}
\end{figure}

Figure~\ref{fig:conditions} contains two top-aligned panels.

\end{document}

```

Code Lab

Change one subcaption to two lines and verify that the panels stay aligned. Then substitute two project-relative image paths and inspect the result at the book's intended print size.

Labels, References, and Automatic Numbering

Numbers are consequences of structure. When a new table is inserted, later tables should renumber and every mention should follow. Labels give document elements stable names so the source can refer to meaning rather than a number that may change.

Learning outcomes

- Attach labels to sections, equations, tables, and figures.
- Use `\ref`, `\pageref`, and `\eqref`.
- Create readable automatic references with `cleveref`.
- Add links and PDF metadata with `hyperref`.
- Diagnose undefined and duplicate labels.
- Replace hard-coded scholarly numbers throughout a project.

11.1 A label is a permanent source name

Attach a label after the command that establishes the number:

Labels in common structures

```
\section{Methods}
\label{sec:methods}

\begin{equation}
  d_i=B_i-A_i.
  \label{eq:paired-difference}
\end{equation}

\begin{figure}[htbp]
  \centering
  \includegraphics{plot.pdf}
  \caption{Simulated paired values.}
  \label{fig:paired-values}
\end{figure}
```

For figures and tables, the caption steps the counter, so the label follows the

caption. For a section, place the label immediately after the heading command.

Use consistent prefixes: `chap:`, `sec:`, `fig:`, `tab:`, and `eq:`. The rest should describe content, not a temporary number. `fig:difference-plot` survives renumbering; `fig:figure-2` becomes misleading.

11.2 Basic references

`\ref` prints a number and `\pageref` prints its page:

Basic reference forms

See Figure~`\ref{fig:paired-values}` on
page~`\pageref{fig:paired-values}`.

Equation~`\eqref{eq:paired-difference}` defines the difference.

The non-breaking space keeps the name and number together. `\eqref` from `amsmath` supplies parentheses around an equation number. The word “Figure” is still typed in this basic form.

11.3 Readable references with `cleveref`

Load `hyperref` and then `cleveref`. The latter can infer the element type from the label:

Intelligent reference forms

As shown in `\cref{fig:paired-values}`, the values are paired.

`\Cref{tab:descriptive-summary,tab:statistical-summary}`
summarise the simulated data.

`\cref` is suitable inside a sentence; `\Cref` capitalises the generated name at the beginning. Multiple labels can produce a compact range or list according to the reference type and language settings.

Good Practice

Write prose that remains grammatical if the number changes. Avoid beginning every sentence “Figure X shows”. Explain the scientific point, then direct the reader to the evidence.

11.4 Links and metadata

`hyperref` turns cross-references, citations, contents entries, and URLs into links. Load it late in the preamble, then set metadata explicitly:

PDF metadata pattern

```
\usepackage{hyperref}
\hypersetup{
  pdftitle={A Reproducible Comparison of Two Measurement Methods},
  pdfauthor={Author Name},
  pdfsubject={Simulated teaching article},
  pdfkeywords={measurement, reproducibility, simulated data}
}
```

Metadata is not a substitute for visible title information. Remove draft authors, old titles, and software defaults before submission. A journal may rebuild the PDF and replace metadata.

Link colour choices must remain readable in print and grayscale. Hidden links can be appropriate for formal submission, while coloured links help an electronic teaching document. Treat that as a design choice governed by the class or venue.

11.5 Why references need repeated compilation

On one pass, L^AT_EX writes the label and its number to an auxiliary file. On a later pass, it reads that number into the reference. A new reference may first appear as question marks. `latexmk` repeats the build until the files stabilise.

Common Error

Symptom: “Reference ... undefined” and ?? in the PDF.

Checks: compare label spelling and capitalisation; confirm the label exists; compile again; inspect whether an earlier fatal error prevented the auxiliary file from being written.

Do not replace ?? with a typed number. That hides the broken connection.

Two elements must not share one label. A duplicate label can silently send a reference to the wrong element while producing a multiply-defined warning. Search the project for the exact label and rename one meaningfully.

Pause and Predict

Insert a new figure before Figure 10.1. Predict what happens to the visible figure number and to every `\cref` pointing to its label. What would happen to text that literally said “Figure 1”?

11.6 A cross-reference audit

Search the source for typed constructions such as “Figure 2”, “Table 1”, or “Section 3.2”. Some may be quotations or fixed external identifiers, but internal references should usually use labels. Then check:

- every numbered element has at most one unique label;
- every label is referenced or intentionally reserved;
- labels follow the project’s naming convention;
- no reference points to an obsolete element;
- link text makes sense to a reader and not only to the author.

Science in Practice

An engineering report may cite a requirement called “Clause 7.3” in an external standard. That external identifier is text, not a $\text{\LaTeX}$ label. By contrast, “Section 7.3 of this report” should be automatic. Distinguish fixed external names from internal numbers.

11.7 Code lab: rename without renumbering

Code Lab

Compile this file twice. Insert a new equation before the labelled one and compile twice again. The prose should follow the new equation number without you touching it. Next, rename the label in only one place to create an undefined reference; repair both occurrences together.

A reference that survives revision

```
\documentclass{article}
\usepackage{amsmath}
\usepackage[colorlinks=true]{hyperref}
\usepackage[nameinlink]{cleveref}

\begin{document}
\section{Calibration}\label{sec:calibration}
The conversion used in \cref{sec:calibration} is
\begin{equation}\label{eq:conversion}
y=1.08x+0.12.
\end{equation}
We apply \cref{eq:conversion} to every simulated observation.
\end{document}
```

11.8 Chapter project

Mini Project

Give every section, equation, table, figure, and appendix in the running article a descriptive unique label. Load `hyperref` followed by `cleveref`. Replace typed internal numbers with `\cref` or `\eqref`. Add accurate PDF metadata with an author placeholder. Compile until no undefined-reference warning remains.

11.9 Summary

Labels are stable source names attached after numbering commands. `\ref` and `\pageref` print numbers and pages; `\eqref` formats equation numbers. `cleveref` supplies element names and handles multiple labels. `hyperref` adds links and metadata. Repeat compilation resolves auxiliary information. Undefined or duplicate warnings must be repaired, not hidden with typed numbers.

11.10 Checklist

- ☐ Labels use descriptive prefixes and unique names.
- ☐ Figure and table labels follow their captions.
- ☐ Internal numbers are generated, not typed.
- ☐ Hyperlinks and metadata are accurate and restrained.
- ☐ The final log has no undefined or multiply defined references.

11.11 Exercises

- 11.1 Label a section, equation, table, and figure.
- 11.2 Refer to an equation with `\eqref` and its page with `\pageref`.
- 11.3 Convert two references to a single plural `\cref`.
- 11.4 Repair a figure label placed before its caption.
- 11.5 Diagnose a reference whose label differs by one capital letter.
- 11.6 Explain why a duplicate label is dangerous even if a PDF is produced.
- 11.7 Audit PDF title and author metadata for stale draft information.
- 11.8 Challenge: remove every hard-coded internal number from a short report.

11.12 Complete Practical Coding

Labels make numbering resilient when objects move.

Complete practical file: automatic cross-references

```

\documentclass{article}
\usepackage{amsmath}
\usepackage{booktabs}
\usepackage{hyperref}
\usepackage[nameinlink,noabbrev]{cleveref}

\begin{document}

\section{Method}
\label{sec:method}
The mean is defined in \cref{eq:mean}.

\begin{equation}
\bar{x}=\frac{1}{n}\sum_{i=1}^nx_i
\label{eq:mean}
\end{equation}

\begin{table}[htbp]
\centering
\caption{Demonstration result.}
\label{tab:result}
\begin{tabular}{lr}
\toprule
Group & Mean \\
\midrule
A & 12.4 \\
B & 16.7 \\
\bottomrule
\end{tabular}
\end{table}

\Cref{tab:result} reports values calculated using the method in
\cref{sec:method} and the definition in \cref{eq:mean}.

\end{document}

```

Code Lab

Move the table before the equation and compile twice. Insert another equation without editing any number in the prose.

Part V

Citations and Scholarly Sources

Citations and Bibliographies

A citation links a statement to a source; a reference gives the reader enough information to identify that source. L^AT_EX can format and renumber citations, but it cannot decide whether a source supports a claim or whether its metadata is true. We therefore begin with responsibility and data before software.

Learning outcomes

- Distinguish citations, references, bibliographies, keys, and styles.
- Create verified records in a `.bib` database.
- Use `biblatex` with `Biber` to cite and print a bibliography.
- Produce textual, parenthetical, multiple, and page-specific citations.
- Recognise records for articles, books, chapters, datasets, software, theses, reports, and websites.
- Explain when a publisher’s BibTeX or natbib workflow must be kept.
- Apply responsible practices for sources, permissions, and AI assistance.

12.1 Concepts before commands

A *citation* is the marker in the prose, such as an author and year or a number. A *reference* is the fuller bibliographic description. A *bibliography* is the organised set of references printed by the document. A *bibliography database* is the separate source file, commonly `references.bib`, that stores records.

Each record has a *citation key*, a private source identifier such as `lamport1994`. The key is not printed. A *citation style* controls the visible markers and bibliography. A DOI is a persistent identifier, not a complete reference and not a guarantee that a source supports a claim.

12.2 One verified record

A BibL^AT_EX record starts with an entry type, key, and fields:

references.bib: a genuine book record

```
@book{lamport1994,
  author    = {Lamport, Leslie},
  title     = {\LaTeX: A Document Preparation System},
```

```

edition    = {2},
publisher  = {Addison-Wesley},
date       = {1994},
isbn       = {9780201529838}
}

```

The record was checked against the publisher and the L^AT_EX Project’s book list. The key could be different, but it must remain unique. Family-name-first input helps BibL^AT_EX parse names. Do not guess a DOI because a book title looks familiar.

Good Practice

Open the source record or publisher page and verify author order, title, container, year, volume, issue, pages, edition, and identifiers. Reference-manager exports are starting data, not proof of accuracy.

12.3 Connect the database with biblatex

In the preamble, load biblatex, select Biber as the backend, and add the database resource:

Complete example: BibL^AT_EX and Biber

```

\documentclass{article}
\usepackage[backend=biber,style=authoryear]{biblatex}
\addbibresource{references.bib}

\begin{document}
\textcite{lampport1994} provides a recognised \LaTeX{} reference.
Structured source is also discussed in literate programming
\parencite{knuth1984}.

\printbibliography
\end{document}

```

This complete project needs the two genuine records in `references.bib`. `\addbibresource` declares the database. `\textcite` places the author naturally in the sentence; `\parencite` produces a parenthetical citation. `\printbibliography` prints cited records in the selected style.

The reliable build command remains:

Automated bibliography build

```
latexmk -pdf main.tex
```

Internally, pdfL^AT_EX writes citation requests, Biber reads the database and creates

formatted data, and pdfL^AT_EX incorporates it on later passes. The current BibL^AT_EX manual documents the data model and citation commands (Kime et al. 2025).

12.4 Useful citation forms

Citation patterns
<code>\textcite{lamport1994}</code> explains the document system.
The source-and-output distinction is longstanding <code>\parencite{lamport1994}</code> .
Several source types can be cited together <code>\parencite{knuth1984,fisherIris}</code> .
One passage can be identified precisely <code>\parencite[see][pp. 97--99]{knuth1984}</code> .

The first optional note appears before the citation and the second after it. Page-specific citations help a reader verify a focused claim in a paginated source. Do not add a page to a web page that has no stable pages.

Pause and Predict
If a style changes from author–year to numeric, which part should remain stable: the citation keys in source or the visible markers? This separation is why keys should describe records rather than their temporary numbers.

12.5 Records for scholarly source types

The project database contains genuine examples of each major type:

- journal article: Knuth (1984);
- book: Lamport (1994);
- chapter in a book: Chiroma and Abawjy (2023);
- dataset: Fisher (1936);
- software: R Core Team (2026);
- doctoral thesis: Heffernan (2020);
- technical standard/report: International Organization for Standardization (2023); and
- website/documentation: The L^AT_EX Project (2026).

Entry types select suitable fields and formatting. For software used in an analysis, obtain the citation from that installed version when possible; the R documentation

recommends its `citation()` function. For datasets, cite the version and persistent identifier supplied by the repository. For a web resource that changes, record an access date.

12.6 Common bibliography failures

Common Error

Symptom: citation keys appear as bold text or question marks and the bibliography is absent.

Checks: confirm the key exists and matches case; confirm the database path; read the Biber log; make sure `\printbibliography` is present; and run the correct build sequence. An earlier $\text{\LaTeX}$ error can prevent a valid control file from reaching Biber.

A warning about an empty bibliography can mean that nothing was cited. Use `\nocite{*}` only when the document intentionally needs every database record, not as a way to hide missing citations.

Special characters in names and titles need correct Bib $\text{\LaTeX}$ encoding. Protect capital letters whose case must remain, such as an acronym, with braces after checking the style behaviour. Do not wrap every title in extra braces; that can defeat style-controlled case conversion.

12.7 Publisher workflows: BibTeX and natbib

Many publishers supply older but valid workflows built around BibTeX, natbib, and .bst bibliography-style files. BibTeX reads .bib data and uses the .bst file to format a .bbl; natbib supplies citation commands for many author–year and numerical styles.

Submission Check

If a journal template uses BibTeX or natbib, follow its current instructions. Do not load biblatex alongside it or replace its bibliography system merely because this chapter uses Bib $\text{\LaTeX}$ for teaching. Submit the .bbl when the journal requests it.

The educational default and a publisher requirement can legitimately differ. The important skill is recognising the existing workflow and building it in the documented sequence.

12.8 Scholarly responsibility

A citation must support the nearby statement. Citing a paper after reading only its title or an AI summary is not sufficient. Read the relevant source, prefer primary

evidence for a primary claim, represent uncertainty honestly, and do not cite a review as though it performed every study it summarises.

Quotation requires exactness and sometimes permission. Figures, questionnaires, and substantial table content can require permissions even when cited. Data and software are research contributions and should be credited using the records their creators request.

AI-assisted tools may help search, reformat, or inspect a draft record, but they can invent authors, titles, dates, and DOIs. The author remains responsible for checking each record and claim, complying with disclosure rules, protecting confidential information, and following publisher or institutional policy.

Science in Practice

The Iris dataset record identifies a repository DOI and an explicit dataset type (Fisher 1936). That citation acknowledges the data source; it does not automatically justify using the dataset for a new scientific claim. The methods must still explain selection, processing, and limitations.

12.9 Code lab: build a bibliography from two files

Code Lab

Create `main.tex` and `references.bib` in the same folder. Run `latexmk` as shown. Then change the citation key in only the source file, observe the warning, and restore the matching key.

main.tex

```
\documentclass{article}
\usepackage[backend=biber,style=authoryear]{biblatex}
\addbibresource{references.bib}

\begin{document}
The system is maintained by the \textcite{latexproject}.
Its documentation is available online \parencite{latexproject}.
\printbibliography
\end{document}
```

references.bib

```
@online{latexproject,
  author = {{The \LaTeX{} Project}},
  title  = {The \LaTeX{} Project},
  url    = {https://www.latex-project.org/},
  urldate = {2026-07-31}}
```

```
}
```

Build the document and its bibliography

```
latexmk -pdf main.tex
```

12.10 Chapter project

Mini Project

Create `references.bib` for the running article. Add the verified ISO measurement-method standard and one verified L^AT_EX/software record used by the workflow. Load `biblatex` with `Biber`, cite sources only where they support the prose, and print the bibliography. Run `latexmk`; correct every undefined citation before continuing.

12.11 Summary

Citations connect claims to references stored as verified database records. Keys are stable source identifiers; styles control visible form. BibL^AT_EX and `Biber` support structured entry types and varied citation commands. `latexmk` automates the required passes. Publisher BibTeX/`natbib` workflows must be preserved when required. Accuracy, relevance, permission, data/software credit, and responsible AI use remain the author's obligations.

12.12 Checklist

- ☐ Every record was checked against an authoritative source.
- ☐ Citation keys are unique, stable, and meaningful.
- ☐ Each citation supports the claim beside it.
- ☐ Data and software used in the work receive appropriate credit.
- ☐ The selected workflow matches the journal or institutional instruction.
- ☐ No undefined citation or empty bibliography warning remains.

12.13 Exercises

12.1 Distinguish a citation key, citation, reference, and bibliography.

12.2 Create a verified `@article` record from a publisher page.

12.3 Produce textual and parenthetical citations to one record.

- 12.4 Cite two records together and add a page-specific postnote.
- 12.5 Diagnose a missing bibliography caused by an absent print command.
- 12.6 Explain why a DOI alone is not evidence that a source supports a claim.
- 12.7 State when to keep a publisher's BibTeX/natbib workflow.
- 12.8 Challenge: audit five records for metadata accuracy and document the authoritative page used for each check.

12.14 Complete Practical Coding

The file creates its own demonstration bibliography database and runs with the BibL^AT_EX–Biber workflow.

Complete practical file: bibliography and citations

```
\begin{filecontents*}{practical-references.bib}
@article{example2026,
  author      = {Researcher, Ada and Researcher, Ben},
  title       = {A Demonstration Measurement Study},
  journaltitle = {Example Journal},
  year        = {2026},
  volume      = {1},
  pages       = {1--10}
}

@book{writing2025,
  author      = {Writer, Cara},
  title       = {A Demonstration Book Record},
  publisher    = {Example Press},
  year        = {2025}
}
\end{filecontents*}

\documentclass{article}
\usepackage[backend=biber,style=authoryear]{biblatex}
\addbibresource{practical-references.bib}

\begin{document}

\textcite{example2026} demonstrates a narrative citation. A
parenthetical citation is also possible \parencite{writing2025}.

\printbibliography

\end{document}
```


Code Lab

Compile with `latexmk -pdf main.tex`. Replace the demonstration records with checked records from your own sources, cite both keys, and confirm that only cited works appear.

Part VI

Controlling a Professional Document

Document Classes, Packages, and Page Design

Once content is structured, design can support it consistently. The class should make the largest decisions; packages should solve defined problems; and local formatting should be the last, smallest layer. Reversing that order produces a preamble full of repairs whose interactions are difficult to trace.

Learning outcomes

- Choose among the standard **article**, **report**, and **book** classes.
- Apply class options and load packages for stated purposes.
- Locate and read current package documentation.
- Set page size, margins, spacing, headers, footers, and numbering.
- Prepare title, landscape, and multi-column pages when justified.
- Explain how **microtype** improves typography without manual repair.

13.1 Choose the broad architecture

The standard classes share many commands but differ in assumptions:

article

Short scholarly works. Sections are the highest normal division; there is no `\chapter`.

report

Longer works with chapters, a title page, and report-like front matter.

book

Books and long theses that need front, main, and back matter, two-sided conventions, and parts or chapters.

Choose from structure, not title. A document called “Annual Report” may be a six-page article-class file; a doctoral thesis may use **book** or an institutional class.

Pause and Predict

What happens if source containing `\chapter` is compiled with `article`? Predict whether the class knows that command. Then decide whether changing the class or changing the structure better matches the task.

13.2 Class options

Options in square brackets modify supported broad choices:

Class options

```
\documentclass[11pt,a4paper,twoside,openright]{book}
```

The example requests an 11-point base size, A4 paper, two-sided layout, and chapters beginning on right-hand pages. Options are class-specific. An unknown option may generate a warning or be passed to another component. Read the class documentation rather than assuming an option exists.

For a publisher class, preserve its class line and options unless the current instructions say otherwise. A generic draft and a publisher submission are different design contexts.

13.3 Packages solve named problems

Load a package in the preamble with `\usepackage`. Several packages can be listed together only when they need no separate options and grouping remains readable:

Purposeful package loading

```
\usepackage{amsmath} % Structured mathematics
\usepackage{graphicx} % External graphics
\usepackage{booktabs} % Professional table rules
```

Comments record purpose. Package order sometimes matters: `hyperref` is usually loaded late and `cleveref` after it. A class may already load a package or provide an alternative interface.

Common Error

Symptom: “Option clash for package ...”.

Cause: the same package was loaded more than once with incompatible options, perhaps once by the class.

Repair: search the preamble and log, load the package once with the needed options, or pass class-compatible options before it is loaded. Test a minimal example before changing a publisher template.

13.4 Read documentation at the point of need

On a local TeX installation, the terminal command

Open a package manual

```
texdoc geometry
```

asks the TeX distribution for the `geometry` documentation. CTAN package pages link to manuals and stable package information. Check the manual version against the installed package when behaviour matters.

Read the introduction, examples, option reference, and compatibility notes. Copying an isolated answer from the web may omit its class, package version, or required loading order.

13.5 Page size and margins

The class establishes defaults. When a generic report has explicit page requirements, `geometry` states them clearly:

Explicit generic report geometry

```
\usepackage[
  a4paper,
  inner=30mm,
  outer=25mm,
  top=25mm,
  bottom=25mm,
  headheight=15pt
]{geometry}
```

Inner and outer margins adapt to odd and even pages in a two-sided document. Binding requirements can justify an asymmetric layout. Do not use `geometry` to override a journal class that owns the submission dimensions.

Page design is a system: paper, margins, type size, line length, header, footer, figures, and tables interact. Narrow margins may create long, tiring lines and leave no safe binding area.

13.6 Line spacing and paragraphs

Some review or institutional workflows require double spacing. The `setspace` package offers declared environments and commands:

Optional double-spaced draft

```
\usepackage{setspace}
```

```
\begin{document}
\ doublespacing
% Draft manuscript content
\end{document}
```

Use this only when required. Increasing `\baselinestretch` by guess can affect footnotes and floats unexpectedly. Paragraph indentation and paragraph spacing are also design choices; avoid combining a large indent with a large blank gap unless a verified style calls for both.

13.7 Headers, footers, and page numbering

The `fancyhdr` package can define restrained running heads:

Two-sided running head pattern

```
\usepackage{fancyhdr}
\pagestyle{fancy}
\fancyhf{}
\fancyhead[LE]{\thepage\quad Short Book Title}
\fancyhead[RO]{\nouppercase{\rightmark}\quad\thepage}
\renewcommand{\headrulewidth}{0.4pt}
```

`\fancyhf{}` clears existing fields. LE means left on even pages; RO means right on odd pages. `\rightmark` contains heading information. Title and chapter-opening pages commonly use a separate `plain` style.

`\pagenumbering{roman}` and `\pagenumbering{arabic}` change numbering and reset the page counter. Long documents should use structural commands such as `\frontmatter` and `\mainmatter` when the class provides them.

13.8 Title pages and special orientations

The `titlepage` environment creates a page whose contents you control. Use it for a report cover or institutionally defined thesis title page after verifying required wording. Keep identity fields as commands or placeholders so they are edited once.

The `pdfscape` package can rotate a genuinely wide page. The `multicol` package can create balanced columns for a glossary or compact reference, but multi-column prose complicates wide equations, figures, and accessibility. A two-column journal class should control its own layout.

Science in Practice

An environmental assessment may use portrait pages for narrative, one landscape page for a wide comparison matrix, and an appendix for detailed site records. Orientation expresses a content need; it should not vary for visual

novelty.

13.9 Microtypography

The `microtype` package applies subtle character protrusion and font expansion where supported. These adjustments can improve justification and reduce distracting spacing without changing the argument or asking the author to insert manual spaces.

A restrained typographic improvement

```
\usepackage{microtype}
```

It is intentionally uneventful. Good microtypography is often noticed as comfortable reading rather than as an effect. Inspect the build log and output; do not add discretionary hyphens throughout source merely to silence one draft line.

Good Practice

Start from the class defaults. Add one package when a named requirement appears. Record the purpose, compile, inspect, and remove settings that no longer solve a problem. A shorter preamble is easier to test and transfer.

13.10 Chapter project

Mini Project

Create two build variants of the running article by changing only documented settings: a readable generic article and a review draft with line spacing or margins required by a hypothetical instruction. Keep the content, labels, and bibliography identical. Record the variant purpose in the README and do not present the hypothetical format as a universal submission rule.

13.11 Summary

Classes provide broad architecture; options modify supported choices; packages add named capabilities. Current manuals are part of the workflow. Geometry, spacing, headers, footers, numbering, title pages, orientation, and columns form an interacting page system. `microtype` can improve fine typography. Publisher and institutional classes should retain control over prescribed design.

13.12 Checklist

- ☐ The class matches the document's structural scale.

- ☐ Every package has a stated purpose and compatible loading order.
- ☐ Page settings follow verified requirements or documented preferences.
- ☐ Headers and page numbers work on odd, even, and opening pages.
- ☐ Special orientations and columns solve real content problems.
- ☐ The preamble contains no unexplained copied block.

13.13 Exercises

- 13.1 Choose a standard class for a short paper, a laboratory report with chapters, and a textbook.
- 13.2 Explain four options in the sample `book` declaration.
- 13.3 Use `texdoc` to locate one installed package manual.
- 13.4 Set asymmetric inner and outer margins for a two-sided generic report.
- 13.5 Create a running header and a separate plain opening-page style.
- 13.6 Diagnose a package option clash in a minimal example.
- 13.7 Explain why a publisher class should not be restyled casually.
- 13.8 Challenge: remove unnecessary packages from a copied preamble and document the evidence for each removal.

13.14 Complete Practical Coding

Centralise page design in the preamble and leave content commands semantic.

Complete practical file: restrained page design

```
\documentclass[11pt]{article}
\usepackage[a4paper,margin=28mm,headheight=14pt]{geometry}
\usepackage{fancyhdr}
\usepackage{microtype}

\pagestyle{fancy}
\fancyhf{}
\fancyhead[L]{Measurement Report}
\fancyhead[R]{\thepage}
\renewcommand{\headrulewidth}{0.4pt}

\title{A Consistently Designed Report}
\author{Your Name}
\date{}
```

```
\begin{document}
\maketitle

\section{Objective}
The document class and packages control recurring design choices.

\section{Method}
The source uses sections rather than manually styled headings.

\newpage
\section{Result}
The header and page number repeat automatically.

\end{document}
```

Code Lab

Change the margin once, replace the header text once, and add a third page.
Do not format individual headings by hand.

Long Documents and Multi-File Projects

A 200-page thesis should not be one scrolling source file. Splitting a project by stable scholarly units makes navigation, collaboration, and debugging easier, provided one main document still owns the class, packages, metadata, and build.

Learning outcomes

- Split a long document with `\input` and `\include`.
- Organise front matter, main matter, back matter, and appendices.
- Generate lists of figures and tables.
- Plan nomenclature, acronyms, a glossary, and an index.
- Define shared semantic commands in one file.
- Maintain references across files and preserve project portability.

14.1 One main document

The root `main.tex` should make the project visible at a glance:

A small multi-file book driver

```
\documentclass[11pt]{book}
\input{preamble/packages}
\input{preamble/commands}

\begin{document}
\frontmatter
\input{frontmatter/abstract}
\tableofcontents

\mainmatter
\include{chapters/introduction}
\include{chapters/methods}

\appendix
\include{appendices/supplementary_methods}

\backmatter
```

```
\printbibliography
\end{document}
```

The included files contain content, not another `\documentclass` or document environment. Relative paths are resolved from the main document during a normal build.

14.2 input and include

`\input{file}` inserts a file at that exact point, much like copying its source into the main file. It is suitable for preamble components, small sections, tables, and front-matter pieces.

`\include{file}` is intended for large divisions such as chapters. It begins on a fresh page and writes a separate auxiliary file. The preamble command `\includeonly` can compile selected included units while retaining stored numbering from earlier full builds:

Draft only one included chapter

```
\includeonly{chapters/methods}
```

Do not nest `\include` inside another included file. Use `\input` for smaller nested material.

Common Error

Symptom: an included file compiles alone but fails from `main.tex`, or the reverse.

Cause: the file assumes a different working directory or duplicates the preamble.

Repair: treat `main.tex` as the official entry point. Keep project-relative paths consistent and move shared setup to the root preamble.

14.3 A durable folder structure

Group files by role rather than by draft date:

Example thesis tree

```
thesis/
|-- main.tex
|-- metadata.tex
|-- preamble/
|-- frontmatter/
|-- chapters/
|-- appendices/
```

```
|-- figures/
|-- data/
|-- bibliography/
|-- README.md
`-- latexmkrc
```

Avoid names such as `final2-really-final.tex`. Version control or an archive records revisions more reliably. Use lowercase, descriptive names and one convention for separators.

14.4 Front, main, and back matter

The `book` class provides `\frontmatter`, `\mainmatter`, and `\backmatter`. Front matter commonly uses Roman page numbers and unnumbered chapters; main matter starts Arabic numbering and numbered chapters; back matter holds references, glossary, and index.

Not every institution uses the same order or numbering. Verify the current thesis manual and map its required components onto this structure. Do not invent a certificate or declaration text.

14.5 Appendices and supplementary material

`\appendix` changes later chapter or section numbering. Appendix files can be included like chapters. Supplementary material may be part of the same PDF or a separate submission file; the venue decides. Give each supplement a stable filename, title, and connection to the main text.

14.6 Lists of figures and tables

`\listoffigures` and `\listoftables` collect captions through auxiliary files. They may need repeat compilation. Short captions can keep lists readable when a full caption is long:

Short list entry and full caption

```
\caption[Simulated method comparison]{Relationship between Method A
and Method B for all simulated paired values. The dashed line marks
equality.}
```

The optional argument becomes the list entry; the required argument remains the full figure caption. It must still identify the figure honestly.

14.7 Nomenclature, acronyms, and glossary

A *nomenclature* lists symbols; an acronym list expands shortened forms; a glossary defines specialised terms. Packages such as `glossaries` can generate and link these lists but require an additional indexing step. Begin with the information architecture:

- define a term or acronym at first use in the prose;
- store each definition once;
- decide whether a separate list genuinely helps the reader;
- document the required build command, often `latexmk` plus a glossary tool configuration.

Do not add a glossary package merely to decorate three abbreviations.

14.8 An index

An index connects reader vocabulary to useful locations; it is not an alphabetical copy of every noun. With the standard `makeidx` workflow:

Minimal index workflow

```
\usepackage{makeidx}
\makeindex

% In the body:
compiler\index{compiler}

% In the back matter:
\printindex
```

MakeIndex processes the generated `.idx` file before a later L^AT_EX pass. Indexing is editorial work: choose synonyms, subentries, and primary pages from the reader's likely search terms.

14.9 Shared commands

If a phrase or notation must remain consistent, define a semantic command in a shared file:

One editable notation definition

```
\newcommand{\MeanDifference}{\ensuremath{\bar{d}}}
```

Then use `\MeanDifference` throughout. Choose a descriptive name and avoid redefining core commands. A custom command is helpful when it captures meaning or repeated complexity; a command for a word used twice may only hide prose.

14.10 Cross-file references

Labels are project-wide. A chapter can refer to a figure in another file because the main build combines their auxiliary information. Every label must still be unique. When using `\includeonly`, run a complete build before submission so removed or renamed labels cannot survive unnoticed in stale auxiliary files.

Science in Practice

A computational thesis may keep one chapter per study, a shared notation file, one bibliography database, generated figures, and a README that records the analysis and document builds. This organisation lets a reviewer trace a result without making chapter files independently pretend to be complete documents.

14.11 Portability audit

A portable project uses relative paths, includes all permitted dependencies, and compiles in a clean location. Check for:

- paths beginning with a user home directory or drive letter;
- local fonts and unshared style files;
- case differences hidden by one file system;
- generated figures whose source or generation notes are absent;
- a main file that is not clearly identified;
- undocumented build tools or shell-escape requirements.

Good Practice

Copy or clone the project into a temporary clean folder and build from the README. A successful build in the author's long-lived directory may depend on forgotten auxiliary files or private packages.

14.12 Chapter project

Mini Project

Split the running article into `main.tex`, a preamble file, and separate section files. Keep title metadata and bibliography registration in the main driver. Use `\input` for sections because a journal article should not force new pages between them. Compile from a clean build directory and update the README with the official entry point.

14.13 Summary

One main file owns the document. `\input` inserts material exactly; `\include` supports large page-starting units and selective drafting. Front, main, back, appendix, and supplementary structures should follow verified requirements. Lists, glossary tools, and indexes use generated files and repeat builds. Shared semantic commands

reduce inconsistency. Relative paths, included dependencies, a README, and a clean test establish portability.

14.14 Checklist

- ☐ The root main file is obvious and contains the only class declaration.
- ☐ Files are split by stable scholarly role.
- ☐ Front, main, back, and appendix order follows verified requirements.
- ☐ Labels remain unique across all files.
- ☐ Shared commands encode meaning, not arbitrary abbreviations.
- ☐ A clean copy builds from relative paths and documented commands.

14.15 Exercises

- 14.1 Choose `\input` or `\include` for a table, preamble file, and thesis chapter.
- 14.2 Draw a folder tree for a five-chapter thesis.
- 14.3 Add lists of figures and tables and explain the repeat pass.
- 14.4 Create two appendix files without typing appendix letters.
- 14.5 Define one shared notation command with a descriptive name.
- 14.6 Find and replace an absolute image path with a portable relative path.
- 14.7 Explain why a clean build is stronger evidence than rebuilding in a long-used folder.
- 14.8 Challenge: split a 500-line source file without changing its PDF content or breaking its references.

14.16 Complete Practical Coding

This single tutorial file creates two chapter files so the complete multi-file pattern can be compiled immediately.

Complete practical file: multi-file book project
<pre>\begin{filecontents*}{practical-introduction.tex} \chapter{Introduction} \label{chap:practical-introduction} State the question and objective here. \end{filecontents*}</pre>


```
\begin{filecontents*}{practical-methods.tex}
\chapter{Methods}
\label{chap:practical-methods}
Describe materials, design, and analysis here.
\end{filecontents*}

\documentclass{book}
\usepackage{hyperref}

\title{A Multi-File Project}
\author{Your Name}
\date{}

\begin{document}
\frontmatter
\maketitle
\tableofcontents

\mainmatter
\include{practical-introduction}
\include{practical-methods}

\backmatter
\chapter*{Project note}
Compile only the root file.
\end{document}
```

Code Lab

After the first build, move the two generated chapter files into a `chapters/` folder, update both paths, and compile only `main.tex`.

Part VII

**Major Scientific Document
Forms**

Building a Thesis or Dissertation

A thesis is both a sustained scientific argument and an institutional record. Its $\text{\LaTeX}$ project must therefore serve two authorities: the research logic and the current regulations of the degree-granting institution. The generic template supplied with this book teaches durable structure while leaving local legal and administrative wording as visible placeholders.

Learning outcomes

- Map verified institutional requirements onto a multi-file project.
- Build title, declaration, certificate, dedication, acknowledgement, abstract, lists, chapters, references, and appendices.
- Control Roman-numbered front matter and Arabic-numbered main matter.
- Manage shared metadata, abbreviations, figures, and bibliography.
- Distinguish generic examples from authorised institutional wording.
- Compile and audit the independent thesis template.

15.1 Requirements come before styling

Obtain the current thesis manual, official class or template, deposit guide, and any school-specific supplement. Record the version or access date. Extract testable requirements: paper size, margins, order, wording, numbering, maximum file size, font policy, accessibility, embargo, and signature procedure.

Submission Check

Never infer a declaration, certificate, approval, or licence from a sample. These statements can carry legal or academic meaning. Use the institution's authorised text and preserve placeholders until it has been verified.

If an official class exists, begin with its documentation and minimal example. Do not pour a generic preamble over it. The class may already own margins, titles, spacing, lists, and chapter openings.

15.2 Tour the supplied project

The complete template is in `templates/thesis/`. It builds independently from its own directory:

Build the thesis template

```
cd templates/thesis
latexmk -pdf main.tex
```

Its top-level organisation separates administrative material from scholarly chapters:

Thesis template map

```
thesis/
|-- main.tex
|-- metadata.tex
|-- frontmatter/
|-- chapters/
|-- appendices/
|-- figures/
|-- references.bib
`-- README.md
```

The README identifies `main.tex` as the entry point and states that the results are simulated. A future reader should not need oral instructions to discover how the project builds.

15.3 Centralise identity and title metadata

The template stores frequently repeated fields in `metadata.tex`:

Editable thesis metadata

```
\newcommand{\ThesisTitle}{[Full approved title]}
\newcommand{\ThesisAuthor}{[Candidate's full name]}
\newcommand{\ThesisDegree}{[Full degree name]}
\newcommand{\ThesisInstitution}{[Institution]}
\newcommand{\ThesisDate}{[Month Year]}
```

Edit each fact once. Use the spelling, punctuation, degree title, and date form required by the official record. Avoid embedding a supervisor's name in five files where a late correction can become inconsistent.

Common Error

Symptom: the title page, PDF metadata, declaration, and deposit form show slightly different titles.

Cause: the same fact was typed independently in several places.

Repair: designate one approved title string and derive every permitted occurrence from it. Recheck systems that require manual entry.

15.4 Design the main driver

Only `main.tex` declares the class, loads shared packages, registers the bibliography, and determines the order:

Thesis life cycle

```
\begin{document}
\frontmatter
\input{frontmatter/title}
\input{frontmatter/abstract}
\tableofcontents
\listoffigures
\listoftables

\mainmatter
\include{chapters/introduction}
\include{chapters/methods}
\include{chapters/results}

\appendix
\include{appendices/supplementary_material}

\backmatter
\printbibliography
\end{document}
```

With the `book` class, `\frontmatter` selects Roman numbering and unnumbered chapter behaviour. `\mainmatter` starts Arabic page numbering and numbered chapters. `\appendix` changes later chapter labels to letters; the author does not type “Appendix A” manually.

Pause and Predict

Suppose a chapter is moved before `\appendix`. Which is safer: changing a typed “Appendix B” title, or allowing the structural command to recompute the letter and every cross-reference? Explain what else may need recompilation.

15.5 Build front matter deliberately

Common components include a title page, candidate declaration, supervisor certificate, dedication, acknowledgements, abstract, contents, and lists. Their names and order vary. Some institutions forbid a certificate; others require precise wording. Some accept electronic approval rather than a scanned signature.

Unnumbered chapters need explicit contents entries when the class or local template does not add them:

An unnumbered abstract in the contents

```
\chapter*{Abstract}
\addcontentsline{toc}{chapter}{Abstract}
```

Hyperlinked documents may also need `\phantomsection` immediately before a manual contents entry, depending on the class and package setup. Test the link, not merely its coloured appearance.

The abstract must stand alone. Define abbreviations, report the question and approach, state the principal findings with scale where appropriate, acknowledge important limitations, and identify the contribution. Follow the verified word limit. Citations in an abstract are often discouraged and complicate database display.

15.6 Lists, abbreviations, and symbols

Lists of figures and tables are generated from captions. They need repeat compilation after changes. Use meaningful short captions if a full caption is too long for a list.

The generic template uses a small manual abbreviations table, keeping its build portable. A large, changing nomenclature may justify `glossaries` or a discipline-specific tool. The decision depends on scale:

- define every abbreviation at first substantive use even if it is listed;
- distinguish scalar symbols, vectors, units, and operators;
- order entries consistently and use one definition per concept;
- document any extra glossary build step in the README.

15.7 Shape a chapter as an argument

A chapter is not merely a container of results. Begin by stating its purpose and relationship to the whole thesis. End by resolving its question and making the transition to the next chapter. In an integrated thesis, repetition should be controlled through cross-references. In a thesis by publication, each paper may need to remain intelligible while the surrounding synthesis explains overlap and co-authorship.

Typical empirical units are introduction, literature context, methods, results, discussion, and conclusion. These are patterns, not universal laws. A mathematical thesis may organise itself around definitions and proofs; a design thesis around requirements, artefacts, evaluation, and reflection.

Science in Practice

A multi-study environmental thesis may share a notation chapter and general methods, then give one results chapter per study. A cross-chapter synthesis should compare findings and limitations rather than repeat each chapter's final paragraph.

15.8 Published and joint work

Where published papers are incorporated, verify the permitted manuscript version, copyright, co-author permissions, required attribution, and the institution's declaration of candidate contribution. Do not assume that being an author permits reuse of a publisher-formatted PDF.

Keep a contribution record while the work is active. State which material is joint, who performed analyses, and what has appeared elsewhere. If a chapter retains a journal's internal bibliography, decide with the institution whether the thesis needs chapter bibliographies or one consolidated list.

15.9 Figures, tables, and generated evidence

Store editable source or generation instructions beside derived figures. The template includes an original simulated comparison figure, not an image copied from the web. Its caption states what the diagonal means and avoids claiming it is a fitted model.

For generated results:

1. keep raw inputs read-only where policy requires;
2. generate clean analysis data through a scripted step;
3. generate values, tables, and figures from versioned code;
4. preserve software versions and environment information;
5. make the document consume stable outputs;
6. archive the release identified in the deposited thesis.

Avoid copying the same number into prose, a table, and a figure annotation by hand. A correction should have one computational source.

15.10 Appendices and supplements

An appendix is part of the thesis PDF. A supplement may be a separate deposited object. Put supporting detail where readers can find it, but do not exile essential methods or reasoning to inaccessible files.

The template's evidence register prompts for a persistent identifier, version, licence, and access condition. For restricted data, describe how qualified readers can request access; do not promise public availability that ethics, consent, law, or contracts prohibit.

15.11 Selective drafting and full builds

During revision, `\includeonly` can shorten a chapter build:

Draft two chapters

```
\includeonly{chapters/methods,chapters/results}
```

It relies on stored auxiliary information for excluded chapters. Remove or comment out `\includeonly` and perform a clean, complete build before review or deposit. Otherwise stale labels, citations, or list entries can masquerade as current output.

15.12 Institutional compliance matrix

Convert prose regulations into an auditable table:

Requirement	Implementation	Evidence
Page size and margins	Official class or documented geometry	Measured PDF page
Declaration text	Authorised source copied verbatim	Manual comparison
Numbering	Front/main matter commands	Page audit
File and font rules	Build configuration	Preflight report
Accessibility	Tagged workflow or required remediation	Accessibility check

If a requirement cannot be met with pdfL^AT_EX or the prescribed template, raise the issue with the graduate office early. A last-minute visual approximation is not evidence of compliance.

Good Practice

Schedule a deposit rehearsal. Build on a clean machine or service, open the PDF in more than one viewer, test links and bookmarks, verify page numbers and fonts, and upload to any validation service while changes are still possible.

15.13 Code lab: assemble a thesis from small files

Code Lab

Create the three files below in one folder. Compile `main.tex`; then add a second chapter file and one more `\include` line. This is the smallest useful rehearsal of a multi-file thesis.

main.tex

```
\documentclass[12pt,oneside]{report}
\usepackage[hidelinks]{hyperref}

\begin{document}
\pagenumbering{roman}
\input{title-page}
\tableofcontents

\clearpage
\pagenumbering{arabic}
\include{chapter-one}

\appendix
\chapter{Supplementary method}
Record enough detail for the method to be reproduced.
\end{document}
```

title-page.tex

```
\begin{titlepage}
  \centering
  {\Large Thesis Title\par}
  \vfill
  Your Name\\
  Institution placeholder\\
  2026
\end{titlepage}
```

chapter-one.tex

```
\chapter{Introduction}
\label{chap:introduction}
State the problem, scope, and thesis structure here.
```

15.14 Chapter project

Mini Project

Build `templates/thesis/main.tex`. Replace the title, candidate, degree, department, institution, supervisor, and date placeholders with clearly marked fictional practice values. Add a sixth chapter file and reference it from the introduction. Then remove the chapter and confirm that a clean build removes its contents and cross-reference data.

15.15 Summary

A thesis project translates current institutional requirements into a durable scientific structure. One driver controls front, main, appendix, and back matter; shared metadata prevents contradictions; chapter files carry the argument; and generated lists and references require a complete build. Legal, administrative, contribution, and repository statements must remain explicit placeholders until verified. Clean compilation and a compliance matrix turn a plausible-looking PDF into an auditable submission candidate.

15.16 Checklist

- ☐ Current institutional rules and the official template are identified.
- ☐ Every placeholder and required approval is visible.
- ☐ Roman and Arabic numbering change at the correct structural points.
- ☐ Title metadata is consistent across the project and deposit record.
- ☐ Lists, citations, cross-references, and appendices survive a clean build.
- ☐ Joint work, prior publication, data access, and contributions are stated.
- ☐ The final PDF is inspected and preflighted before deposit.

15.17 Exercises

- 15.1 Map five requirements from a real or supplied thesis manual to files.
- 15.2 Explain why declaration wording is not suitable for creative adaptation.
- 15.3 Add a nomenclature entry for a vector, scalar, and SI unit.
- 15.4 Create an appendix and refer to it without typing its letter.
- 15.5 Use `\includeonly`, then explain why a later full build is mandatory.
- 15.6 Design a contribution statement for a fictional joint chapter.
- 15.7 Write a reproducibility record for one generated thesis figure.
- 15.8 Challenge: produce a compliance matrix with automated and manual tests.

15.18 Complete Practical Coding

The following generic thesis driver makes front matter, main chapters, and an appendix explicit. Replace placeholders only after checking local rules.

Complete practical file: organised thesis driver

```
\documentclass[11pt,oneside]{report}
\usepackage[a4paper,margin=30mm]{geometry}
\usepackage{setspace}
\onehalfspacing

\title{Thesis Title}
\author{Your Name}
\date{2026}

\begin{document}
\pagenumbering{roman}
\maketitle

\chapter*{Declaration}
Insert the verified declaration required for the submission.

\chapter*{Abstract}
State the objective, method, main result, and conclusion.

\chapter*{Acknowledgements}
Record assistance accurately and concisely.

\tableofcontents
\clearpage
\pagenumbering{arabic}

\chapter{Introduction}
Define the problem, gap, and objectives.

\chapter{Literature Review}
Organise prior work around the research question.

\chapter{Methods}
Describe the design and analysis reproducibly.

\chapter{Results}
Present findings with numbered evidence.

\chapter{Discussion}
Interpret findings, limitations, and implications.

\chapter{Conclusion}
Answer the objectives directly.

\appendix
\chapter{Supplementary Methods}
Place supporting detail here.
```

```
\end{document}
```

Code Lab

Replace the title metadata, add one subsection to Methods, and create one appendix. Then compare the structure with the verified institutional checklist.

Preparing a Journal Manuscript

A journal submission is a scientific argument inside a changing operational workflow. The journal decides the current class, file set, anonymous-review rules, declarations, reference style, and upload process. Authors must preserve meaning while adapting to those verified constraints.

Learning outcomes
• Translate current author instructions into a manuscript plan. • Build title, author, affiliation, abstract, keywords, and IMRaD sections. • Prepare anonymous and identified variants without hidden identity leaks. • Write contribution, funding, interest, ethics, data, and code statements. • Decide when a publisher class should replace a generic draft class. • Package clean source, figures, bibliography, and supplementary files.

16.1 Start with the target journal

Before formatting, locate the journal’s current author instructions, manuscript type, template, reference guidance, data policy, ethics requirements, and file limits. Record the access date. A copied template from an old submission or a colleague may encode retired requirements.

Build a small compliance table with columns for instruction, source URL or document, manuscript location, and verification status. Separate mandatory rules from recommendations. If the journal requires its own class, compile the publisher’s unchanged sample before moving content into it.

Submission Check
Confirm whether the first upload expects publisher formatting, a readable generic manuscript, or a format-free submission. These are different workflows. Do not redesign a publisher class to imitate a personal house style.

16.2 Tour the independent article

The project under `templates/scholarly_article/` is deliberately generic. It contains one driver, section files, a bibliography database, and an original simulated figure:

Build the scholarly article

```
cd templates/scholarly_article
latexmk -pdf main.tex
```

The template makes no claim to match a particular journal. Its README tells the author to replace placeholders and consult current instructions. This is an important form of honesty: a reusable starting point should not pretend to be an authorised publisher template.

16.3 Title and short title

A useful title identifies the subject, relationship or design, and sometimes the population. Avoid claims stronger than the evidence and abbreviations that database readers cannot decode. Many submission systems ask for both a full title and a running title with a strict character limit.

Store these strings semantically:

Title metadata

```
\newcommand{\manuscripttitle}{Agreement between two
simulated measurement methods}
\newcommand{\shorttitle}{Simulated method agreement}
\title{\manuscripttitle}
```

Check capitalisation rules and ensure the title entered in the submission form matches the PDF. Metadata discrepancies create avoidable production queries.

16.4 Authors, affiliations, and correspondence

Establish authorship and order through the project’s accepted process before submission. Record each author’s preferred publishing name, affiliation during the work, current address if required, persistent identifier, and contribution. The corresponding author field usually needs a durable professional address.

Do not guess superscript syntax. Publisher classes commonly define their own author and affiliation commands. In a generic draft, a compact manual layout is acceptable, but it must be replaced using documented publisher syntax later.

Common Error

Symptom: the anonymous PDF contains no author block, yet identity is revealed by acknowledgements, self-citations, PDF properties, file names, tracked changes, repository links, or embedded figure metadata.

Repair: perform an anonymity audit across the entire submission package and the submission form. Follow the journal’s rule for referring to prior work; do not automatically distort citations.

16.5 A safe anonymity toggle

For a generic draft, a conditional can switch the visible author block:

Anonymous display variant

```
\newif\ifanonymous
\anonymoustrue % change to \anonymousfalse after review

\ifanonymous
  \author{Anonymous author(s)}
\else
  \author{[Authors and affiliations]}
\fi
```

This is only one layer. An anonymous build may also need a separate declarations file or placeholders for repository identifiers. Keep the identified data in version control with appropriate access; avoid deleting it and reconstructing it from memory after review.

16.6 Write a structured abstract

An abstract commonly contains four moves:

1. the problem or objective;
2. essential design, sample, and analysis;
3. principal results with interpretable scale;
4. a conclusion bounded by the study.

Some journals require labelled headings; others prohibit them. Check the word limit and abbreviation policy. An abstract should not rely on a figure, table, equation number, or unexplained citation. Values in the abstract must match the main text and submission form.

Choose keywords for discovery and indexing, not as a list of words already in the title. Use controlled vocabulary when the field or journal requires it.

16.7 IMRaD as a claim structure

Introduction, Methods, Results, and Discussion remain a useful empirical pattern:

Introduction

Establish the knowledge gap and end with the objective, question, or hypothesis.

Methods

Provide enough design, measurement, analysis, and governance information to evaluate and reproduce the work.

Results

Report observations and analyses without turning the section into a second Methods or Discussion.

Discussion

Interpret the results, compare them with prior work, state limitations, and give a proportionate conclusion.

IMRaD is not compulsory for theory, methods, qualitative research, reviews, or data papers. Use the article type’s reporting guideline and the journal’s headings. The logical test is whether a reader can find the question, evidence, and reasoning.

16.8 Results that agree across forms

One result may appear in the abstract, prose, a table, a figure, and a response to reviewers. Make a results consistency sheet containing the analysis variable, value, unit, rounding rule, source output, and every manuscript location.

Tables should report exact values efficiently; figures should reveal patterns. Do not duplicate every table cell in prose. Lead with the scientific finding, then direct the reader to the display:

Evidence linked to a stable reference
Method B was higher by $\text{0.18}\,\text{mm}$ on average (\cref{tab:summary}).

If analysis changes, regenerate the display and re-audit the abstract and discussion. Changing the PDF figure alone is not a complete correction.

16.9 Declarations are part of the scholarship

The exact headings differ, but modern submissions commonly ask about:

- author contributions;
- funding and the funder’s role;
- competing or relevant interests;
- ethics review, consent, and registration where applicable;
- data, code, materials, and protocol availability;
- acknowledgements and non-author assistance.

Use the journal’s current wording and taxonomy. “Not applicable” is a claim that should be assessed, not a default escape. Data availability statements must reflect real access conditions. A persistent repository identifier is more useful than “available on request” when lawful and ethical sharing is possible.

Science in Practice

A field ecology article may archive de-identified observation data and analysis code openly while restricting precise coordinates for an endangered species. Its availability statement should name both the public objects and the reasoned restriction, rather than calling all data either open or unavailable.

16.10 Publisher classes and package restraint

When moving to an official class:

1. preserve a working generic copy or branch;
2. build the unmodified publisher sample;
3. transfer metadata and plain structure in small increments;
4. use the class's title, abstract, theorem, and bibliography interfaces;
5. add packages only for capabilities the class does not provide;
6. inspect warnings after every meaningful change.

Publisher classes may redefine floats, captions, headings, citations, and two-column behaviour. Packages that worked with `article` can conflict. Visual differences from the generic draft are not automatically defects.

Advanced Topic

Some workflows accept a `.bib` database; others require a generated `.bbl`; still others rebuild references from submission metadata. Submit only the required representation, but retain the authoritative database and build record in the research archive.

16.11 Line numbers and review spacing

Reviewers may require line numbers or double spacing. Treat these as build variants, not handwritten edits. The `lineno` and `setspace` packages are useful with compatible generic classes, but publisher templates may provide their own options.

Inspect line numbering around equations, floats, and multi-column layouts. Confirm whether continuous or page-wise numbering is required. Remove review marks from the accepted production source unless instructed otherwise.

16.12 Supplementary files

Every supplement needs a stable name, title, citation from the manuscript, and appropriate legend or README. Verify whether figure and table numbering restarts with an “S” prefix, and whether references are shared or separate.

Code and data supplements should contain machine-readable files, not only screenshots. Include a data dictionary, licence, environment or dependency record, random seed where relevant, and exact execution order. Never upload restricted or identifiable data simply because the interface offers a slot.

16.13 A clean submission package

Create the upload package from versioned, required sources rather than sending the entire working directory. Exclude auxiliary files, private notes, obsolete figures, local caches, credentials, and unrelated data. Include every custom style or permitted font dependency required by the venue.

Run a clean build in a new directory, then verify:

- the designated main file is obvious;
- every figure opens and uses the accepted format and resolution;
- citations resolve and bibliography data are complete;
- supplementary references and file names match;
- PDF fonts, page size, links, and metadata meet instructions;
- no comments or annotations reveal confidential review history.

Good Practice

Save the submitted PDF, source package, supplementary files, submission-form metadata, cover letter, and a timestamped manifest together. This release is the baseline for responses to reviewers.

16.14 Code lab: switch between review and identified copies

Code Lab

Compile with `\anonymousfalse`, inspect the author line, then change it to `\anonymoustrue` and compile again. One switch should control every identity-bearing field; searching and deleting names by hand is unreliable.

A review-mode switch in a complete article

```
\documentclass[12pt]{article}
\usepackage{setspace}
\usepackage{lineno}

\newif\ifanonymous
\anonymousfalse % Change to \anonymoustrue for anonymous review.
```

```
\title{A Reproducible Comparison of Two Measurement Methods}
\ifanonymous
  \author{Anonymous authors}
\else
  \author{Author Name\\Institution}
\fi

\begin{document}
\doublespacing
\linenumbers
\maketitle
\begin{abstract}
This demonstration uses simulated data to compare two methods.
\end{abstract}
\section{Introduction}
State the question and the contribution.
\section{Methods}
Describe data, analysis, and software sufficiently for reproduction.
\end{document}
```

16.15 Chapter project

Mini Project

Build `templates/scholarly_article/main.tex`. Produce identified and anonymous variants using the documented toggle. Audit the anonymous PDF and source archive for six identity channels. Then replace one simulated result and list every manuscript location that would require regeneration or verification.

16.16 Summary

A journal manuscript begins with current author instructions and a bounded claim, not with decorative formatting. Titles, author metadata, abstracts, IMRaD sections, results displays, declarations, and supplements each serve a scholarly function. Anonymous review requires a package-wide audit. Official classes should be adopted through their documented interfaces, and a clean, minimal, versioned submission release preserves what the journal actually saw.

16.17 Checklist

- ☐ The manuscript type and current journal instructions are recorded.
- ☐ Title, author order, affiliations, and correspondence are confirmed.
- ☐ Abstract numbers and claims match the main article.

- ☐ Methods and declarations satisfy the applicable reporting requirements.
- ☐ Anonymous and identified variants do not leak or lose information.
- ☐ Figures, tables, citations, supplements, and repository links resolve.
- ☐ The clean package contains required files and no private working debris.

16.18 Exercises

16.1 Convert five journal instructions into testable compliance rows.

16.2 Revise an overclaiming title so it describes the actual design.

16.3 Write a four-move abstract for the simulated article.

16.4 Identify eight possible anonymity leaks beyond the author line.

16.5 Draft accurate funding, interest, and data statements for a fictional study.

16.6 Explain when a figure and a table are complementary rather than duplicate.

16.7 Produce a manifest for a clean source submission.

16.8 Challenge: migrate the template into a supplied publisher class and log every class-specific change.

16.19 Complete Practical Coding

Keep the scientific structure stable and isolate temporary review formatting behind one switch.

Complete practical file: journal-ready generic article

```
\documentclass[11pt]{article}
\usepackage[a4paper,margin=25mm]{geometry}
\usepackage{setspace}

\newif\ifreviewcopy
\reviewcopytrue % change to \reviewcopyfalse for a compact draft
\ifreviewcopy
  \doublespacing
\fi

\title{Concise, Informative Article Title}
\author{First Author \and Second Author}
\date{}

\begin{document}
\maketitle
```

```
\begin{abstract}
State the question, method, main result, and conclusion.
\end{abstract}

\section{Introduction}
End with the objective or hypothesis.

\section{Materials and Methods}
Describe the design, measurements, and analysis.

\section{Results}
Report findings in the same order as the methods.

\section{Discussion}
Interpret the result, limitation, and implication.

\section{Conclusion}
Give the main answer without repeating the abstract.

\section*{Declarations}
\textbf{Competing interests:} The authors declare none.\par
\textbf{Data availability:} Replace with the accurate statement.\par
\textbf{Author contributions:} Replace with verified contributions.

\end{document}
```

Code Lab

Compile both review-switch states. When converting to a verified journal class, move the scientific sections unchanged and replace only class-specific metadata and declarations.

Writing a Technical or Project Report

A technical report helps a defined audience make, record, or review a decision. Its value lies not in a formal cover but in the traceable path from purpose and evidence to findings, limitations, recommendations, owners, and next actions.

Learning outcomes

- Define audience, decision, scope, status, and handling requirements.
- Build a cover, document-control record, executive summary, and body.
- Separate observations, interpretations, recommendations, and decisions.
- Present risk, uncertainty, actions, and evidence registers clearly.
- Maintain version, approval, distribution, and archival information.
- Compile and adapt the independent technical-report template.

17.1 Begin with the decision

Ask who will read the report, what they must decide, when they must decide it, and which evidence is within scope. A report without a decision context often becomes a chronological diary of work.

Write a one-sentence commissioning statement:

This report evaluates [evidence] so that [decision owner] can decide [specific decision] by [date], subject to [important boundary].

That sentence controls relevance. Useful material outside the boundary can be recorded as future work rather than crowding the main argument.

Pause and Predict

Two audiences need the same results: a technical reviewer must reproduce the analysis, while an executive must choose whether to fund a validation study. Which information belongs in the executive summary, the methods, and an appendix? Identify what both audiences still need in the main findings.

17.2 Tour the report project

The complete independent project is in `templates/technical_report/`. It uses separate files for document control, executive summary, context, methods, findings,

recommendations, limitations, and appendices:

Build the technical report
<pre>cd templates/technical_report latexmk -pdf main.tex</pre>

All organisations, identifiers, results, and dates are placeholders or simulated. The README warns users to verify branding, confidentiality, and approval rules. A reusable template must keep those obligations visible.

17.3 The cover is controlled metadata

A report cover commonly identifies:

- exact title and report or project identifier;
- client, sponsor, or decision owner;
- authoring organisation and responsible authors;
- issue date, version, and status;
- distribution or classification marking where applicable.

Treat “Draft”, “Approved”, “Public”, and “Confidential” as controlled states, not visual decorations. The cover, headers, filename, document register, and repository record must agree. Do not apply a restriction merely because an old template contains one, and do not remove one without authority.

Common Error
Symptom: a PDF named <code>final.pdf</code> has “Draft” in the footer and a different version in the document-control table. Cause: status metadata was typed in independent locations. Repair: define shared commands for report ID, version, date, and status; generate the release filename from a controlled convention where practical; then add a pre-release consistency check.

17.4 Document control and distribution

The version table records meaningful issued states, not every keystroke. A row should state the version, date, change, author, reviewer or approver as required. Version control preserves fine-grained source history; the issued report table records milestones relevant to recipients.

A distribution statement names authorised recipients or the public repository, handling rules, and superseded versions. If a report changes after approval, issue a new controlled version rather than silently replacing the archived PDF.

17.5 Write the executive summary last

Although it appears first, write the executive summary after the findings and recommendations have stabilised. It should enable the intended decision without requiring the reader to reconstruct the entire analysis. A strong pattern is:

Decision required

The precise choice and decision owner.

Evidence and finding

The most decision-relevant result, with scale and uncertainty.

Recommendation

The action proposed and its conditions.

Principal risks

Consequences, uncertainty, and unresolved constraints.

Next step

Owner, evidence of completion, and due date.

Do not introduce numbers that cannot be located in the main report. Avoid an executive summary that merely promises what later sections contain.

17.6 Purpose, scope, and assumptions

Define the system, population, sites, dates, configurations, data, and questions covered. State exclusions. An “out of scope” section prevents reasonable but unsupported generalisation.

List assumptions that affect interpretation, then distinguish them from facts. For each consequential assumption, record how it was checked or how a decision would change if it failed. A vague assumption list at the end of an appendix does not manage risk.

17.7 Methods and assurance

Methods should allow a competent reviewer to assess the work. Record data sources, collection or simulation, inclusion and exclusion rules, equipment and calibration where applicable, preprocessing, equations, software, uncertainty, review, and deviations from the plan.

Use stable labels to connect methods and findings. If the report states a paired difference, define its direction once:

Direction made explicit

```
\begin{equation}
  d_i = B_i - A_i.
  \label{eq:report-difference}
\end{equation}
```

Positive values therefore indicate that method B is higher.

Without the last sentence, a later “positive bias” can be interpreted in two directions.

Science in Practice
An engineering qualification report can separate test execution, data reduction, acceptance criteria, results, non-conformances, and disposition. A failed criterion must remain visible even if an authorised decision accepts the risk; the report should not rewrite failure as success.

17.8 Findings are not recommendations

Keep four statement types distinct:

- 1. **Observation:** what the data or inspection records show.
- 2. **Interpretation:** what the observation means under stated assumptions.
- 3. **Recommendation:** what the author proposes should happen.
- 4. **Decision:** what the authorised owner has chosen.

For example, “Method B averaged 0.18 mm higher” is an observation. “The offset is acceptable” is an interpretation requiring a tolerance. “Proceed to validation” is a recommendation. “The programme board approved validation on [date]” is a decision with a record.

17.9 Use tables and figures for decisions

A decision table can connect evidence to consequence:

Finding	Decision significance	Required response
Mean paired offset observed	Possible calibration difference	Compare with approved tolerance
Tolerance not pre-specified	Acceptability cannot be concluded	Obtain decision-owner approval
Data are simulated	No operational evidence	Conduct controlled empirical study

Figures should answer a question that the text names. Captions must identify simulated or illustrative data, define reference lines, units, and uncertainty, and remain legible when printed in grayscale. Avoid dashboard colour without a semantic key.

17.10 Risk and uncertainty

Risk statements should name cause, event, and consequence. A practical register may include likelihood, impact, existing controls, owner, action, and residual risk, but scales must be defined. Numbers that resemble precision without a calibrated basis are not automatically better than transparent categories.

Statistical uncertainty and project risk are related but different. A confidence interval describes an inferential procedure under assumptions; “high operational risk” combines probability, consequence, and context. Do not translate one into the other without an explicit model.

Good Practice

Put acceptance criteria in the protocol or report before looking at outcomes. If a criterion changes, record the rationale, authority, date, and effect rather than overwriting the earlier state.

17.11 Recommendations must be executable

A recommendation needs an action verb, owner, completion evidence, and where appropriate priority and due date. Compare:

Weak: Further study is recommended.

Actionable: The study lead shall submit a paired validation protocol defining the acceptance tolerance and calibration controls to the decision owner by [date]; approval of that protocol is the completion evidence.

Avoid assigning an owner who has not accepted responsibility. If authority has not been confirmed, retain a placeholder and flag it before issue.

17.12 Limitations and non-conformances

Limitations define the boundary of a claim; they are not ritual self-criticism. State their likely direction or consequence where possible. A small convenience sample may limit generalisability; missing calibration records may make the cause of an offset indeterminate.

Record deviations and non-conformances with identifiers, evidence, impact, disposition, authority, and closure status. Never remove an adverse result from a final report merely because corrective action was later completed; document the sequence.

17.13 Appendices and evidence registers

Appendices can hold detailed protocols, data dictionaries, calculation checks, large tables, drawings, approval records, and a release checklist. Every appendix should be cited from the main report when it supports a claim.

An evidence register gives each object an ID, description, version, location, owner, and access condition. The issued PDF should refer to immutable or versioned evidence rather than a mutable personal folder.

17.14 Release and archival workflow

Before issue:

1. complete independent technical and editorial review;
2. resolve comments or record authorised dispositions;
3. reproduce headline findings from source evidence;
4. verify status, identifiers, dates, classification, and approval;
5. build and inspect a clean PDF;
6. create a file manifest and checksum if required;
7. place the release in the controlled repository;
8. notify authorised recipients and supersede earlier versions visibly.

Keep the signed or approved record separate if inserting a signature image would weaken the security or validity of the approval process. Follow organisational policy rather than inventing a scanned-signature convention.

17.15 Code lab: build a decision-facing report

Code Lab Compile this report, replace the placeholder decision, and add one finding. Then move the recommendation above the evidence and decide whether the new order helps or weakens the reader’s decision trail.
--

A compact report that ends in an action <pre> \documentclass{report} \usepackage[margin=28mm]{geometry} \usepackage{booktabs} \title{Method Verification Report} \author{Prepared for: Client or laboratory placeholder} \date{2026} \begin{document} \maketitle \begin{abstract} \textbf{Decision required:} approve, revise, or reject the proposed method.</pre>

```

\end{abstract}
\tableofcontents

\chapter{Objective}
Assess whether the method meets the stated acceptance criterion.

\chapter{Findings}
The simulated trial produced the following summary.
\begin{center}
\begin{tabular}{lr}
\toprule
Criterion & Simulated result \\
\midrule
Mean recovery & 94.8\% \\
\bottomrule
\end{tabular}
\end{center}

\chapter{Recommendation}
Repeat the trial with an independent sample before adoption.
\end{document}

```

17.16 Chapter project

Mini Project

Build `templates/technical_report/main.tex`. Choose a fictional decision owner and add a shared command for the report ID, version, date, and status. Write one evidence-backed finding, one conditional recommendation, and one risk with an owner. Confirm that the executive summary and main report use exactly the same headline value and decision condition.

17.17 Summary

A technical report is organised around a defined decision. Controlled metadata identifies the report's status; an executive summary compresses the evidence and recommendation; methods establish assurance; and findings remain distinct from interpretations, recommendations, and authorised decisions. Action owners, evidence registers, limitations, review, and archival controls make the report operationally trustworthy as well as readable.

17.18 Checklist

- ☐ Audience, decision, scope, exclusions, and deadline are explicit.
- ☐ Report ID, version, date, status, and handling markings agree everywhere.

- ☐ Executive-summary values and conditions match the body.
- ☐ Methods, acceptance criteria, uncertainty, and deviations are traceable.
- ☐ Observations, interpretations, recommendations, and decisions are distinct.
- ☐ Every action has a realistic owner and completion evidence.
- ☐ The approved release and evidence manifest are archived immutably.

17.19 Exercises

- 17.1 Write a one-sentence commissioning statement for a laboratory report.
- 17.2 Turn a vague executive summary into the five-move structure.
- 17.3 Classify eight sentences as observations, interpretations, recommendations, or decisions.
- 17.4 Define a three-level risk scale without false numerical precision.
- 17.5 Rewrite “further work is needed” as an owned, verifiable action.
- 17.6 Create a document-control table for three issued versions.
- 17.7 Design an evidence register for data, code, figures, and approvals.
- 17.8 Challenge: audit a report package for contradictory status metadata.

17.20 Complete Practical Coding

This complete report moves from evidence to an explicit recommendation.

Complete practical file: technical report
<pre>\documentclass{article} \usepackage[a4paper,margin=25mm]{geometry} \usepackage{booktabs} \title{Technical Assessment Report} \author{Your Name} \date{\today} \begin{document} \maketitle \section*{Executive summary} State the decision, strongest evidence, and required action. \section{Objective}</pre>

Assess whether all validation checks were completed.

`\section{Method}`

Review the checklist and record unresolved actions.

`\section{Findings}`

`\begin{table}[htbp]`

`\centering`

`\caption{Demonstration validation status.}`

`\begin{tabular}{lr}`

`\toprule`

Measure & Value \\\

`\midrule`

Completed checks & 12 \\\

Open actions & 2 \\\

`\bottomrule`

`\end{tabular}`

`\end{table}`

`\section{Recommendations}`

`\begin{enumerate}`

`\item` Resolve both open actions.

`\item` Repeat the validation check after revision.

`\end{enumerate}`

`\section{Limitations}`

This demonstration does not contain operational data.

`\end{document}`

Code Lab

Replace the objective, table, and recommendations with one real reporting scenario. Check that every recommendation follows from a stated finding.

Part VIII

Working Efficiently and Safely

Collaboration and Reproducibility

A document that builds only in one author’s long-lived folder is not yet a reproducible scholarly object. Collaboration exposes hidden assumptions: an unshared image, a local font, an absolute path, an unexplained command, or a revision whose author and purpose cannot be recovered.

Learning outcomes

- Organise and name project files for readers and collaborators.
- Write comments, semantic commands, and a practical README.
- Compare versioned copies, cloud editing, change marking, and Git.
- Share a complete, licensed, permission-aware source package.
- Reproduce a document build in a clean environment.
- Design backups and releases that can be restored and audited.

18.1 Reproducible means reconstructable

For a $\text{\LaTeX}$ document, reproducible compilation means that an authorised reader can obtain the stated output from the preserved source, dependencies, assets, and instructions. It does not imply that the underlying scientific result is valid. Computational reproducibility additionally asks whether data and code regenerate the reported values, tables, and figures.

Record the boundary. A document may be fully rebuildable while sensitive raw data remain restricted. In that case, provide permitted derived material, metadata, access conditions, and a test data set where useful.

Classroom Note

“It compiles on my computer” is evidence about one environment. A clean build from a newly copied project tests whether the project itself contains the dependencies it claims to contain.

18.2 Names are navigation

Choose short, descriptive, portable names:

A readable project

```

measurement_article/
|-- main.tex
|-- preamble.tex
|-- sections/methods.tex
|-- figures/method_comparison.pdf
|-- data/simulated_measurements.csv
|-- analysis/check_results.R
|-- references.bib
`-- README.md

```

Prefer one case convention and avoid ambiguous dates. ISO-style dates such as 2026-08-01 sort chronologically. Spaces can be supported, but simple ASCII names with hyphens or underscores reduce shell, archive, and service friction. Never encode confidential participant information in filenames.

Generated files need not masquerade as source. Keep build products in a documented output directory or ignore them in version control. Do not call a working draft `final-final-2.tex`; use a revision system and meaningful release tags.

Try It Now

List the ten most important files in one current project. Rename two ambiguous files on a disposable copy, update their references, and compile. Note whether case-only changes behave correctly on your file system.

18.3 Comments explain reasons

The percent sign begins a $\text{\LaTeX}$ comment. Use comments for purpose, constraints, and temporary diagnostics:

Comments that retain context

```

% Journal requires continuous line numbers for review.
% Remove the review option only after editorial acceptance.
\documentclass[review]{publisherclass}

% Equality line, not a fitted regression.
\includegraphics{figures/method_comparison.pdf}

```

Avoid comments that repeat the code or preserve whole obsolete paragraphs indefinitely. Version history can recover removed text. Delete resolved draft notes before release, and search for project-specific markers such as `TODO`, `FIXME`, and `[PLACEHOLDER]`.

18.4 Custom commands define a shared language

Semantic commands give collaborators one controlled notation:

Shared scientific notation

```
\newcommand{\MeanDifference}{\ensuremath{\bar{d}}}
\newcommand{\MethodA}{\textsc{Method A}}
```

Place them in a small shared preamble file and document non-obvious arguments. Choose names that reveal meaning. Do not redefine familiar L^AT_EX commands or create private abbreviations for ordinary prose.

Pause and Predict

If the notation for the mean difference changes from $\bar{d}$ to Δ , how many edits are required when the source uses a semantic command? What new risk appears if some authors typed the symbol directly?

18.5 Four ways to manage revision

Different teams need different levels of history and access.

18.5.1 Numbered snapshots

For a brief solo project, dated archive copies can preserve milestones. Keep one working source and write protected snapshots outside it. A directory containing five competing main files invites compilation of the wrong version.

18.5.2 Cloud L^AT_EX collaboration

Services such as Overleaf provide shared projects, an editor, compilation, comments, and history features whose exact availability can depend on the current service and plan. Agree on the main file, TeX engine, review practice, and who resolves conflicts. Export complete source snapshots at milestones; cloud convenience is not a backup policy by itself.

18.5.3 Visible change marking

Packages such as `changes` or `latexdiff` can show additions and deletions. Their output is a review artefact, not automatically the final source. Complex commands, moved text, citations, and floats need manual inspection. Retain an accepted clean version and verify that no change markup survives.

18.5.4 Git and hosted repositories

Git records snapshots of text files efficiently. Hosting services add remote collaboration, issues, review, and releases. This is optional advanced tooling, not a prerequisite for writing L^AT_EX.

A small Git cycle

```
git status
git diff
git add chapters/methods.tex
git commit -m "Clarify paired-difference definition"
```

Commits should describe a coherent change. Inspect the diff before staging. Never commit credentials, private keys, confidential raw data, or files whose licence prohibits redistribution. A public repository is a publication action.

Advanced Topic

A useful branch strategy can be modest: one protected main branch, short-lived branches for substantial changes, review before merging, and tagged releases for submissions. Complex branching without a team agreement can obscure rather than improve the scholarly record.

18.6 Resolve collaboration at the level of ownership

Assign responsibility by scholarly unit: one author owns Methods, another validates analyses, and a corresponding author controls the submission release. Ownership does not prevent review; it prevents simultaneous uncoordinated edits to the same sentences.

Record decisions in an issue, meeting note, or commit message. A useful record states the question, evidence considered, decision, owner, and date. Email alone can scatter the rationale across private inboxes.

Common Error

Symptom: two correct revisions overwrite one another or leave half of each argument in the final paragraph.

Cause: both collaborators edited the same unit without a merge and review convention.

Repair: designate an owner, compare changes explicitly, resolve the scientific meaning before resolving punctuation, and compile the merged result.

18.7 README as the first reproducibility test

A README should answer:

- What is this project, and are its data real or simulated?
- Which file is the official entry point?
- Which TeX engine, bibliography tool, and extra tools are required?

- What command creates the document and generated figures or tables?
- Where do inputs come from and what are their licences or restrictions?
- Which placeholders or manual steps remain?
- Where should generated output appear?

A concise build statement

```
% README.md (shown as plain text)
Build: latexmk -pdf main.tex
Main file: main.tex
Data: simulated teaching values in data/
Analysis check: Rscript analysis/check_results.R
```

The listing is README text, not commands to place in a TeX source file. Use Markdown code fences in the actual README.

18.8 Pin what affects the build

Record the engine and important tool versions. A minimal environment record can include TeX distribution and date, `latexmk`, Biber or BibTeX, custom class versions, and required fonts. If a package behaviour is critical, preserve the authorised class or style file and its licence when distribution permits.

A container or continuous-integration build can strengthen repeatability, but it also needs maintenance. The simplest adequate solution may be a current TeX Live release plus a documented command. Do not introduce a complex platform that no collaborator can repair.

18.9 Avoid absolute paths

This source works only for one account:

Non-portable figure path

```
\includegraphics{/Users/name/Desktop/project/figure1.pdf}
```

Use a project-relative path:

Portable figure path

```
\includegraphics{figures/figure1.pdf}
```

The same rule applies to bibliography databases, data imports, and scripts. Check case exactly: `Figure1.pdf` and `figure1.pdf` may be different files on the collaborator's system.

18.10 Backups need independent copies

Synchronisation protects availability but can synchronise accidental deletion or corruption. A robust pattern keeps multiple copies, on more than one medium, with at least one versioned or offline/off-site copy. Test restoration.

Back up source, data permitted by governance, analysis, figure sources, bibliography, README, licences, approvals, and released PDFs. Auxiliary files can usually be rebuilt. Encrypt and control access where required; a backup is not exempt from data-protection obligations.

18.11 Licence and permission awareness

Distinguish four questions:

1. May the team use the material for this research?
2. May it be redistributed with source?
3. May it be modified?
4. What attribution, notice, or share-alike terms apply?

Copyright, data licences, software licences, consent, confidentiality, and publisher agreements are related but not interchangeable. Record the source, creator, licence, permission, and required credit for each external asset.

Science in Practice

A geological report may use an openly licensed base map, confidential sampling coordinates, an author's original plot, and a publisher-reused diagram under written permission. The source package cannot simply be made public as one undifferentiated object.

18.12 The clean-room rehearsal

Create a fresh directory, copy only the files named by the release manifest, and follow the README as if you were a new collaborator. A successful rehearsal checks filenames, paths, tool order, missing dependencies, and unstated manual steps. Compare the resulting page count or checksum only when deterministic output is expected; timestamps and tool versions can legitimately change bytes.

Good Practice

Make the release from a known version, run the numerical verification, build the document, inspect the log and PDF, and archive the exact inputs and outputs together. A release is more than the latest file in a synchronised folder.

18.13 Chapter project

Mini Project

Open the running article's README. From a fresh copy, run the base-R result check and compile `main.tex`. Add one comment explaining why the difference is defined as B minus A and one semantic command for a repeated notation. Create a manifest of the files another scientist needs; exclude rebuildable auxiliary files and identify the original source of each figure.

18.14 Summary

Collaboration depends on shared meaning, visible ownership, and reconstructable releases. Descriptive names, purposeful comments, semantic commands, one main file, a complete README, relative paths, and tested backups reduce hidden assumptions. Cloud editors, change-marking tools, and Git solve different revision problems. Reproducible sharing also requires licences, permissions, access controls, environment records, a clean build, and an exact release manifest.

18.15 Checklist

- ☐ File and folder names are descriptive, portable, and privacy-safe.
- ☐ Comments explain constraints and no unresolved draft markers remain.
- ☐ The main file, engine, tools, and build commands are documented.
- ☐ Collaborators know ownership, review, and merge conventions.
- ☐ Paths are relative and all permitted dependencies are included.
- ☐ Licences, permissions, and restricted materials are recorded.
- ☐ Backups can be restored and the release builds in a clean directory.

18.16 Exercises

- 18.1 Rename four poorly named files and explain your convention.
- 18.2 Distinguish a synchronised folder, version history, and a backup.
- 18.3 Write a README for a three-file article and its bibliography.
- 18.4 Replace two absolute paths with project-relative paths.
- 18.5 Compare numbered snapshots, change markup, Overleaf, and Git for a team.
- 18.6 Design an asset-permission register for four figure types.

18.7 Plan a clean-room build test and define its success evidence.

18.8 Challenge: create a release manifest that separates public, restricted, generated, and rebuildable files.

18.17 Complete Practical Coding

Portable source keeps editable values centralised and records the files needed to reproduce the PDF.

Complete practical file: portable reproducible note

```
\documentclass{article}
\usepackage{booktabs}

\newcommand{\projectname}{Measurement Demonstration}
\newcommand{\datasetversion}{v1.0}
\newcommand{\analysisdate}{2026-08-02}

\title{\projectname}
\author{Project Team}
\date{\analysisdate}

\begin{document}
\maketitle

\section{Reproducibility record}
This report uses dataset \datasetversion. All paths in the source are
relative to the project root.

\begin{table}[htbp]
  \centering
  \caption{Files required for a clean rebuild.}
  \begin{tabular}{ll}
    \toprule
    File & Purpose \\
    \midrule
    \texttt{main.tex} & Root document \\
    \texttt{data/measurements.csv} & Input data \\
    \texttt{figures/result.pdf} & Generated figure \\
    \texttt{references.bib} & Reference records \\
    \bottomrule
  \end{tabular}
\end{table}

\section{Change record}
Record what changed, why it changed, and who checked the new PDF.

\end{document}
```

Practical build commands

```
latexmk -C main.tex  
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

Code Lab

Move the project to a clean folder, run the two commands, and record every missing input. Replace absolute paths instead of copying files outside the project silently.

Troubleshooting L^AT_EX Systematically

The most intimidating L^AT_EX log often begins with one small structural mistake. A useful debugging habit is therefore conservative: preserve the failing case, read the first meaningful error, reduce the search area, test one hypothesis, and compile again.

Learning outcomes
• Read the first meaningful error and its source context. • Diagnose braces, environments, commands, files, packages, and Unicode. • Create a minimal working example and isolate a failing section. • Repair citations, references, bibliography builds, floats, and boxes. • Decide when stale auxiliary files should be cleared. • Document a reproducible bug report without sharing confidential content.

19.1 The eight-step debugging loop

Use this sequence before making broad changes:

1. Save the source and reproduce the failure.
2. Read the *first meaningful* error, not only the final stop message.
3. Note the named file and line, but inspect several lines around it.
4. Check braces, brackets, math delimiters, and environment pairs nearby.
5. Reduce the document by commenting out large units or using a minimal file.
6. Check the class or package documentation for the command and options.
7. Clear generated auxiliary files only when evidence suggests they are stale.
8. Rebuild in the required sequence and verify the correction in the PDF.

TeX reports where it noticed a contradiction, which can be later than where the mistake began. An omitted closing brace in a caption may trigger an error at the next section. Read backwards through the logical unit.

Try It Now

Open a successful build log and locate the engine version, loaded class, output PDF line, and any warning. Learning the shape of a healthy log makes unusual messages easier to recognise.

19.2 Preserve the evidence

Before debugging, make a recoverable snapshot. Record the exact command, main file, engine, and whether the failure appears in a clean build. Do not begin by updating every package or deleting the entire project; that changes multiple variables and may destroy useful evidence.

The terminal command

Capture a repeatable build

```
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

stops at a real error while retaining a log. An editor's summary may shorten or reorder messages; consult the full `.log` when the diagnosis is unclear.

19.3 Read an error record

A typical record contains an error beginning with an exclamation mark, context, and a line number:

Representative undefined-command report

```
! Undefined control sequence.  
1.42 The result was \qyt{3.2}{\metre}.
```

The engine does not know `\qyt`. The likely repair is the intended `\qty` command from `siunitx`, assuming that package is loaded. Do not define a new `\qyt` merely to silence a typographical error.

Pause and Predict

If the line shown by an error contains only `\end{table}`, which earlier elements inside the table would you inspect first? Why can the closing line be innocent?

19.4 Minimal working examples

A *minimal working example* (MWE) is the smallest complete document that still demonstrates the problem. Start from a copy, never dismantle the only working research source.

Minimal figure test

```
\documentclass{article}
\usepackage{graphicx}
\begin{document}
\includegraphics[width=.5\linewidth]{figures/sample.pdf}
\end{document}
```

Remove unrelated packages and prose while preserving the failure. If removing a package makes the failure disappear, restore it and test loading order or a smaller interaction. An MWE answers whether the cause belongs to the content, the preamble, a file, or the larger project.

When material is confidential, replace names and values while retaining the same commands and structure. Never post protected data in a public support forum.

19.5 Binary isolation

For a long file, comment out roughly half the content. If the failure remains, search that half; otherwise search the removed half. Repeating this process shrinks the suspect region quickly. Preserve necessary structure: commenting out the beginning of an environment but not its end creates a new error.

For multi-file books, temporarily use `\includeonly` or comment complete `\input` lines. Once repaired, restore the complete project and rebuild so cross-file references and lists are current.

19.6 L^AT_EX First-Aid Clinic

The following short cases pair a symptom with a local repair. The broken code is shown as evidence; do not paste it into a working manuscript unchanged.

19.6.1 Undefined control sequence

Broken: misspelt command

The sample was `\emhp{sterile}`.

Corrected command

The sample was `\emph{sterile}`.

If the spelling is correct, check whether the defining package or custom-command file is loaded before use.

19.6.2 Missing closing brace

Broken: open argument

```
\textbf{Important result  
The next sentence is ordinary prose.
```

Corrected group

```
\textbf{Important result}  
The next sentence is ordinary prose.
```

A missing brace can absorb many later tokens, so the reported line may be far from the opening command. Editor brace matching helps, but inspect nested commands manually.

19.6.3 Mismatched environment

Broken: environment names disagree

```
\begin{enumerate}  
  \item Calibrate the instrument.  
\end{itemize}
```

Corrected environment pair

```
\begin{enumerate}  
  \item Calibrate the instrument.  
\end{enumerate}
```

Every `\begin{name}` needs a later `\end{name}` of the same type, properly nested.

19.6.4 Math delimiter not closed

Broken: unclosed inline mathematics

The estimate $\hat{\theta}$ is unbiased.

Corrected inline mathematics

The estimate $\hat{\theta}$ is unbiased.

Do not repair this by adding a random dollar sign at the end of the document. Find the intended boundary between mathematics and prose.

19.6.5 File not found

Broken on a case-sensitive system
<code>\includegraphics{Figures/Result.PDF}</code>

If the actual file is `figures/result.pdf`, correct the path and case:

Corrected relative file name
<code>\includegraphics{figures/result.pdf}</code>

Also verify the main build directory and accepted graphic formats. Avoid fixing a relative-path problem with an absolute personal path.

19.6.6 Missing package or class

“File `name.sty` not found” means the TeX search path cannot locate a required package. Confirm that the name is genuine and required, then install it through the TeX distribution’s supported manager or use a documented alternative. For a publisher class, include the authorised file when redistribution permits. Do not download an arbitrary similarly named file from an unverified mirror.

19.6.7 Unicode character error

With a current UTF-8 workflow, many characters work directly, but the engine, font, and context still matter. A copied Unicode minus sign, non-breaking space, or mathematical symbol may trigger an error or render inconsistently.

Inspect the exact code point. Replace semantic mathematical symbols with L^AT_EX commands, for example `\minus` is not a standard command but a normal binary minus is typed as `-` in math mode. Use an engine and font configuration that intentionally supports needed scripts; do not delete a researcher’s name or linguistic data to make an error disappear.

19.6.8 Undefined reference

Reference and target
See <code>\cref{fig:growth-curve}</code> .
<i>% Later:</i>
<code>\label{fig:growth-curve}</code>

Check exact label spelling, uniqueness, and placement after the caption. Then compile again. Cross-references commonly require two L^AT_EX passes because the label value is written to and read from an auxiliary file.

19.6.9 Undefined citation or empty bibliography

Check four layers:

1. the citation key exactly matches an entry in the `.bib` file;
2. the database is registered with the correct command and path;
3. the chosen backend, Biber or BibTeX, actually ran;
4. L^AT_EX ran again after the backend created bibliography data.

For a `biblatex` and Biber workflow, `latexmk` normally determines the sequence. An empty bibliography can also mean that no entries were cited and the style prints only cited works.

19.6.10 Package option clash

An option clash often means a class or package already loaded the same package with different options. Search the log for the first loading location. Pass compatible options before the load or remove the duplicate call. Do not load the same package repeatedly in different preamble fragments.

19.7 Errors, warnings, and visual problems

An error prevents a reliable build. A warning signals a condition requiring judgement. A PDF can also be wrong without either: a transposed table value, misleading caption, tiny label, or hidden placeholder is not necessarily a TeX error.

19.7.1 Overfull boxes

An overfull box extends beyond its allotted width; the log reports an amount. Inspect the PDF at that location. Common causes are long URLs, unbreakable code, wide equations, and tables.

Repair the content structure: allow URL breaks, enable code wrapping, split an equation, revise a column, or use a landscape page. Avoid a global `\sloppy` setting or shrinking the whole document to conceal one object.

19.7.2 Underfull boxes

An underfull box has poor spacing within a constrained line or page. It may be harmless in a short caption, narrow table cell, or intentionally ragged region. Inspect rather than mechanically silencing it. Rewording or widening a column can help; forced line breaks often create more fragile layout.

19.7.3 Float placement

Figures and tables are floats, so L^AT_EX balances their preferences with page constraints. If a float travels too far:

- ensure it is referenced near its definition;
- use a reasonable width and concise caption;
- offer placement choices such as `[htbp]`;
- move the float source after the first explanatory paragraph;

- reduce a backlog of large consecutive floats.

Use [H] only with a deliberate reason and supporting package. Repeated forced placement can create blank space and unstable pagination.

19.8 When auxiliary files are stale

Auxiliary files hold cross-references, citations, contents, lists, index data, and build state. Clear them when the log suggests corrupted or incompatible state, after a major tool or class change, or when an impossible old label survives. Use the build tool’s clean command in the specific project:

Clean generated L^AT_EX files

```
latexmk -C
latexmk -pdf main.tex
```

Cleaning is not a first reflex. It costs useful state and does not repair bad source. Never use a broad deletion command pointed at an uncertain directory.

19.9 Compile in the correct sequence

A project with citations and an index may need L^AT_EX, bibliography processing, index processing, and further L^AT_EX passes. `latexmk` automates dependency detection. If working manually, follow the toolchain documented for that project; Biber and BibTeX are not interchangeable commands.

Repeated-pass warnings after a stable `latexmk` build may indicate a value that changes every run, a conflicting output directory, or an auxiliary file whose path is not writable. Compare the changing files rather than blindly increasing the maximum number of passes.

19.10 Package documentation and bug reports

Use `texdoc package-name` or the package’s authoritative documentation to verify syntax and compatibility. When requesting help, provide:

- a minimal working example;
- exact build command and complete first error;
- engine, distribution, class, and relevant package versions;
- expected and actual behaviour;
- whether the problem survives a clean build.

Remove confidential content and credentials. Screenshots of an editor can help with visual UI problems, but selectable error text and a compilable example are more useful for L^AT_EX diagnosis.

Good Practice

Change one factor at a time and write down the result. The smallest successful repair teaches more than a copied preamble containing twelve unrelated packages. After repair, add a test, comment, or README note if the same failure could recur.

19.11 Chapter project

Mini Project

Copy the running article to a temporary practice directory. Introduce one misspelt command, one mismatched environment, one missing figure path, and one bad citation key, one at a time. For each, save the first meaningful error, predict the cause, make the smallest correction, and verify the PDF. Finish with a clean complete build and restore the unbroken project.

19.12 Summary

Systematic debugging preserves evidence and reduces uncertainty. Reproduce the failure, read the first meaningful error, inspect nearby structure, isolate the case, consult documentation, and test one repair. Errors in commands, braces, environments, files, packages, Unicode, references, citations, and build order have distinct evidence. Box and float warnings require visual judgement. Auxiliary cleaning is a targeted remedy, and a minimal example is the strongest foundation for useful support.

19.13 Checklist

- ☐ The exact failure and build command are reproducible.
- ☐ The first meaningful error, file, and nearby source were inspected.
- ☐ Braces, delimiters, and environments are balanced.
- ☐ The problem survives a reduced example or has been isolated to a unit.
- ☐ Package syntax and build sequence match authoritative documentation.
- ☐ Warnings were inspected in the PDF, not merely hidden.
- ☐ The full project builds after the minimal repair.

19.14 Exercises

19.1 Explain why the last error in a log may be less useful than the first.

19.2 Repair three brace, delimiter, or environment mistakes in supplied code.

- 19.3 Reduce a failing figure document to a minimal working example.
- 19.4 Diagnose an undefined citation across the four bibliography layers.
- 19.5 Give two legitimate and two poor responses to an overfull box.
- 19.6 Explain when cleaning auxiliary files is evidence-based.
- 19.7 Write a privacy-safe bug report containing all reproducibility details.
- 19.8 Challenge: use binary isolation to locate a planted error in a long file and record the number of compilation tests required.

19.15 Complete Practical Coding

Start from a file that certainly works, then add only enough source to reproduce the first error.

Complete practical file: minimal diagnostic shell

```
\documentclass{article}
\usepackage{amsmath}

\begin{document}

\section{Smallest failing unit}
Add the suspected source below this sentence one logical unit at a time
.

\begin{equation}
\bar{x}=\frac{1}{n}\sum_{i=1}^nx_i
\end{equation}

Reserved characters in text are escaped: 95\%, research \& teaching,
sample\_01, and \$25.

\end{document}
```

Practical diagnostic commands

```
latexmk -C main.tex
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
rg -n "^!|undefined|Overfull" main.log
```

Code Lab

Introduce one mismatched brace, compile, and repair the first error. Then paste one failing unit from another project into this shell and reduce it until the cause is obvious.

From Source Files to Submission

Finishing is a controlled transition from an editable research project to an exact release. The final PDF, source archive, supplementary files, and portal metadata must describe the same work. No universal checklist replaces the current instructions of a journal, institution, funder, repository, or client.

Learning outcomes

- Freeze a candidate release and apply a venue-specific compliance matrix.
- Proofread source, PDF, references, citations, equations, tables, and figures.
- Inspect metadata, fonts, page properties, links, and accessibility needs.
- Prepare clean source, supplementary files, a manifest, and a final PDF.
- Archive an editable reproducible project and the exact submitted release.
- Conduct a final submission rehearsal and record evidence.

20.1 Define the release candidate

Choose a version that has completed scientific review, author approval, and the required ethics or organisational checks. Tag it or record an immutable commit identifier. During final quality control, accept only deliberate corrections; unbounded rewriting restarts review.

Create a release register with:

- document title, version, date, and main-file path;
- venue and manuscript or deposit type;
- source-version identifier;
- responsible author and approver;
- required files and submission deadline;
- status of scientific, editorial, technical, and permission checks.

Submission Check

Download or open the current official instructions at the start of the finishing stage. Record their version or access date. Recheck them immediately before upload because file, declaration, and metadata requirements can change.

20.2 Build a compliance matrix

Turn every instruction into a check with evidence:

Requirement	Implementation	Verification evidence
Anonymous main file	Anonymous build variant	Identity audit
Figure formats	Separate accepted files	File inspection
Reference style	Venue class or style	Sample comparison
Data statement	Declarations section and form	Author approval
Page limit	Final generated PDF	PDF page count

Write “not applicable” only with a reason. An unresolved instruction should be escalated to the appropriate editorial office, graduate office, repository, or client rather than settled by appearance.

20.3 Proofread at three levels

20.3.1 Scientific and logical review

Check the question, methods, analyses, results, limitations, and conclusions as a chain. Confirm sample sizes, denominators, exclusions, equation definitions, units, uncertainty, statistical directions, and the difference between association and causation. Trace each headline claim to evidence.

20.3.2 Source review

Search the source for draft markers, manual numbering, broken labels, duplicate metadata, obsolete files, comments not intended for release, and private notes. Inspect changes since the last approved version. Source review can find a hidden conditional or unused old section that the PDF alone cannot reveal.

20.3.3 Rendered-PDF review

Read the PDF in page order, ideally on screen and in a print-like view. Check page breaks, widows and orphans where severe, clipped content, running heads, blank pages, table continuation, float order, caption proximity, footnotes, bookmarks, and link destinations. Compare the table of contents and lists with the body.

Pause and Predict

Why can source spell-checking not detect a figure label clipped outside the page? Why can PDF reading not reveal an author name hidden in an unused source file included in the submission archive? Assign each issue to the appropriate review layer.

20.4 Cross-references and citations

Search the final log for undefined references and citations. Click a sample of internal and external links, including entries near page boundaries. Confirm that each “above”, “below”, table number, figure number, equation number, chapter, and appendix still points correctly after pagination changed.

For citations:

- verify that each source supports the attached claim;
- compare author, title, year, venue, pages, DOI, and edition with an authoritative record;
- distinguish preprints, accepted manuscripts, versions of record, data, software, standards, and personal communications;
- remove uncited entries if the venue expects only cited works;
- check that retracted, corrected, or superseded sources are handled appropriately.

Never fabricate a DOI, page range, or access date to make a record look complete. Use a visible placeholder until the fact is verified.

20.5 Equations and scientific notation

Read equations independently from the prose. Check:

- every symbol is defined before or immediately after use;
- vectors, matrices, variables, operators, and units use consistent style;
- signs, indices, parentheses, limits, and equation directions match the analysis;
- displayed equations fit at final column width;
- numbering and punctuation support the surrounding sentence;
- significant figures and rounding are consistent with the evidence.

Have a domain expert verify equations whose visual plausibility can conceal a scientific error. $\text{\LaTeX}$ cannot know that a denominator is wrong.

20.6 Tables

Audit every table against its generating output or source record. Check row and column order, headings, units, denominators, missing-value notation, rounding, footnotes, statistical definitions, and totals. Confirm that the caption and notes make the table intelligible without searching the prose.

Inspect tables at the final page and column width. Rules should not cross text; decimal alignment should not hide signs; symbols must be defined; repeated header rows should appear on multi-page tables. A screenshot of a spreadsheet is not an editable, accessible substitute for a real table.

20.7 Figures and image preflight

For each figure, verify content and file properties:

1. compare plotted values with the final analysis;
2. confirm axes, units, legends, uncertainty, sample size, and panel labels;
3. state whether data are simulated, illustrative, or empirical;
4. inspect colour contrast and grayscale legibility;
5. check text at intended physical size, not only when zoomed;
6. preserve vector graphics for lines and text where accepted;
7. verify raster pixel dimensions and the venue's effective-resolution rule;
8. record creator, source, licence, permission, and required credit.

Effective resolution depends on placed size. A 1200-pixel image placed at 4 in provides 300 pixels per inch; enlarging it to 8 in provides only 150. Do not claim universal resolution thresholds; follow the venue's current specifications for each image type.

Science in Practice

A microscopy plate may require an unaltered original archive, a documented processing history, scale bars derived from calibration, panel-level labels, and separate high-resolution files. A visually sharp PDF cannot prove those scientific conditions were met.

20.8 Metadata

PDF properties, portal fields, source commands, title pages, repository records, and supplementary covers should agree on title, author names and order, affiliations, abstract, keywords, identifiers, funders, and version.

An anonymous review PDF should not expose authors in document properties. An identified archival PDF should use verified names. Avoid including unnecessary personal contact details in public metadata. Inspect rather than assume what the engine or publisher class writes.

20.9 Fonts, page properties, and PDF features

If the venue requires embedded fonts, use a PDF inspection tool to confirm that each font is embedded or subset as allowed. Do not rely on the fact that the PDF looks correct on the author's system. Check page size, orientation, crop boxes, security restrictions, file size, and PDF version when specified.

Font embedding does not grant redistribution rights. A font can be technically embedded yet contractually unsuitable. Prefer fonts distributed for the chosen TeX workflow or otherwise licensed for the intended use.

20.10 Accessibility

Current accessibility obligations vary by venue and jurisdiction. Logical headings, real text, meaningful link text, readable contrast, table headers, and textual explanations of figures improve the source, but a conventional pdfL^AT_EX document is not automatically a fully tagged accessible PDF.

Determine the required standard and validation process. If remediation is needed, preserve the structured source and record manual changes so an updated document can be regenerated. Provide alt text or long descriptions through the venue’s accepted mechanism; a caption and alternative text serve related but different purposes.

20.11 Remove draft apparatus

Search for:

Typical release searches
<pre>rg -n 'TODO FIXME PLACEHOLDER DRAFT \?\' ' . rg -n 'showkeys linenumbers draftwatermark' --glob '*.tex' .</pre>

Interpret results: a teaching document may intentionally print the word “placeholder”, while a submission should not contain unresolved fields. Remove line numbers, watermarks, visible changes, margin notes, draft figures, and review comments unless the recipient explicitly requires them.

20.12 Supplementary files

For every supplement, verify title, numbering, citation from the main document, file name, format, size, legend, and access condition. Data and code should have a README, licence, dictionary, environment record, and execution order where appropriate. Scan archives for hidden files, credentials, personal paths, and restricted content.

Open the files after packaging. A valid ZIP archive can still contain the wrong version. Ensure that supplementary equation, figure, table, and reference numbers match the final manuscript and upload descriptions.

20.13 Create a clean submission folder

Build the folder from a release manifest. Include only required source, bibliography, class or style files that may be redistributed, figures, and supplements. Exclude auxiliary files, caches, old drafts, private correspondence, credentials, raw restricted data, and unlicensed assets.

Illustrative release tree
<pre>submission_v1/ -- main.tex -- sections/ -- figures/ -- references.bib -- supplementary/ -- README.md `-- MANIFEST.txt</pre>

Do not blindly flatten paths if the project uses its folder structure. Build from the clean folder using the documented command.

20.14 Inspect the final PDF

The final PDF is the recipient’s reading object even when source is supplied. Perform automated and visual checks:

- build log has no fatal, undefined-reference, or missing-file messages;
- page count and dimensions match expectations;
- fonts and images meet specified requirements;
- text extraction is plausible where required;
- every page is rendered to images and scanned for clipping or blank output;
- representative dense pages are inspected at full resolution;
- title, last page, references, appendices, and special orientations are checked.

Automation can identify candidates; a human must judge scientific and visual meaning.

20.15 Upload rehearsal

If the submission system allows drafts, rehearse early. Portal conversion can reorder files, replace fonts, move figures, or generate a combined PDF. Download and inspect the system-generated proof. Compare entered metadata with the source and secure approval from all authors when required.

Do not click the irreversible final-submission control until the responsible author has verified the proof and declarations. Retain the confirmation and identifier.

Common Error
Symptom: the local PDF is correct, but the submission proof lacks figures or references. Cause: the portal rebuilt an incomplete source archive or interpreted the

wrong main file.

Repair: inspect the portal log and required file designation, correct the clean package, rebuild locally from that exact package, and regenerate the proof. Do not assume the portal will discover undeclared dependencies.

20.16 Archive both release and working project

Preserve:

- exact submitted PDF and source archive;
- supplementary files and manifest with checksums where appropriate;
- source-version identifier and environment record;
- author approvals, permission records, and submission confirmation;
- portal metadata or exported record;
- editable working project with its reproducible analysis.

The submitted release should be immutable. Continue revisions in a new version or branch. A response to reviewers begins from what was actually submitted, not from whichever draft happens to be open later.

Good Practice

Use a two-person release check for high-stakes documents: one person operates the build and upload, while another reads the checklist and verifies the evidence. Both confirm the final proof and archive location.

20.17 Code lab: make a clean release candidate

Code Lab

Run these commands from a copy of your project, not from your only working copy. The result should be a rebuilt PDF, a human-readable file list, and a checksum that identifies the exact submitted PDF.

Rebuild and inspect the release candidate

```
latexmk -C
latexmk -pdf main.tex
latexmk -pdf main.tex
pdffonts main.pdf
```

List the files that belong in the archive

```
find . -type f \
  \( -name '*.tex' -o -name '*.bib' -o -name '*.cls' \
    -o -name '*.sty' -o -name '*.bst' -o -name '*.bbx' \
    -o -name '*.cbx' -o -name '*.pdf' -o -name '*.png' \
    -o -name '*.jpg' -o -name '*.jpeg' -o -name '*.csv' \
    -o -name '*.R' -o -name 'README.md' -o -name 'latexmkrc' \) \
  | sort > MANIFEST.txt
```

Record the identity of the final PDF

```
shasum -a 256 main.pdf > CHECKSUMS.txt
```

Open `MANIFEST.txt`. Remove auxiliary products from the archive, but keep the source, bibliography, figures, instructions, final PDF, manifest, and checksum. On Linux, `sha256sum` is the usual equivalent of the last command.

20.18 Chapter project

Mini Project

Use the running article to create a practice release folder and manifest. Run its numerical check, compile from that folder, search for unresolved markers, inspect citations and links, verify PDF properties and fonts, and complete the included submission checklist. Record every result as pass, fail, or not applicable with a reason; do not submit the simulated article to a journal.

20.19 Summary

Submission is a controlled release, not merely the last compilation. A current compliance matrix guides scientific, source, and PDF proofreading. Citations, equations, tables, figures, metadata, fonts, accessibility, supplements, and draft apparatus need distinct checks. The clean source package must rebuild, the final PDF must be inspected, and the submission-system proof must be treated as a new output. Archive the exact release and retain the editable reproducible project for later correction or review.

20.20 Checklist

- ☐ A reviewed source version is frozen as the release candidate.
- ☐ Current venue instructions are mapped to evidence-backed checks.
- ☐ Scientific claims and repeated numerical results are consistent.
- ☐ References, equations, tables, figures, and permissions are verified.

- ☐ Metadata, fonts, PDF properties, links, and accessibility needs are checked.
- ☐ The clean folder rebuilds and contains no private or obsolete material.
- ☐ The portal proof and exact submitted release are inspected and archived.

20.21 Exercises

- 20.1 Create a compliance matrix from ten supplied venue instructions.
- 20.2 Design a results consistency sheet for an abstract, table, and discussion.
- 20.3 Calculate effective resolution for three raster images at placed size.
- 20.4 Audit a bibliography record against an authoritative source.
- 20.5 List source-only, PDF-only, and portal-only failure modes.
- 20.6 Build a release tree and justify each included or excluded file.
- 20.7 Write an archive record for a fictional thesis deposit.
- 20.8 Challenge: conduct a two-person-style submission rehearsal and document evidence for every checklist decision.

20.22 Complete Practical Coding

The release file below includes PDF metadata and prints its own final checklist.

Complete practical file: submission-ready PDF shell

```
\documentclass{article}
\usepackage[
  pdftitle={Demonstration Submission},
  pdfauthor={Your Name},
  pdfsubject={Technical demonstration},
  pdfkeywords={typesetting, reproducibility},
  hidelinks
]{hyperref}

\title{Demonstration Submission}
\author{Your Name}
\date{\today}

\begin{document}
\maketitle

\section{Final content}
Replace this text with the checked document content.
```

```
\section*{Release checklist}
\begin{itemize}
  \item All placeholders and draft notes are resolved.
  \item Every figure, table, equation, and citation is checked.
  \item Page size, margins, and page numbering are correct.
  \item Fonts are embedded and the PDF was inspected page by page.
  \item The archived source reproduces this exact PDF.
\end{itemize}

\end{document}
```

Practical PDF checks

```
latexmk -C main.tex
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
pdfinfo main.pdf
pdffonts main.pdf
```

Code Lab

Run the clean build and both PDF checks. Compare the result with the verified submission specification, then archive the exact source and generated PDF.

Part IX

Capstone Projects

Three Complete Projects

The capstones turn isolated commands into complete scholarly workflows. Each project begins with a task, records planning decisions, uses one of the supplied independent source trees, compiles to a PDF, and ends with a release audit. The sample evidence is simulated; the projects teach structure rather than claim new scientific results.

Learning outcomes					
• Plan and complete an article, thesis, and technical report. • Navigate each supplied source tree and explain important commands. • Compile all three projects independently from clean folders. • Replace placeholders without inventing institutional or scholarly facts. • Apply document-specific final checklists and extension tasks. • Choose a document form from audience, purpose, evidence, and authority.					

21.1 How to use the capstones

Choose the form closest to your immediate work, but inspect all three. They share equations, tables, figures, citations, and reproducibility practices while serving different readers:

Form	Primary audience	Governing straint	con-	Main outcome
Article	Journal readers and reviewers	Current workflow	journal	Bounded scholarly claim
Thesis	Examiners and institution	Degree regulations		Sustained original argument
Report	Named decision owner	Commission controls	and	Traceable recommendation

The template folders contain complete source, not excerpts from this chapter. Build them before editing. Preserve an untouched copy so that a failed experiment can be compared with the known-good baseline.

Good Practice

For each capstone, keep a short decision log: requirement, choice, reason, evidence, and date. The log separates an externally mandated rule from a local preference and makes later migration easier.

21.2 Project 1: A publication-ready scholarly article

21.2.1 The task

Prepare a generic article titled “Agreement between two simulated measurement methods”. It must state that the data are simulated, define the signed paired difference, report the result in prose and a professional table, include an original figure, cite authentic sources, and provide complete declarations. Then create an anonymous review variant and a clean submission rehearsal.

“Publication-ready” here means structurally and technically ready to adapt. It does not mean accepted by every journal or scientifically publishable as empirical research. A real target journal and valid study are still required.

21.2.2 Planning notes

Record before editing:

- target article type and intended journal placeholder;
- question, unit, definition of d_i , and allowed claims;
- abstract word limit and section pattern to verify later;
- authorship, affiliations, correspondence, and anonymity plan;
- required declarations, data objects, figure files, and bibliography;
- build command and release evidence.

The generic template uses `article` because no publisher has been selected. This decision is explicit and reversible.

21.2.3 Expected folder structure

Complete article source tree

```
templates/scholarly_article/
|-- main.tex
|-- sections/
|   |-- abstract.tex
|   |-- introduction.tex
|   |-- methods.tex
|   |-- results.tex
|   |-- discussion.tex
|   |-- declarations.tex
```

```
|  \--- supplementary_methods.tex
|-- figures/method_comparison.pdf
|-- references.bib
\--- README.md
```

21.2.4 Complete source and important commands

The complete editable source is in `templates/scholarly_article/`. The root file loads packages, registers `references.bib`, defines title metadata, and inputs the section files. The conditional

Review identity switch
<pre>\newif\ifanonymous \anonymousfalse</pre>

selects the visible author block. It does not remove identity from all files or metadata, so the README requires a wider audit.

Methods defines $d_i = B_i - A_i$ and the results refer to table and figure labels with `\cref`. `siunitx` typesets quantities; `booktabs` supplies table rules; `biblatex` and `Biber` produce the reference list. Declarations retain placeholders where facts cannot be inferred.

21.2.5 Compilation instructions

Compile Project 1
<pre>cd templates/scholarly_article latexmk -pdf main.tex</pre>

Read the console and `main.log`; then inspect `main.pdf`. A clean build needs L^AT_EX and Biber passes, which `latexmk` schedules.

21.2.6 Final article checklist

- ☐ Target journal and current article instructions are recorded.
- ☐ Title, abstract, results, and conclusion make the same bounded claim.
- ☐ Author identities and contribution statements are approved.
- ☐ The anonymous package has been audited beyond the author line.
- ☐ Data, code, ethics, funding, interest, and acknowledgement statements are accurate.
- ☐ Table, figure, citations, supplement, units, and labels resolve.
- ☐ A clean package builds and its portal-style proof has been inspected.

21.2.7 Extension exercises

1. Add a second panel that shows signed difference against pair mean.
2. Generate the summary values from a script rather than fixed source.
3. Add a reporting-guideline checklist as a named supplement.
4. Migrate a copy into an official journal class and document every change.
5. Create separate anonymous and accepted-release manifests.

21.3 Project 2: A complete thesis or dissertation

21.3.1 The task

Prepare a generic five-chapter thesis that uses simulated method-agreement data to demonstrate a sustained structure. Include title and administrative placeholders, dedication, acknowledgements, abstract, contents, lists, abbreviations, literature context, methods, results, conclusion, appendix, and bibliography. Preserve explicit warnings wherever institutional wording must be verified.

21.3.2 Planning notes

Build an institutional compliance matrix before changing layout. Identify:

- official class, thesis manual, degree, institution, and deposit route;
- required order and wording of front matter;
- page size, margins, spacing, font, numbering, and accessibility rules;
- integrated or thesis-by-publication structure;
- joint-work, prior-publication, ethics, copyright, and permissions records;
- appendix, data, code, repository, embargo, and file-size requirements.

Do not fill a certificate or signature with invented approval. A visible placeholder is the correct state until authorised information exists.

21.3.3 Expected folder structure

Complete thesis source tree

```
templates/thesis/
|-- main.tex
|-- metadata.tex
|-- frontmatter/
|-- chapters/
|-- appendices/
|-- figures/
|-- references.bib
`-- README.md
```

The full tree contains individual files for all front-matter elements, five chapters, and supplementary material. `metadata.tex` owns repeated identity fields.

21.3.4 Complete source and important commands

The complete source is in `templates/thesis/`. The structural sequence is

Thesis numbering architecture

```
\frontmatter
% preliminary pages and generated lists
\mainmatter
% numbered chapters
\appendix
% lettered appendices
\backmatter
% reference list
```

`\input` inserts small front-matter units; `\include` gives each chapter its own auxiliary file and page start. `\listoffigures` and `\listoftables` read caption data from earlier passes. `\printbibliography` prints the cited sources and adds them to the contents using its documented heading option.

The project uses a manual abbreviations table because its list is small. A real thesis with hundreds of terms may adopt a glossary tool and document its extra build dependency.

21.3.5 Compilation instructions

Compile Project 2

```
cd templates/thesis
latexmk -pdf main.tex
```

Inspect Roman and Arabic page numbering, title and declaration links, contents, lists, running heads, appendix letter, citations, and the final reference list. Perform one build from a fresh copy without `\includeonly`.

21.3.6 Final thesis checklist

- ☐ The current institutional class, manual, and deposit rules are authoritative.
- ☐ Every identity field and required administrative statement is verified.
- ☐ Front, main, appendix, and back matter follow the mandated order.
- ☐ Chapter arguments connect and the synthesis addresses the thesis question.
- ☐ Joint work, prior publication, permissions, and contributions are declared.
- ☐ Figures, tables, equations, lists, references, data, and code are reproducible.

- The complete PDF passes compliance, accessibility, and deposit rehearsal.

21.3.7 Extension exercises

1. Replace the manual abbreviation table with a generated glossary workflow.
2. Add a sixth chapter and a cross-chapter synthesis table.
3. Implement an institution-supplied title page without duplicating metadata.
4. Create a thesis-by-publication contribution statement and permission log.
5. Build a public repository package that safely excludes restricted data.

21.4 Project 3: A professional technical report

21.4.1 The task

Prepare a decision-oriented report on the same simulated measurement comparison. The report must contain controlled cover metadata, document history, distribution information, an executive summary, scope, methods, findings, a table and original figure, limitations, owned recommendations, references, an evidence register, and a release checklist.

The report must say that the simulated analysis cannot approve an operational method. Its recommendation is a controlled validation with a prespecified tolerance.

21.4.2 Planning notes

Identify the commissioning statement, decision owner, primary and technical audiences, deadline, scope, exclusions, evidence classes, acceptance criteria, reviewers, approvers, distribution, and archival repository. Define version and status conventions before preparing the cover.

Separate the analysis finding from a tolerance judgement and the recommendation from the authorised decision. This distinction is the organising logic of the report.

21.4.3 Expected folder structure

Complete report source tree

```
templates/technical_report/
|-- main.tex
|-- sections/
|   |-- document_control.tex
|   |-- executive_summary.tex
|   |-- context.tex
|   |-- methods.tex
|   |-- findings.tex
|   |-- recommendations.tex
|   |-- limitations.tex
|   |-- appendices.tex
|-- figures/difference_plot.pdf
|-- references.bib
```

```
`-- README.md
```

21.4.4 Complete source and important commands

The complete source is in `templates/technical_report/`. Its `titlepage` creates the controlled cover. Roman numbering covers control material and lists;

Report page-number transition
<pre>\pagenumbering{roman} % control material and contents \clearpage \pagenumbering{arabic} % decision report body</pre>

starts the body at page 1. A restrained `fancyhdr` style repeats project and classification placeholders. `tabularx` allows findings and action tables to use the page width. The action plan connects each recommendation with an owner and completion evidence.

21.4.5 Compilation instructions

Compile Project 3
<pre>cd templates/technical_report latexmk -pdf main.tex</pre>

Inspect the cover and header classification together, compare the executive summary with the body, verify table wrapping and figure legibility, and ensure appendix evidence IDs resolve to preserved objects.

21.4.6 Final report checklist

- ☐ The commissioning statement and authorised decision owner are named.
- ☐ Title, ID, version, date, status, and classification agree.
- ☐ Executive-summary findings and conditions match the technical body.
- ☐ Scope, assumptions, criteria, deviations, uncertainty, and limitations are visible.
- ☐ Recommendations have accepted owners and objective completion evidence.
- ☐ Technical review, approval, permissions, and distribution are recorded.
- ☐ The immutable issued PDF, source, evidence, and manifest are archived.

21.4.7 Extension exercises

1. Add a risk register with defined likelihood and impact categories.

2. Create a landscape appendix for a genuinely wide assurance matrix.
3. Add a requirement-to-evidence traceability table.
4. Define central commands for report ID, status, version, and issue date.
5. Produce public and restricted distribution variants without changing findings.

21.5 Compare the same evidence across forms

The simulated mean difference appears in all three projects, but its rhetorical job changes. The article presents a scholarly result and limitation. The thesis places it inside a sustained research argument. The report asks what decision can responsibly follow. This is not merely formatting: audience and authority change what must be stated.

Science in Practice

One environmental monitoring programme might produce a methods article, a doctoral thesis containing several site studies, and an agency report recommending sampling changes. Shared analysis can support all three, yet each document needs its own scope, provenance, approvals, and release record.

21.6 Compile every example from one control point

The book repository supplies scripts that compile all standalone examples and all three templates. From the project root:

Repository-wide build checks

```
./scripts/compile_all_examples.sh
./scripts/compile_templates.sh
./scripts/check_project.sh
```

These checks complement, not replace, reading the PDFs. A passing compiler cannot confirm that an author contribution statement is true, a thesis declaration is authorised, or a report recommendation has an owner.

Common Error

Symptom: a capstone compiles after editing but its claims still refer to the simulated example while the table now contains real project values.

Cause: content was replaced file by file without a results-consistency and placeholder audit.

Repair: regenerate evidence from one analysis source, trace each value and limitation across the document, search all placeholders and simulation labels, then obtain scientific review.

21.7 Code lab: compile all three capstones

Code Lab

Treat compilation as a test, not a ceremonial final step. Run each build from its own folder, open every PDF, and record the result beside the relevant checklist. A capstone is complete only when both source and output survive the test.

Project 1: scholarly article

```
cd templates/scholarly_article
latexmk -pdf main.tex
```

Project 2: thesis or dissertation

```
cd templates/thesis
latexmk -pdf main.tex
```

Project 3: technical report

```
cd templates/technical_report
latexmk -pdf main.tex
```

For a genuine submission, replace the generic build command when a publisher or institution specifies a different engine or bibliography sequence. The project README should state that exception explicitly.

21.8 Chapter project

Mini Project

Build all three templates without editing them. Choose one and copy it into a new practice folder. Write a one-page project brief, complete its planning notes with fictional but explicitly labelled values, make one structural and one evidence change, and run the final checklist. Preserve the baseline and a diff or change record explaining why every modification was made.

21.9 Summary

The capstones provide complete, independent source for a scholarly article, thesis, and technical report. Each project connects a task and planning record to a transparent folder tree, important commands, build procedure, final audit, and extensions. The same L^AT_EX mechanisms serve all three, but audience, governing requirements, and decision authority change their scholarly form. Successful compilation is the

beginning of final verification, not its end.

21.10 Checklist

- ☐ All three unmodified template baselines compile independently.
- ☐ The chosen form matches audience, purpose, evidence, and authority.
- ☐ Planning notes distinguish requirements, conventions, and preferences.
- ☐ Complete source, assets, bibliography, README, and build command are present.
- ☐ Simulated evidence and unverified placeholders remain clearly labelled.
- ☐ Document-specific scientific and release checklists are complete.
- ☐ The edited capstone can be reconstructed from its archived project.

21.11 Exercises

21.1 Match six scholarly scenarios to article, thesis, or report and justify each.

21.2 Identify the shared and form-specific components of the three projects.

21.3 Explain five important commands from one template's root file.

21.4 Compile every template and compare page architecture and reference handling.

21.5 Replace one placeholder category without creating unverifiable facts.

21.6 Design a clean release manifest for each project.

21.7 Conduct a peer audit using the appropriate final checklist.

21.8 Challenge: derive two document forms from one reproducible analysis while preserving their different claims, audiences, and approvals.

21.12 Complete Practical Coding

This final single-file capstone combines structure, mathematics, a table, a figure, cross-references, and a bibliography.

Complete practical file: compact scholarly capstone

```
\begin{filecontents*}{capstone-references.bib}
@article{example2026,
  author      = {Researcher, Ada and Researcher, Ben},
  title       = {A Demonstration Measurement Study},
  journaltitle = {Example Journal},
```

```
year      = {2026},
volume    = {1},
pages     = {1--10}
}
\end{filecontents*}

\documentclass{article}
\usepackage{amsmath}
\usepackage{booktabs}
\usepackage[backend=biber,style=authoryear]{biblatex}
\usepackage{hyperref}
\usepackage[nameinlink,noabbrev]{cleveref}
\addbibresource{capstone-references.bib}

\title{Comparison of Two Demonstration Groups}
\author{Your Name}
\date{}

\begin{document}
\maketitle

\begin{abstract}
Two simulated groups were compared to demonstrate a complete workflow.
\end{abstract}

\section{Introduction}
\textcite{example2026} represents a checked source record. The
    objective is
to compare two demonstration means.

\section{Methods}
The mean was calculated as
\begin{equation}
\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.
\label{eq:mean}
\end{equation}

\section{Results}
\begin{table}[htbp]
\centering
\caption{Simulated group summaries.}
\label{tab:capstone}
\begin{tabular}{lrr}
\toprule
Group & Mean & SD \\
\midrule
A & 12.4 & 1.8 \\
B & 16.7 & 2.1 \\
\bottomrule
\end{tabular}
\end{table}
```

```
\end{tabular}
\end{table}

\begin{figure}[htbp]
  \centering
  \fbox{\rule{0pt}{35mm}\rule{.75\linewidth}{0pt}}
  \caption{Placeholder for a project-relative result figure.}
  \label{fig:capstone}
\end{figure}

\section{Discussion}
\Cref{tab:capstone} shows the larger demonstration mean in Group B.
The calculation follows \cref{eq:mean}; \cref{fig:capstone} marks where
the checked graphic belongs.

\section{Conclusion}
The file contains the core components of a reproducible article.

\printbibliography
\end{document}
```

Code Lab

Replace the simulated question, values, figure placeholder, and bibliography record with checked project material. Compile from a clean directory and inspect every cross-reference before release.

Command quick-reference guide

This guide recalls patterns taught in the book. It is not a substitute for the current class or package manual. Items shown as `<file>` or `<text>` must be replaced; square brackets normally mark optional arguments and braces required arguments.

Document skeleton and build

Source or command	Purpose
<code>\documentclass</code> <code>[11pt]{article}</code>	Select broad structure and supported options.
<code>\usepackage</code> <code>[options]{package}</code>	Load one documented extension in the preamble.
<code>\begin{document}</code> ...	Delimit visible document content.
<code>\end{document}</code>	
<code>latexmk -pdf main.tex</code>	Build pdf _L A _T E _X dependencies and repeat passes automatically.
<code>latexmk -C</code>	Remove generated build files in the current project; use deliberately.
<code>texdoc geometry</code>	Open locally installed documentation for a named package.

Text and structure

Pattern	Purpose
<code>\part{Title}</code>	Major book-level division.
<code>\chapter{Title}</code>	Chapter in classes that support it.
<code>\section{Title}</code>	Numbered section; use lower levels only when needed.
<code>\section*{Title}</code>	Unnumbered section; not necessarily in the contents.
Blank source line	Start a new paragraph.
<code>\emph{text}</code>	Semantic emphasis that adapts to context.
<code>\textbf{text}</code>	Bold text for a specific visual or structural reason.
<code>\texttt{text}</code>	Monospaced inline text such as short filenames or code.
<code>"quoted text"</code>	English typographic quotation marks.
<code>\footnote{text}</code>	Insert a numbered footnote at the point of reference.

Pattern	Purpose
<code>% & # _ { }</code>	Print common reserved characters literally.
<code>\url</code> <code>{https://example.org}</code>	Typeset a breakable literal URL.
<code>\href{URL}{meaningful text}</code>	Create a descriptive external link with <code>hyperref</code> .

Lists, quotations, and verbatim material

Environment or command	Purpose
<code>itemize</code> with <code>\item</code>	Unordered list.
<code>enumerate</code> with <code>\item</code>	Ordered sequence.
<code>description</code> with <code>\item[Term]</code>	Labelled definitions or categories.
<code>quote</code>	Short displayed quotation.
<code>quotation</code>	Multi-paragraph displayed quotation.
<code>\verb literal text </code>	Short inline literal text; delimiter cannot appear inside.
<code>verbatim</code>	Literal block for plain source examples.

Mathematics

Pattern	Purpose
<code>\$x^2\$</code> or <code>\(x^2\)</code>	Inline mathematics.
<code>\[x^2\]</code>	Unnumbered display.
<code>equation</code>	One numbered display.
<code>align</code> and <code>align*</code>	Aligned multi-line mathematics, numbered or unnumbered.
<code>x^{2}</code> and <code>x_{i}</code>	Superscript and subscript; braces group multiple tokens.
<code>\frac{a}{b}</code>	Fraction.
<code>\sqrt[n]{x}</code>	Square or n th root.
<code>\sum_{i=1}^n</code>	Summation with limits.
<code>\int_a^b</code>	Definite integral with an upright differential.
<code>f(x)\,,\mathrm{d}x</code>	
<code>\sin</code> , <code>\log</code> , <code>\exp</code>	Upright named mathematical operators.
<code>\operatorname{Var}</code>	Declare an occasional operator in place.
<code>\DeclareMathOperator</code> <code>{\Var}{Var}</code>	Declare a repeated operator in the preamble.
<code>\left(...\right)</code>	Automatically sized delimiters; use when needed.

Pattern	Purpose
<code>cases</code>	Piecewise definitions inside mathematics.
<code>pmatrix, bmatrix</code>	Matrices with round or square delimiters.
<code>\text{words}</code>	Short prose inside mathematics with <code>amsmath</code> .

Quantities and units with `siunitx`

Pattern	Purpose
<code>\num{12345.6}</code>	Format a number consistently.
<code>\qty{value}</code>	Typeset a number and unit.
<code>{unit expression}</code>	
<code>\unit{\micro\mole}</code>	Typeset a unit without a value.
<code>\qtyrange{2}{5}{\gram}</code>	Typeset a range with consistent unit handling.
<code>\qtylist{1;2;3}{\metre}</code>	Typeset a list of quantities.
<code>\DeclareSIUnit</code>	Define a documented specialist unit once.
<code>{\sampleunit}{symbol}</code>	
<code>S[table-format=3.2]</code>	Align numeric table entries by value and decimal marker.

Tables

Pattern	Purpose
<code>table</code>	Floating wrapper for caption, label, and a tabular object.
<code>\centering</code>	Centre content locally inside a float.
<code>tabular{lcr}</code>	Table grid with left, centred, and right columns.
<code>tabularx</code>	Table with a paragraph column that consumes available width.
<code>{\textwidth}{lX}</code>	
<code>longtable</code>	Table that may continue across pages.
<code>& and \\</code>	Separate cells and finish a row.
<code>\toprule, \midrule, \bottomrule</code>	Professional horizontal rules from <code>booktabs</code> .
<code>\multicolumn</code>	Span cells across columns.
<code>{n}{format}{text}</code>	
<code>\caption{text}</code>	Name and label a table; place the label after the caption.
<code>\label{tab:key}</code>	

Figures

Pattern	Purpose
<code>\includegraphics</code> <code>[width=.8\linewidth]</code> <code>{figures/result.pdf}</code>	Insert a project-relative image with <code>graphicx</code> .
<code>figure</code>	Floating wrapper for an image, caption, and label.
<code>\caption[Short</code> <code>entry]{Full explanatory</code> <code>caption}</code>	Provide a short list entry and full caption.
<code>subfigure</code> <code>[t]{.48\textwidth}</code>	Create a labelled panel with subcaption.
<code>\graphicspath</code> <code>{figures/}</code>	Declare a searched graphics directory; explicit paths can be clearer.
<code>\rotatebox</code> , <code>\resizebox</code>	Transform content with <code>graphicx</code> ; use sparingly.

Labels and cross-references

Pattern	Purpose
<code>\label{fig:growth}</code>	Store the current numbered target under a unique semantic key.
<code>\ref{fig:growth}</code>	Print only the target number.
<code>\pageref{fig:growth}</code>	Print its page number.
<code>\cref{fig:growth}</code>	Print type and number with <code>cleveref</code> .
<code>\Cref{fig:growth}</code>	Capitalised form at the beginning of a sentence.
<code>\eqref{eq:model}</code>	Parenthesised equation number with <code>amsmath</code> .
<code>\autoref{sec:methods}</code>	Hyperlinked type and number with <code>hyperref</code> .

Use consistent prefixes such as `chap:`, `sec:`, `fig:`, `tab:`, `eq:`, and `app:`. They aid navigation; L^AT_EX does not require those exact strings.

Citations and bibliography with biblatex

Pattern	Purpose
<code>\usepackage[options]</code> <code>{biblatex}</code>	Select the bibliography system and style.
<code>\addbibresource</code> <code>{references.bib}</code>	Register a database in the preamble.
<code>\parencite{key}</code>	Parenthetical citation in an author–year style.
<code>\textcite{key}</code>	Author integrated into the sentence.
<code>\citeauthor{key}</code> ; <code>\citeyear{key}</code>	Print selected citation components when justified.

Pattern	Purpose
<code>\printbibliography</code>	Print cited works at the chosen location.
<code>@article{key, ...}</code>	A BibL ^A T _E X/BibT _E X database entry; verify every field.

Long documents and page control

Pattern	Purpose
<code>\input{file}</code>	Insert a small source unit exactly at this point.
<code>\include{chapters/file}</code>	Insert a large page-starting unit with separate auxiliary data.
<code>\includeonly</code> <code>{chapters/methods}</code>	Draft selected included files; remove for the final full build.
<code>\frontmatter,</code> <code>\mainmatter, \backmatter</code>	Apply book-class matter conventions.
<code>\appendix</code>	Change later chapter or section numbering to appendix form.
<code>\tableofcontents</code>	Generate contents from headings.
<code>\listoffigures,</code> <code>\listoftables</code>	Generate lists from captions.
<code>\pagenumbering{roman}</code>	Select a numbering representation and reset the page counter.
<code>\newcommand</code> <code>{\Name}[n]</code> <code>{definition}</code>	Define a shared semantic command.
<code>\makeindex, \index{term},</code> <code>\printindex</code>	Collect and print an index with a configured index tool.

Spacing, pages, and drafts

Pattern	Purpose
<code>\usepackage</code> <code>[margin=25mm]</code> <code>{geometry}</code>	State page geometry for a generic document.
<code>\pagestyle{fancy};</code> <code>\fancyhf{}</code>	Select and clear a fancyhdr page style.
<code>\thispagestyle{empty}</code>	Change the style of the current page.
<code>titlepage</code>	Create a separately composed title or cover page.
<code>\onehalfspacing,</code> <code>\doublespacing</code>	Apply documented draft spacing with <code>setspace</code> .

Pattern	Purpose
<code>landscape</code>	Rotate a page using <code>pdflscape</code> when a wide object justifies it.
<code>\hypersetup</code> <code>{pdftitle={...}}</code>	Configure link and PDF metadata options supported by <code>hyperref</code> .

Release reminders

Build from the documented main file; read the first meaningful error; search for unresolved labels, citations, and placeholders; compile examples and templates; inspect the log; render and inspect every final PDF page; verify metadata and fonts where required; and archive the exact source, PDF, supplements, manifest, permissions, and environment record.

Common symbols

Symbols acquire meaning from a discipline and document; this guide gives L^AT_EX input and a common reading, not a universal definition. Define every symbol in your work, distinguish variables from units, and consult the relevant style or standard. Type the examples in mathematics mode unless stated otherwise.

Greek letters

Output	Input	Uppercase	Example use
α	<code>\alpha</code>	A (<code>A</code>)	angle, level, coefficient
β	<code>\beta</code>	B (<code>B</code>)	coefficient, type-II error
γ	<code>\gamma</code>	Γ (<code>\Gamma</code>)	rate, gamma function
δ	<code>\delta</code>	Δ (<code>\Delta</code>)	small change; finite change
ϵ, ε	<code>\epsilon</code> , <code>\varepsilon</code>	E	error or small positive value
ζ	<code>\zeta</code>	Z	damping ratio or function
η	<code>\eta</code>	H	efficiency or parameter
θ, ϑ	<code>\theta</code> , <code>\vartheta</code>	Θ (<code>\Theta</code>)	angle or unknown parameter
ι	<code>\iota</code>	I	index or embedding
κ	<code>\kappa</code>	K	curvature or agreement coefficient
λ	<code>\lambda</code>	Λ (<code>\Lambda</code>)	wavelength, eigenvalue, rate
μ	<code>\mu</code>	M	population mean or micro prefix
ν	<code>\nu</code>	N	frequency or degrees of freedom
ξ	<code>\xi</code>	Ξ (<code>\Xi</code>)	random variable or co-ordinate
o	Latin o	O	omicron normally uses the Latin shape
π, ϖ	<code>\pi</code> , <code>\varpi</code>	Π (<code>\Pi</code>)	constant or product
ρ, ϱ	<code>\rho</code> , <code>\varrho</code>	P	density or correlation
σ, ς	<code>\sigma</code> , <code>\varsigma</code>	Σ (<code>\Sigma</code>)	standard deviation or sum
τ	<code>\tau</code>	T	time constant or rank correlation
v	<code>\upsilon</code>	Υ (<code>\Upsilon</code>)	particle or variable

Output	Input	Uppercase	Example use
ϕ, φ	<code>\phi, \varphi</code>	Φ (<code>\Phi</code>)	angle, potential, distribution function
χ	<code>\chi</code>	X	chi-squared statistic component
ψ	<code>\psi</code>	Ψ (<code>\Psi</code>)	wavefunction or function
ω	<code>\omega</code>	Ω (<code>\Omega</code>)	angular frequency; sample space

Relations and operations

Output	Input	Common reading
$a = b$	<code>a=b</code>	equal
$a \neq b$	<code>a\neq b</code>	not equal
$a < b, a > b$	<code>a<b, a>b</code>	less than, greater than
$a \leq b, a \geq b$	<code>\leq, \geq</code>	less/greater than or equal
$a \approx b$	<code>\approx</code>	approximately equal
$a \sim b$	<code>\sim</code>	similar, distributed as, or asymptotic relation; define context
$a \propto b$	<code>\propto</code>	proportional to
$a \equiv b$	<code>\equiv</code>	identically or definitionally equivalent; define use
$a \pm b$	<code>\pm</code>	plus or minus
$a \mp b$	<code>\mp</code>	minus or plus
$a \times b$	<code>\times</code>	multiplication or Cartesian product
$a \cdot b$	<code>\cdot</code>	centred product
$a \div b$	<code>\div</code>	division symbol; fractions are usually clearer
$a \circ b$	<code>\circ</code>	composition or circle operation
$a \cap b, a \cup b$	<code>\cap, \cup</code>	intersection and union
$a \in A, a \notin A$	<code>\in, \notin</code>	is/is not an element of
$A \subseteq B$	<code>\subseteq</code>	subset, allowing equality
$A \supseteq B$	<code>\supseteq</code>	superset, allowing equality
$\emptyset$	<code>\emptyset</code>	empty set
$\forall x$	<code>\forall</code>	for all
$\exists x$	<code>\exists</code>	there exists
$p \wedge q, p \vee q$	<code>\wedge, \vee</code>	logical and, logical or
$\neg p$	<code>\neg</code>	logical negation
$p \Rightarrow q$	<code>\Rightarrow</code>	implies
$p \Leftrightarrow q$	<code>\Leftrightarrow</code>	if and only if

Arrows, calculus, and linear algebra

Output	Input	Common reading
$a \rightarrow b$	<code>\to</code> or <code>\rightarrow</code>	maps, tends, or reacts; define context
$a \mapsto b$	<code>\mapsto</code>	maps an element to an image
$a \leftrightarrow b$	<code>\leftrightharrow</code>	bidirectional relation
$\uparrow, \downarrow$	<code>\uparrow</code> , <code>\downarrow</code>	increase/decrease or reaction product notation
$\sum_{i=1}^n x_i$	<code>\sum_{i=1}^n x_i</code>	finite sum
$\prod_{i=1}^n x_i$	<code>\prod</code> with limits	finite product
$\int_a^b f(x) \, dx$	<code>\int</code> with limits	definite integral
$\iint_A f \, dA$	<code>\iint</code>	double integral
$\partial f / \partial x$	<code>\partial</code>	partial derivative
∇f	<code>\nabla</code>	gradient; operator depends on context
$\lim_{x \rightarrow 0} f(x)$	<code>\lim_{x \rightarrow 0}</code>	limit
∞	<code>\infty</code>	infinity
$\mathbf{x}$	<code>\mathbf{x}</code>	bold Latin vector by one convention
$\boldsymbol{\theta}$	<code>\boldsymbol{\theta}</code>	bold Greek symbol
$\mathbf{A}^T$	<code>\mathbf{A}^T</code>	matrix transpose
$\mathbf{A}^{-1}$	<code>\mathbf{A}^{-1}</code>	inverse matrix
$\det(\mathbf{A})$	<code>\det</code>	determinant
$\ \mathbf{x}\ $	<code>\lVert ... \rVert</code>	norm; define which norm
$\langle x, y \rangle$	<code>\langle ... \rangle</code>	inner product or ordered pairing

Statistics and probability

Output	Input	Possible meaning
n, N	Latin letters	sample size, population size; define explicitly
$\bar{x}$	<code>\bar{x}</code>	sample mean
$\hat{\theta}$	<code>\hat{\theta}</code>	estimator or estimate
s, σ	Latin and <code>\sigma</code>	sample and population standard deviation by a common convention
s^2, σ^2	superscript 2	sample and population variance
$\text{SE}(\hat{\theta})$	<code>\operatorname{SE}</code>	standard error
$P(A)$	Latin P	probability of event A
$P(A \mid B)$	<code>\mid</code>	conditional probability
$X \sim \mathcal{N}(\mu, \sigma^2)$	<code>\sim \mathcal{N}</code>	normally distributed, if defined

Output	Input	Possible meaning
$E(X)$	Latin E	expectation; an operator command can enforce style
$\text{Var}(X)$	<code>\operatorname{Var}</code>	variance
r, ρ	Latin and <code>\rho</code>	sample and population correlation by a common convention
p	Latin p	probability or p -value; distinguish the uses
α	<code>\alpha</code>	prespecified significance level or another parameter
CI	<code>\mathrm{CI}</code>	confidence interval abbreviation after definition

Scientific notation and discipline-specific forms

Output or concept	Suggested input	Note
6.02×10^{23}	<code>\num{6.02e23}</code>	Scientific notation with <code>si-units</code> .
9.81 m s^{-2}	<code>\qty{value}{unit expression}</code>	Value and SI unit.
H_2O	<code>\mathrm{H_2O}</code>	Basic upright chemical formula; complex chemistry benefits from a chemistry package.
Chemical equation	<code>\ce{2H2 + O2 -> 2H2O}</code>	Optional; available only after deliberately loading <code>mhchem</code> .
<i>Homo sapiens</i>	<code>\textit{Homo sapiens}</code>	Genus and species; follow disciplinary rules for authorities and strains.
5'-ACGT-3'	mixed math and text	A short orientation example; databases use machine-readable sequence text.
$E = mc^2$	ordinary mathematics	Define energy, mass, speed, and units in context.
ΔG	<code>\Delta G</code>	Finite change in Gibbs energy when defined.
$\lambda = 532 \text{ nm}$	<code>\qty{532}{\nano\metre}</code>	Keep the unit upright and spacing controlled.

Common error-message guide

Read the first meaningful error in the complete log, note its file and line, then inspect the surrounding logical unit. The message reports where TeX noticed a failure, not always where it began. Preserve the failing source and change one factor at a time.

Message or symptom	Likely causes	First checks and repairs
Undefined control sequence	Misspelt command; defining package or custom file not loaded; command belongs to another engine or package version.	Read the displayed token; compare spelling and case; locate its authoritative definition; load the necessary package once or correct the command.
Missing } inserted or runaway argument	Unclosed brace or command argument, often earlier than the reported line.	Inspect backwards through the paragraph, caption, cell, or heading; use editor brace matching; reduce nested commands.
Extra }, or forgotten \$	Closing brace without a matching opening brace; math delimiter mismatch.	Check group and math boundaries rather than deleting the reported character blindly.
\begin{...} ended by \end{...}	Environment names differ or environments overlap instead of nesting.	Pair every begin and end by name; indent nested source; reduce the failing region to one environment.
\begin{document} ended by \end{...}	An inner environment never closed, so TeX reaches the document end.	Search backwards for the last unmatched begin; compare environment counts only as a clue, since listings and comments may contain examples.
File name.sty not found	Package not installed, false package name, or incomplete TeX installation.	Confirm the package on CTAN or official source; install through the supported distribution manager, or use a documented alternative.

Message or symptom	Likely causes	First checks and repairs
File <code>figure.pdf</code> not found	Wrong relative path, case, extension, build directory, or missing asset.	List the actual file; match case exactly; compile from the documented main directory; keep the repair project-relative.
I can't write on file ...	Output directory absent, file locked, permission denied, or storage unavailable.	Verify the exact output path and permissions; create required auxiliary subdirectories; close conflicting processes; do not redirect into protected locations without authority.
Option clash for package ...	The class or another file already loaded the package with different options.	Find the first load in the log; remove duplication or pass compatible options before loading; test the smallest package interaction.
Command ... already defined	A custom name collides with the class, a package, or an earlier definition.	Choose a distinctive semantic name; use renewal only when intentional and documented; never overwrite a core command casually.
Environment ... undefined	Misspelt environment or missing defining package.	Check exact case and documentation; load the correct package in the preamble; do not invent a same-named empty environment.
Unicode character ... not set up	Unsupported character, copied typographic symbol, unsuitable engine/font, or incorrect file encoding.	Identify the code point; use semantic L ^A T _E X input for mathematical notation or configure an engine and font that intentionally supports the script.
Missing \$ inserted	Math-only command used in prose, reserved underscore/caret in text, or unclosed mathematics.	Inspect the boundary; put mathematical content in math mode; escape a literal underscore; do not add random dollar signs.

Message or symptom	Likely causes	First checks and repairs
Misplaced alignment tab character &	Literal ampersand in prose or an alignment marker outside a supported table or alignment.	Use <code>\&</code> for a literal ampersand; check table row and environment structure for intended alignment.
Extra alignment tab has been changed to <code>\cr</code>	Too many cell separators for the declared table columns.	Count the cells in the failing row, remembering that n columns need $n - 1$ ordinary separators; inspect multicolumn spans.
There's no line here to end	Forced line break used where no line is active, often after vertical space or a section command.	Remove unnecessary <code>\;</code> ; use a blank line for paragraphs and structural commands for headings.
Not in outer par mode	A float or similar outer object was placed inside a box, minipage, or another float where it cannot float.	Move the float outward; inside a minipage use non-floating content with an appropriate caption mechanism if truly needed.
Float(s) lost	Float placed in a context that discards or cannot place it; invalid nesting.	Check whether a figure/table is inside another float, box, or output-sensitive environment; isolate the smallest float.
Too many unprocessed floats	A backlog of large or overly constrained floats cannot be placed.	Offer reasonable placement choices, resize or rearrange floats, insert explanatory prose, and avoid long runs of forced figures.
Overfull <code>\hbox</code>	Content extends horizontally: long URL/code, wide equation, table, or word.	Inspect the exact PDF line; allow breaks, revise or split content, change table architecture, or use justified landscape layout.
Underfull <code>\hbox</code>	Loose spacing in a constrained line, narrow cell, ragged region, or forced break.	Inspect visually; reword or widen the region when distracting. Some warnings are harmless and should be documented rather than hidden.

Message or symptom	Likely causes	First checks and repairs
<code>Underfull \vbox</code>	Page could not be filled evenly, often near floats or forced breaks.	Inspect page architecture; remove unnecessary page breaks or float constraints; accept intentional short pages when structurally correct.
Label(s) may have changed	Cross-reference information changed on this pass.	Run <code>L^AT_EX</code> again or use <code>latexmk</code> ; investigate only if the warning never stabilises.
Reference ... undefined	Missing, misspelt, duplicate, or stale label; insufficient passes.	Compare key exactly; place label after the numbered caption or heading; ensure uniqueness; rebuild fully.
Citation ... undefined	Bad citation key, database not registered, backend not run, or stale build data.	Check key and <code>.bib</code> path, run the configured Biber-/BibTeX sequence, then repeat <code>L^AT_EX</code> .
Bibliography heading appears but entries do not	No cited entries, backend failure, wrong database, refsection scope, or filters.	Inspect backend log; cite a known valid key in an MWE; confirm whether the style prints cited works only.
Please (re)run Biber	Bibliography control data changed or no current <code>.bbl</code> exists.	Run Biber through the documented build tool, then rerun <code>L^AT_EX</code> until stable; do not run BibTeX as a substitute.
Rerun to get cross-references right	Contents, labels, bookmarks, or citations changed.	Use <code>latexmk</code> ; if the state changes indefinitely, compare the changing auxiliary files for non-deterministic content.
destination with the same identifier	Duplicate hyperlink anchor, often from resetting page numbering while anchors are active or duplicating labels.	Keep PDF page anchors unique, disable them around non-numbered duplicate title pages when appropriate, and restore them before normal numbering.

Message or symptom	Likely causes	First checks and repairs
Token not allowed in a PDF string	Mathematics or formatting command entered a bookmark or metadata string.	Supply a plain-text optional heading or use documented PDF-string alternatives; check the visible bookmark.
Font shape ... not available	Requested family/series/shape lacks a matching font and L ^A T _E X substitutes.	Check the font package's supported shapes; accept documented substitution or choose a family with the required design.
PDF has blank or stale content despite a successful message	Wrong main file, old output opened, cached viewer, output-directory mismatch, or content excluded by a conditional.	Compare output path and modification time; identify the main file; inspect conditionals and <code>\includeonly</code> ; make a clean build when justified.

A compact first-aid decision tree

1. If compilation stops, preserve the log and repair the first meaningful error.
2. If it completes with undefined items, verify keys and run the complete multi-pass toolchain.
3. If it completes with box or float warnings, inspect the named pages.
4. If the PDF is scientifically wrong, trace content to data and analysis; the absence of a TeX warning is irrelevant.
5. If a problem survives, reduce it to an MWE and consult authoritative documentation with engine and package versions.

Clear auxiliary files only when evidence suggests stale or incompatible state, using the project build tool in the exact project directory. After any repair, restore the complete source and verify the final PDF.

Package guide

Load a package to solve a named requirement. Check its current manual, class compatibility, engine support, licence, and build implications. A package listed here is a useful category example, not a universal recommendation.

Package or tool	Main purpose	Use and cautions
<code>amsmath</code>	Structured displayed mathematics.	Provides <code>align</code> , <code>gather</code> , <code>cases</code> , <code>\text</code> , and robust equation tools. Prefer it to legacy display environments.
<code>amssymb</code> and <code>amsfonts</code>	Additional mathematical symbols and fonts.	Load when their symbol sets are used; verify that a publisher class does not already provide them.
<code>mathtools</code>	Extensions and refinements to <code>amsmath</code> .	Useful for paired delimiters, specialised alignment, and mathematical tools. Read its manual before adopting convenience commands.
<code>amsthm</code>	Theorem-like environments and proof styling.	Define theorem structures once and follow the mathematical venue's numbering convention. Some classes provide their own theorem interface.
<code>siunitx</code>	Numbers, quantities, units, and numeric table columns.	Use <code>\num</code> , <code>\qty</code> , <code>\unit</code> , and <code>S</code> . Version 3 syntax differs from older examples; consult the installed manual.
<code>graphicx</code>	Insert and transform external graphics.	Use project-relative paths and meaningful placed size. Keep editable or generated figure sources and permission records.
<code>caption</code>	Caption formatting and options.	Generic classes benefit from its interface, but publisher classes may control captions and conflict with local restyling.

Package or tool	Main purpose	Use and cautions
subcaption	Subfigures and subtables.	Provides panel environments and labels. It builds on <code>caption</code> ; verify class compatibility and avoid the obsolete <code>subfigure</code> package.
booktabs	Restrained professional table rules.	Use top, middle, and bottom rules; avoid vertical rules and double-rule grids unless a domain-specific requirement says otherwise.
array	Extended column definitions.	Useful for custom alignment and paragraph-column modifiers. Keep definitions readable and local to a documented need.
tabularx	Tables with columns that fill a target width.	The <code>X</code> column wraps prose. Combine cautiously with numeric columns and avoid over-constrained width specifications.
longtable	Tables that break across pages.	Runs directly in the page stream rather than as an ordinary float. Define repeating headers and continuation rules.
xcolor	Named colours and colour models.	Use colour sparingly, maintain contrast and grayscale meaning, and obey venue rules. Colour cannot be the only carrier of information.
microtype	Subtle typographic protrusion and font expansion.	Often improves justified text under pdfL ^A T _E X. Support depends on engine and font; inspect rather than expecting a visible effect.
geometry	Explicit page dimensions and margins.	Appropriate for generic documents with known requirements. Do not override publisher or institutional page design without permission.

Package or tool	Main purpose	Use and cautions
<code>fancyhdr</code>	Running headers and footers.	Clear inherited fields, set a sufficient head height, and define a separate plain style for opening pages when needed.
<code>titlesec</code>	Heading format and spacing in generic classes.	Useful for a controlled house design; can conflict with specialist classes and should not be used to imitate publisher formatting.
<code>setspace</code>	Single, one-and-a-half, and double spacing.	Use when a verified review or thesis requirement asks for it. It handles more contexts than guessing a baseline-stretch value.
<code>pdfscape</code>	Landscape pages with PDF rotation metadata.	Use for genuinely wide tables or figures. Inspect orientation, headers, and printed output.
<code>multicol</code>	Balanced multiple text columns.	Suitable for compact reference material; awkward for wide mathematics, figures, reading order, and accessibility.
<code>enumitem</code>	List labels and spacing.	Define restrained reusable list styles. Excessively compressed lists reduce readability and may violate a class design.
<code>listings</code>	Typeset source code without shell execution.	Configure language, wrapping, escape behaviour, and captions. It is a typesetting tool, not a syntax validator.
<code>minted</code>	Syntax highlighting through Pygments.	Can provide richer highlighting but normally requires external execution and a Python dependency. Use only when the build and security policy permit it.
<code>tcolorbox</code>	Breakable pedagogical or technical boxes.	Offers many optional libraries and dependencies. Keep visual hierarchy modest and verify page breaks in long boxes.

Package or tool	Main purpose	Use and cautions
<code>csquotes</code>	Context-sensitive quotation commands and bibliography integration.	Recommended with many <code>biblatex</code> language workflows. It does not decide whether a quotation is licensed or accurately attributed.
<code>biblatex</code>	Citation and bibliography management in L ^A T _E X.	Select a style and backend deliberately. Publisher workflows may require BibTeX, a generated <code>.bbl</code> , or their own system instead.
<code>Biber</code>	Bibliography backend commonly used with <code>biblatex</code> .	It is a separate program. Run it through <code>latexmk</code> or the documented sequence; do not substitute BibTeX when the project specifies Biber.
<code>natbib</code>	Author–year and numeric citation commands for BibTeX-era styles.	Widely used by publisher classes. Do not load alongside <code>biblatex</code> unless explicit compatibility documentation supports the setup.
<code>hyperref</code>	Links, anchors, bookmarks, and PDF metadata.	Usually load late. Supply plain bookmark text for complex headings and audit anonymous metadata.
<code>cleveref</code>	Type-aware cross-references such as Figure 3.	Load after <code>hyperref</code> ; configure names for custom environments and preserve semantic labels.
<code>varioref</code>	Contextual page-aware references.	Useful in some long works; automatic phrases can become awkward after pagination changes, so proofread the final text.
<code>makeidx</code>	Basic index collection and printing.	Requires an indexing pass. Index terms editorially rather than marking every occurrence.
<code>imakeidx</code>	Enhanced index management.	Supports multiple indexes and build integration; may require shell or build configuration depending on the workflow.

Package or tool	Main purpose	Use and cautions
<code>glossaries</code>	Generated glossaries, acronym lists, and symbols.	Powerful for large projects but adds database and build complexity. Document the glossary tool and define terms at first use in prose.
<code>acronym</code>	Focused acronym definition and use.	Can suit a moderate acronym list. Compare with <code>glossaries</code> and a manual list rather than loading both without a plan.
<code>mhchem</code>	Chemical formulae and reaction notation.	Use <code>\ce</code> according to the current manual. Verify conventions for isotopes, charges, states, equilibrium, and specialist chemistry.
<code>chemfig</code>	Programmatic chemical structure diagrams.	Useful for editable structures; complex source and disciplinary validation are still required.
<code>algorithmicx</code> with <code>algpseudocode</code>	Structured pseudocode.	Define inputs, outputs, and semantics in prose. Do not confuse an algorithm float, its pseudocode language, and executable validation.
<code>tikz</code> and PGF	Programmatic vector diagrams and plots.	Excellent for reproducible simple diagrams. Large graphics can slow builds; externalise only with a documented toolchain and preserve original source.
<code>pgfplots</code>	Data-driven scientific plots using PGF/TikZ.	Set a compatibility version, keep source data, and audit axes and uncertainty. For large data, generate a vector figure in analysis software instead.
<code>standalone</code>	Compile individual figures or snippets independently.	Useful for original TikZ figures. Record the source-to-PDF build and include only the derived PDF in workflows that prohibit shell execution.

Package or tool		Main purpose	Use and cautions
lineno		Review line numbers.	Inspect behaviour around equations, floats, and two-column layouts. A publisher class may supply its own line-number option.
changes latexdiff	and	Visible revision markup and source comparison.	Review generated markup carefully, especially commands and moved text. Remove change apparatus from a clean accepted release unless requested.
latexmk		Dependency-aware build automation.	Configure the main engine, bibliography, index, glossary, and output directory in a project-level file. A successful run still needs log and PDF inspection.

Package-selection checklist

1. Name the missing capability.
2. Check whether the class or kernel already provides it.
3. Read the current package introduction, minimal example, options, and compatibility notes.
4. Add the package once with the smallest needed configuration.
5. Compile an MWE and the complete document.
6. Record extra programs, permissions, or security requirements.
7. Remove the package if its capability is no longer used.

Glossary

Absolute path

A file location beginning from a file-system root or user-specific location. It is normally unsuitable for a portable project.

Accessibility

The practice of making content usable by people with varied sensory, motor, and cognitive access needs, including structured navigation and meaningful alternatives to visual information.

Affiliation

The institution or organisation associated with an author under the target venue's rules.

Argument

Information supplied to a command. Required arguments are commonly placed in braces; optional arguments in square brackets.

Auxiliary file

A generated file used between compilation passes to store references, contents, citations, index entries, or other state.

Backend

A separate program used by a $\text{\LaTeX}$ package, such as Biber processing bibliography control data for `biblatex`.

Back matter

Material after the main body, such as references, glossary, solutions, index, or author information.

Baseline

The imaginary line on which most letters sit; also a known version used as a comparison point in testing.

Bibliography database

A structured text file, commonly ending in `.bib`, that stores verified source metadata under citation keys.

Bookmark

A navigational entry in a PDF outline, usually derived from a structural heading.

Build

The complete process that transforms source and dependencies into output, including repeat L^AT_EX, bibliography, glossary, or index passes.

Build log

The engine's technical record of files, messages, warnings, errors, fonts, and output from a compilation.

Caption

Text that identifies and explains a figure or table sufficiently for its role in the document.

Checksum

A computed digest used to detect whether a file's bytes have changed; it does not establish scientific validity or authorship.

Citation

A reference marker in the text connecting a claim or discussion to a bibliography record.

Citation key

The unique database identifier used inside a citation command, for example `\lmpart1994`.

Class option

A supported choice passed in square brackets to `\documentclass`, such as `base` size or `two-sided` layout.

Clean build

Compilation after removing or isolating generated state so that missing dependencies and stale outputs cannot hide easily.

Command

A L^AT_EX instruction, normally beginning with a backslash.

Compilation

Interpretation of source to produce output and diagnostic records.

Compiler or engine

The program that reads TeX/L^AT_EX source, such as pdfTeX, XeTeX, or LuaTeX in an appropriate format.

Compliance matrix

A table mapping each external requirement to its implementation and verification evidence.

Conditional

Source logic that selects one branch, such as an anonymous or identified author block.

Continuous integration

An automated service that builds or tests a versioned project after defined events.

Controlled vocabulary

An authorised set of terms used for consistent indexing, metadata, or domain description.

Cross-reference

A generated connection to a numbered or named target such as a figure, equation, section, or appendix.

Data dictionary

A structured explanation of variables, units, codes, formats, missing values, and constraints in a data set.

Declaration

A formal statement about matters such as funding, interests, ethics, authorship, originality, or data access. Required wording must be verified.

Dependency

A class, package, font, program, asset, or other object required to reproduce a build.

Document body

Content between `\begin{document}` and `\end{document}`.

Document class

The broad structural and visual foundation selected by `\documentclass`.

Document preparation system

A system in which authors describe content and structure from which formatted output is constructed.

Effective resolution

Raster pixels divided by the image's final physical dimensions, commonly expressed as pixels per inch.

Engine

See *compiler or engine*.

Environment

A structured region opened by `\begin` and closed by a matching `\end`.

Error

A condition that prevents the engine from interpreting source reliably; the first meaningful error is usually the best starting point.

Evidence register

A controlled list of data, code, figures, reviews, approvals, and other objects supporting a report or release.

Float

A figure or table object whose final placement L^AT_EX chooses under declared preferences and page constraints.

Font embedding

Inclusion of a font or subset inside the PDF so the viewer need not obtain it separately. Embedding does not settle licence.

Front matter

Preliminary material such as title, declarations, abstract, contents, and lists, often with distinct numbering.

Generated file

An output created from source, such as a PDF figure, table fragment, auxiliary record, or compiled document.

Glossary

An alphabetical or structured list defining specialised terms used in a document.

Identifier

A stable name assigned to an object, such as a DOI, accession number, report ID, figure label, or repository record.

Index

An editorial map from reader search terms to useful pages, not merely an alphabetical word list.

Input file

A source, data, bibliography, figure, or configuration file consumed by a build.

Interoperability

The capacity of source and outputs to work across relevant systems, tools, or repositories under documented conditions.

Label

A unique source key attached to a numbered target for later cross-reference.

L^AT_EX project

The complete group of source, bibliography, figure, data, configuration, documentation, permission, and build files needed for a defined output.

Licence

Terms granting specified permissions to use, modify, or share a copyrighted work. A public URL is not itself a licence.

Line breaking

The typographic process of distributing words across lines under width, hyphenation, and spacing constraints.

Log

See *build log*.

Main document

The source file given to the engine first; it normally owns the class, shared setup, and document order.

Manifest

A list of files in an exact release, sometimes accompanied by sizes, versions, checksums, origins, or access classes.

Metadata

Structured descriptive information such as title, authors, abstract, keywords, version, date, identifiers, and rights.

Minimal working example (MWE)

The smallest complete source that still reproduces a behaviour or problem.

Monospaced

A font design in which characters normally occupy equal horizontal width, often used to distinguish source and filenames.

Nomenclature

An organised list of symbols and their meanings.

Option

A command or package choice commonly placed in square brackets; accepted options are defined by the receiving interface.

Orphan

A typographic term often used for the first line of a paragraph left at the bottom of a page; definitions vary across style guides.

Output directory

The location assigned to generated build files.

Overfull box

A TeX box whose content exceeds its allocated dimension, potentially producing text beyond a margin.

Package

A specialised extension loaded for capabilities not supplied by the selected class and L^AT_EX kernel alone.

Page anchor

An internal PDF destination associated with a page. Duplicate page numbers need distinct anchors.

Persistent identifier

A managed identifier intended to remain stable, such as a DOI; persistence depends on its governance and metadata.

Placeholder

An explicit marker for information that is unavailable or unverified. It must be resolved or deliberately retained before release.

Preamble

Source before `\begin{document}`, containing the class and document-wide setup.

Proof

A document version intended for checking rather than final distribution; a submission-system proof is a newly generated output.

Provenance

The documented origin, transformations, responsibility, and history of data, code, figures, or claims.

Raster image

An image represented by pixels; useful for photographs and microscopy but resolution-dependent at placed size.

Reference list

The formatted collection of cited source records.

Relative path

A file location expressed from the project or current document directory, enabling portability when the structure is preserved.

Release

An identified, controlled set of source and outputs intended for a recipient, submission, deposit, or publication.

Repository

A managed storage and discovery location for source, data, software, documents, or records, with its own policies and identifiers.

Reproducible compilation

The ability of an authorised reader to obtain the stated document output from preserved source, dependencies, and instructions.

Reserved character

A character with a syntactic role in $\text{\LaTeX}$ source that needs special treatment when printed literally.

Resolution

The density or number of samples in an image. Venue rules usually distinguish raster type and final placed size.

Semantic command

A custom command named for meaning, allowing notation or presentation to be changed consistently in one definition.

Shell escape

A TeX capability that permits external commands. It can support some packages but expands the build's security surface.

Source file

Editable plain text containing content and $\text{\LaTeX}$ instructions, commonly with extension `.tex`.

Structured data

Data organised under a documented schema so variables and records can be interpreted and processed consistently.

Submission package

The exact set of files sent to a journal, institution, repository, or client under its current instructions.

Supplementary information

Material distributed alongside a main work, governed by the recipient's format, numbering, review, and access rules.

TeX distribution

A maintained collection of engines, formats, packages, fonts, documentation,

and utilities.

Toolchain

The ordered set of programs and configurations that transforms inputs into outputs.

Underfull box

A box whose content cannot be spaced to the desired dimension without looseness; it requires visual judgement.

Vector graphic

An image represented by paths and objects, normally scalable for line art, diagrams, plots, and text.

Version control

A system that records identified snapshots and changes, allowing comparison, collaboration, and recovery.

Warning

A diagnostic condition that permits output but requires judgement, additional passes, or visual inspection.

Widow

A typographic term often used for the final line of a paragraph isolated at the top of a page; definitions vary across style guides.

Working directory

The directory from which a process resolves relative paths. The project should document its expected main build directory.

Concise exercise solutions

These concise solutions support checking after an honest attempt. Open-ended responses can differ while still showing sound reasoning.

Chapter 0

- 0.1 A source file is editable plain text containing content and $\text{\LaTeX}$ instructions. A compiler interprets that source. An output file is the resulting typeset document, commonly a PDF.
- 0.2 Editing the PDF does not normally edit `main.tex`. The next build from unchanged source can recreate the old value. Edit the source, compile, and inspect the new PDF.
- 0.3 Only the line `Hello,  $\text{\LaTeX}$ !` appears as ordinary document text.
- 0.4 Any two defensible examples, such as cross-references, many equations, a large bibliography, repeated chapter structure, or reproducible builds.
- 0.5 Results paragraph: content. `book`: class. Bibliography toolbox: package. `report.pdf`: output.
- 0.6 Free positioning and visual collaboration may be more direct in a graphics editor; choosing $\text{\LaTeX}$ would add source-editing work without an evident structural benefit.
- 0.7 Answers vary. A good answer names concrete structures such as chapters, methods, numbered equations, figures, tables, citations, appendices, and declarations.

Chapter 1

- 1.1 The correct source contains a class declaration, matching document environment, and body text. A direct comparison should reveal missing backslashes, braces, or lines.
- 1.2 `article` is the class that supplies broad document rules; `document` is the environment enclosing visible content.
- 1.3 Two ordinary lines without a blank line form one paragraph and the source line ending acts like a space.
- 1.4 Answers vary. Only the body-text line needs to change.
- 1.5 Add the backslash: `\begin{document}`.

- 1.6 A compiler checks whether it can interpret the source. It cannot validate scientific claims, complete reasoning, figure accessibility, or compliance with an external submission guide.
- 1.7 Results may differ slightly because of engine, class, font, or TeX distribution versions. Record the environment before diagnosing.

Chapter 2

- 2.1 Command: `\documentclass`; optional argument: `[12pt]`; required argument: `{book}`.
- 2.2 Their roles are defined by the command. Braces supply the required class name; square brackets supply an optional class choice.
- 2.3 Five lines with no blank line normally form one paragraph.
- 2.4 One valid source sentence is `Accuracy was 95\% for sample\_7`.
- 2.5 Replace `\end{article}` with `\end{document}`.
- 2.6 The broken body contains `Method A & Method B`; the correction is `Method A \& Method B`. Include the class and document environment in both versions.
- 2.7 Changing five lines weakens the test because several possible causes change together. A one-change experiment gives clearer evidence.
- 2.8 Answers vary. A successful MWE remains complete and preserves the same symptom after unrelated content and packages are removed.

Chapter 3

- 3.1 A source line ending normally becomes interword space; a blank line ends one paragraph and begins another.
- 3.2 Remove the forced breaks and leave ordinary source lines inside one paragraph; use a blank line only at the intended paragraph boundary.
- 3.3 Use the correct two grave accents to open and two apostrophes to close the quoted word, as shown in Chapter 3.
- 3.4 Use an en dash for pages 4–9, a hyphen for evidence-based, and an em dash or suitable punctuation for the interruption.
- 3.5 A brief explanatory aside can suit a footnote; an essential method or result should remain in the main argument.
- 3.6 Write `\textit{Escherichia coli}`; after definition, a discipline-appropriate *E. coli* may be used.

3.7 A literal percentage is `50\%`; an unescaped percent begins a comment.

3.8 Answers vary; credit choices tied to meaning rather than manual visual decoration.

Chapter 4

4.1 `\title` stores metadata; `\maketitle` produces the title block at its location.

4.2 Add, for example, `\date{1 August 2026}` in the preamble.

4.3 Replace a manual bold line with the appropriate `\section` or lower structural command.

4.4 All later automatically numbered sections increase by one; references update after compilation.

4.5 Keyword placement and syntax are defined by the target class or venue, not by the generic article class.

4.6 Place `\appendix` once, then use two normal section or chapter commands.

4.7 Move the abstract environment after `\begin{document}`, normally after `\maketitle`.

4.8 Answers vary; a good hierarchy has no skipped levels and every heading represents a genuine division of the argument.

Chapter 5

5.1 Materials normally use `itemize`, an ordered protocol `enumerate`, and labelled terms `description`.

5.2 Put the lines between `\begin{enumerate}` and its matching end, beginning each with `\item`; delete typed numbers.

5.3 Nest a second list inside the first `\item`, keeping environments properly paired.

5.4 Add `\item` before the first entry.

5.5 `quote` suits a short quotation; `quotation` supports multiple paragraphs and paragraph indentation.

5.6 Answers vary; parallel verbs should describe actions at the same grammatical level.

5.7 A source comment can record a private drafting reason without entering the visible document.

5.8 Credit a redesign that identifies the reader task and reduces cognitive load without hiding necessary evidence.

Chapter 6

6.1 `$E=mc^2$`.

6.2 Use `x_{ij}`.

6.3 `\frac{x-\mu}{\sigma}`.

6.4 Use `\sin`, for example `\sin(x)`.

6.5 Put two rows in `align` and place `&` before each equals sign.

6.6 Put `\label{eq:key}` inside the numbered equation and refer with `\eqref{eq:key}` or `\cref`.

6.7 The double-dollar form bypasses L^AT_EX display conventions; `eqnarray` has inferior spacing and alignment compared with `amsmath`.

6.8 Answers vary; require defined symbols, correct notation, and colleague feedback about scientific meaning.

Chapter 7

7.1 Use `equation` containing `aligned`, with line breaks inside.

7.2 Use `cases`; put conditions in `\text{...}`.

7.3 Use `pmatrix` and `bmatrix`, with ampersands between columns.

7.4 A two-column matrix row needs one ampersand; remove the extra separator.

7.5 Example: `\int_a^b f(x)\, \mathrm{d}x`.

7.6 Give `definition` a counter and declare lemma and theorem to share it.

7.7 Section-based numbering helps long technical works; it can clutter a short paper whose numbering is already unambiguous.

7.8 Answers vary; every inference and notational convention must be mathematically and typographically reviewable.

Chapter 8

8.1 Use `\num{1250000}` and `\qty{2.5e-6}{\metre}`.

8.2 Use `\qtyrange{4}{9}{\milli\gram}`.

8.3 With configured separate uncertainty, use `\qty{7.2 +- 0.4}{\centi\metre}`.

8.4 A suitable declaration is `S[table-format=-2.2]`.

- 8.5 Decimal places are formatting; measurement process, uncertainty, calibration, and analysis establish defensible precision.
- 8.6 Use `\qty{20}{\degreeCelsius}` and `\qty{95}{\percent}`.
- 8.7 `\ce` typesets chemical formulae and equations and is supplied by `mhchem`.
- 8.8 Credit an audit that checks units, definitions, rounding, uncertainty, missingness, and the underlying evidence.

Chapter 9

- 9.1 Declare two columns in `tabular`; each row has one ampersand and ends with a row terminator.
- 9.2 Remove the separator that creates a third cell, or intentionally add and define a third column.
- 9.3 Load `booktabs`, use top/mid/bottom rules, and remove vertical bars from the column specification.
- 9.4 Use `tabularx` at `\textwidth` with an `X` description column.
- 9.5 Use an `S` column whose table format includes a sign and enough digits.
- 9.6 The caption and note must say values are simulated, define units and abbreviations, and identify the reader task.
- 9.7 `longtable` performs its own multi-page output and is not an ordinary float placed inside `table`.
- 9.8 Answers vary; require a named task, preserved evidence, and an omission log.

Chapter 10

- 10.1 Photograph and scanned map are primarily raster; logo and line plot are preferably vector when original paths are available.
- 10.2 Use `\includegraphics[width=.5\linewidth]{figure.pdf}`.
- 10.3 Put `\label` immediately after `\caption` inside `figure`.
- 10.4 The option is a placement preference; float size, order, and page constraints may prevent placement at the source location.
- 10.5 Use two `subfigure` environments with panel captions and labels, followed by one overall caption and label.
- 10.6 Use distinguishable shapes/line styles and direct labels or patterns; retain adequate luminance contrast.

- 10.7 Replace the personal absolute path with the actual project-relative path and include the file.
- 10.8 Credit audits covering data fidelity, effective resolution, readable scale, non-colour meaning, caption, alt-description need, source, and rights.

Chapter 11

- 11.1 Use distinct semantic keys such as `sec:methods`, `eq:model`, `tab:summary`, and `fig:growth`.
- 11.2 Use `\eqref{eq:model}` and `\pageref{eq:model}`.
- 11.3 Pass both comma-separated keys to one `\cref`; compatible targets are pluralised automatically.
- 11.4 Move the label after the caption so it records the figure counter.
- 11.5 Match key case exactly in both `\label` and the reference.
- 11.6 Duplicate labels can silently make several references resolve to the same, wrong target.
- 11.7 Compare `hyperref` title and author fields with the approved visible metadata and anonymity state.
- 11.8 Replace typed numbers with labels and semantic reference commands, then insert an earlier object to test stability.

Chapter 12

- 12.1 The key identifies a database entry; the citation is the in-text marker; a reference is one formatted record; the bibliography is their collection.
- 12.2 Answers vary; compare all fields with the article's authoritative publisher record and preserve the DOI exactly.
- 12.3 Use `\textcite{key}` and `\parencite{key}`.
- 12.4 Use one citation command with both keys and a documented postnote, for example `\parencite[p. 12]{key1,key2}`.
- 12.5 Add `\printbibliography` at the intended location, then run the full backend sequence.
- 12.6 A DOI identifies an object; it does not show that the object was read or that it supports the nearby claim.
- 12.7 Retain the publisher workflow when the official class and current instructions require it.

12.8 Credit a record-by-record audit with authoritative URLs, dates, and documented corrections.

Chapter 13

- 13.1 Use `article`, `report`, and `book`, respectively, subject to any official specialist class.
- 13.2 In `[11pt,a4paper,twoside,openright]`, the options request base size, paper size, two-sided layout, and right-hand chapter openings.
- 13.3 Run `texdoc package-name`; success opens the installed manual.
- 13.4 Use `geometry` with explicit `inner` and `outer` values in a two-sided generic document.
- 13.5 Define the `fancyhdr` fields and separately redefine or select the `plain` style for chapter openings.
- 13.6 Find the first package load in the log and arrange one compatible option source rather than loading the package twice.
- 13.7 The publisher class encodes production constraints and can conflict with local packages; visual imitation is not compliance.
- 13.8 Credit each retained package only when a used command or verified requirement demonstrates its purpose.

Chapter 14

- 14.1 Use `\input` for the table and preamble; use `\include` for a page-starting thesis chapter.
- 14.2 A sound tree has one root driver, metadata/preamble, five chapter files, front matter, appendices, figures, bibliography, and README.
- 14.3 Add `\listoffigures` and `\listoftables`; later passes read caption entries written during earlier passes.
- 14.4 Call `\appendix` once and include two normally titled chapter files.
- 14.5 Example: `\newcommand{\MeanDifference} {\ensuremath{\bar d}}`.
- 14.6 Use `figures/result.pdf`, preserving the project structure.
- 14.7 A clean build proves the source contains its declared dependencies instead of relying on stale or private state.
- 14.8 Split at stable units, keep one main preamble, preserve labels, and compare clean PDFs and logs.

Chapter 15

- 15.1 Answers vary; each row must cite the official requirement, destination file, implementation, and verification evidence.
- 15.2 Declaration text can carry legal and academic meaning and must come from the authorised institution, not creative paraphrase.
- 15.3 Example entries: vector $\mathbf{v}$, scalar T , and unit $\mathbf{K}$, each with an explicit definition.
- 15.4 Call `\appendix`, include the new file, label it, and refer using `\cref`; do not type its letter.
- 15.5 `\includeonly` relies on stored auxiliary data; a later complete clean build removes stale numbering and validates every chapter.
- 15.6 A valid fictional statement distinguishes conception, analysis, writing, review, and accountability by named initials.
- 15.7 Record data and code inputs, environment, command, figure source, output, version, creator, licence, and verification.
- 15.8 Credit matrices that separate automated dimensions/log checks from manual wording, scientific, permission, and visual reviews.

Chapter 16

- 16.1 Each row needs the exact instruction, authoritative source, manuscript implementation, responsible person, and verification evidence.
- 16.2 Replace causal or general claims with the measured relationship, design, population, and simulated status actually supported.
- 16.3 Include problem, essential method, principal numeric result, and bounded conclusion within the verified word limit.
- 16.4 Possible leaks include PDF properties, filename, acknowledgements, self-identifying repository links, self-citation wording, comments, tracked changes, and embedded figure metadata.
- 16.5 Statements must use explicit fictional facts and venue-appropriate absence wording; they cannot be inferred from the result.
- 16.6 A table provides precise values while a figure reveals a pattern, distribution, or relationship not duplicated by reading cells.
- 16.7 Include only the main source, section files, required class/styles, bibliography, figures, supplements, README, and manifest.
- 16.8 Credit a migration log that starts from the publisher sample, records class interfaces, and verifies every resulting difference.

Chapter 17

- 17.1 Example: This report evaluates calibrated assay comparisons so the laboratory director can decide whether to authorise validation by the review date, excluding routine deployment.
- 17.2 Supply the decision, finding with uncertainty, recommendation, principal risk, and owned next step.
- 17.3 Observations report evidence; interpretations assign meaning; recommendations propose action; decisions record authorised choice.
- 17.4 Define, for example, low, moderate, and high using explicit consequence and likelihood descriptions rather than arbitrary scores.
- 17.5 Name the action, owner, due date or condition, and objective completion evidence.
- 17.6 Use dated rows for initial draft, reviewed draft, and approved issue, recording change and responsible roles.
- 17.7 Give each data, code, figure, and approval object an ID, version, location, owner, and access condition.
- 17.8 Compare cover, headers, filename, control table, metadata, repository, and distribution notice for one consistent state.

Chapter 18

- 18.1 Answers vary; prefer descriptive lowercase portable names with one separator convention and no confidential identifiers.
- 18.2 Synchronisation mirrors current state, version history records changes, and a backup is an independent recoverable copy.
- 18.3 The README must state purpose, simulated/real status, main file, engine, bibliography tool, build command, dependencies, and output.
- 18.4 Replace user-root locations with paths such as `figures/plot.pdf` relative to the documented build root.
- 18.5 Snapshots suit small milestones; change markup supports visible review; Overleaf supports shared editing; Git gives detailed distributed history.
- 18.6 Record creator, source, licence, permission, required credit, allowed modification, redistribution, and access restrictions.
- 18.7 Copy only manifested files to a fresh directory, follow the README, regenerate analysis and PDF, and retain log plus visual-check evidence.

- 18.8 Classify exact files by access and provenance; generated assets name their source, while rebuildable auxiliaries are excluded.

Chapter 19

- 19.1 Later messages can be consequences of the first structural failure; the first meaningful error usually has the least corrupted context.
- 19.2 Close the missing group, restore matching math delimiters, and make begin/end names and nesting identical.
- 19.3 Retain only the class, `graphicx`, document environment, and one failing `\includegraphics` line with the same asset/path.
- 19.4 Check key, database registration, backend execution, and later L^AT_EX passes in that order.
- 19.5 Legitimate: allow a break or redesign a wide object. Poor: hide all warnings globally or shrink the entire document indiscriminately.
- 19.6 Clean when logs or a major tool change indicate stale incompatible generated state, not as a substitute for source diagnosis.
- 19.7 Include an anonymised MWE, exact command, first error, expected/actual result, versions, and clean-build status.
- 19.8 With halving, a single planted error in n roughly equal-sized lines should need about $\lceil \log_2 n \rceil$ isolation decisions, plus builds that confirm repair.

Chapter 20

- 20.1 Each of ten rows needs instruction, source/version, implementation, owner, status, and evidence.
- 20.2 Record result name, authoritative output, value, unit, rounding, and every occurrence in abstract, table, prose, figure, and discussion.
- 20.3 Effective pixels per inch equal pixels across the dimension divided by placed inches; compute each dimension and compare with current rules.
- 20.4 Compare author, title, date, venue, pages, version, and DOI with the publisher or other authoritative record and confirm claim support.
- 20.5 Source-only examples include hidden comments; PDF-only examples include clipping; portal-only examples include conversion order or metadata errors.
- 20.6 Include required source, assets, bibliography, permitted styles, supplements, README, and manifest; exclude caches, drafts, secrets, and restricted or unlicensed material.

- 20.7 Record thesis identifier, version, source commit, environment, files, checksums if used, approvals, access, embargo, date, and repository ID.
- 20.8 Credit a rehearsal with separate operator/verifier roles and retained evidence for build, PDF, metadata, package, proof, and archive checks.

Chapter 21

- 21.1 Articles serve bounded journal claims, theses sustained degree arguments, and reports named decisions; scenario answers must use audience and authority.
- 21.2 Shared components include source, evidence, figures, references, and builds; form-specific components include journal declarations, thesis front matter, and report control/action records.
- 21.3 Answers vary; explanations must identify each command's structural or build effect, not merely its visible appearance.
- 21.4 The article has compact sequential pages, the thesis distinct matter and chapter numbering, and the report controlled cover and decision sections; all use Biber through independent builds.
- 21.5 Replace only facts that are supplied or explicitly fictional; leave legal, institutional, authorship, and approval data as visible placeholders.
- 21.6 Each manifest names exact source, bibliography, figures, supplements, README, output, versions, rights, and excluded restricted dependencies.
- 21.7 Use the article, thesis, or report checklist and record each item as pass, fail, or not applicable with evidence.
- 21.8 Credit two outputs that share one versioned analysis but maintain form-specific claims, metadata, permissions, approvals, and release archives.

Further reading and verified documentation

Begin with maintained, authoritative documentation: the L^AT_EX Project for the core system, CTAN for package records and manuals, and Overleaf for its current service. Access dates show when web instructions were checked.

- Chiroma, H. and J. H. Abawjy (2023). ‘Systematic Literature Review from a Computing Perspective and Research Problem Formulation’. In: *Computing Research Survival Manual: A Practical Handbook for Beginners*. IOP Publishing, pp. 3-1–3-25. DOI: 10.1088/978-0-7503-5017-4ch3.
- Collins, J. (2026). *latexmk: Fully Automated L^AT_EX Document Generation*. URL: <https://ctan.org/pkg/latexmk> (visited on 01/08/2026).
- Fisher, R. A. (1936). *Iris*. UCI Machine Learning Repository. DOI: 10.24432/C56C76.
- Heffernan, K. (1st Oct. 2020). ‘Problem-Solving Recognition in Scientific Text’. PhD thesis. University of Cambridge. DOI: 10.17863/CAM.78657.
- International Organization for Standardization (July 2023). *Accuracy (Trueness and Precision) of Measurement Methods and Results – Part 1: General Principles and Definitions*. International Standard ISO 5725-1:2023. International Organization for Standardization. URL: <https://www.iso.org/standard/69418.html> (visited on 01/08/2026).
- Kime, P., M. Wemheuer and P. Lehman (10th July 2025). *The biblalex Package: Programmable Bibliographies and Citations*. Version 3.21. URL: <https://ctan.org/pkg/biblalex> (visited on 01/08/2026).
- Knuth, D. E. (1984). ‘Literate Programming’. In: *The Computer Journal* 27.2, pp. 97–111. DOI: 10.1093/comjnl/27.2.97.
- Lamport, L. (1994). *L^AT_EX: A Document Preparation System*. 2nd ed. Addison-Wesley. ISBN: 9780201529838.
- Overleaf (2026a). *Downloading a Project*. URL: <https://docs.overleaf.com/managing-projects-and-files/downloading-a-project> (visited on 01/08/2026).
- (2026b). *Recompiling Your Project*. URL: <https://docs.overleaf.com/getting-started/recompiling-your-project> (visited on 01/08/2026).
- (2026c). *The Main Document*. URL: <https://docs.overleaf.com/getting-started/recompiling-your-project/the-main-document> (visited on 01/08/2026).
- (2026d). *Your First Project*. URL: <https://docs.overleaf.com/getting-started/your-first-project> (visited on 01/08/2026).
- R Core Team (2026). *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. URL: <https://www.R-project.org/> (visited on 01/08/2026).
- The L^AT_EX Project (2026). *L^AT_EX Documentation*. URL: <https://www.latex-project.org/help/documentation/> (visited on 01/08/2026).

Index

- L^AT_EX project, 10
- L^AT_EX
 - advantages, 2
- abstract, 38
- accessibility, 171
- align environment, 52
- anonymous review, 132
- appendix, 38
- argument, 20
 - optional, 19
 - required, 19
- automatic numbering, 88
- auxiliary files, 13, 161
- backup, 152
- BibL^AT_EX, 96
- Biber, 96
- bibliography, 96
- bibliography database, 96
- booktabs package, 74
- calculus, 59
- capstone project, 182
- caption, 81
- citation, 96
- citation key, 96
- citation style, 96
- cleveref package, 88
- collaboration, 152
- command, 19
- commands, 4, 19
- comment, 22
- compilation, 3, 10
- compiler, ii, 3
- cross-reference, 88
- decimal alignment, 74
- declaration, 132
- dissertation, 122
- document body, 13, 24
- document class, 4, 106
- document control, 141
- document preparation system, 2
- document structure, 38
- DOI, 96
- emphasis, 32
- environment, 12, 19, 21
- equation, 52
- error message, 161
- executive summary, 141
- figure, 81
- first document, 10
- float, 75
- font embedding, 171
- footnotes, 32
- front matter, 122
- geometry package, 106
- Git, 152
- graphics
 - raster, 81
 - vector, 81
- header, 106
- hyperref package, 88
- include command, 113
- index, 113
- input command, 113
- institutional requirements, 122
- journal manuscript, 132
- label, 88
- list
 - bulleted, 45
 - description, 45
 - numbered, 45
- log, 6
- long document, 113
- main document, 11

- manifest, 171
- math mode, 52
- mathematics, 52
 - advanced, 59
- matrix, 59
- metadata, 171
- minimal working example, 25, 161, 162
- multi-file project, 113
- MWE, 25

- nomenclature, 115
- non-breaking space, 34

- optional arguments, 20
- Overleaf, 10

- package, ii, 4, 106
- page design, 106
- preamble, ii, 13, 19, 24

- quantities, 66
- quotation, 45
- quotation marks, 32

- raster image, 81
- README file, 152
- recommendation, 141
- reference, 96
- reproducibility, 152
- required argument, 20
- reserved characters, 23

- siunitx package, 66
- source-to-PDF workflow, 2
- special characters, 32
- subfigure, 81
- submission, 171
- supplementary material, 132

- table, 74
- tabularx package, 74
- technical report, 141
- template
 - article, 182
 - technical report, 182
 - thesis, 182
- terminal, 13
- TeX distribution, 10, 11
- theorem, 59
- thesis, 122
- title metadata, 38
- troubleshooting, 161
- typography, 32

- uncertainty, 66
- units, 66

- vector graphic, 81
- version control, 152

- WYSIWYM, 2

About the author

Krishnendu Sinha, Ph.D., is an Associate Professor of Zoology at Siksha Bhavana, Visva-Bharati, Santiniketan, West Bengal, India. He received his Ph.D. in Molecular Medicine from Bose Institute, Kolkata, after completing an M.Sc. at the University of Calcutta and a B.Sc. at Presidency College. Before joining Visva-Bharati in 2026, he served in the West Bengal Education Service at Jhargram Raj College.

His research combines molecular evolution with computational methods. His interests include sequence analysis, adaptive molecular evolution, phylogenetics, protein evolution, bioinformatics, and reproducible workflows. He directs the Sinha Molecular Evolution Group and develops resources including BABAPPAlign and BABAPPASnake.

Current profile: https://sinhakrishnendu.github.io/Lab_website/

ORCID: 0000-0001-9590-3875

